Latvijas Universitāte

Faculty of Computer Science

Course code and title: DatZ2072: Software Engineering

eHouse

Scientific advisor: Mg. sc. comp. J. Iljins

Done by:

Mg. oec. Alexey Leontyev (al19087)

Anastasija Zujeva (az19108)

Indu Tharanga Samarasinghe Pathirana (is19021)

David Shuvalov (ds19083)

Ilja Kozirevs (ik17255)

Elbek Tursunov (et19042)

Rīga 2020

Abstract

Project work describes “eHouse” web application for “managing company – inhabitant” interaction, allowing to access the functionality and services, provided by the managing company to inhabitants via web-application. The document has a general description of the application, database models and design, main modules (User Management, Apartment Management, Inhabitant Management, Data Submission, Billing and Notification) description, functions and requirements, non-functional requirements and user interface concepts. The document is developed according to the ISO/IEC/IEEE 29148:2018 standard.

Keywords: house management system, management company, inhabitant, apartment.

Anotācija

Projekta darbs apraksta “eHouse” tīmekļa lietojumprogrammu “uzņēmums – iedzīvotājs” mijiedarbībai, ļaujot piekļūt funkcionālitātei un pakalpojumiem, ko apsaimniekošanas uzņēmums sniedz iedzīvotājiem, izmantojot tīmekļa lietojumprogrammu. Dokumentā ir vispārējais programmas apraksts, datu bāzes modeļi un dizains, galveno moduļu (Lietotāju pārvaldība, Dzīvokļu pārvaldība, Iedzīvotāju pārvaldība, Datu iesniegšana, Rēķini un Paziņošanas) apraksts, funkcijas un prasības, nefunkcionālās prasības un lietotāja saskarnes koncepcijas. Dokuments ir izstrādāts saskaņā ar ISO/IEC/IEEE 29148:2018 standartu.

Atslēgas vārdi: māju apsaimniekošanas sistēma, vadības uzņēmums, iedzīvotājs, dzīvoklis.

Table of contents

List of figures 5
Glossary 7
Introduction 8
1. Overall description 10
1.1. AS IS 10
1.2. Client 10
1.3. Product vision 10
1.4. Business requirements 11
1.5. System Users 11
1.6. General restrictions 12
1.7. Assumptions and dependencies 12
2. Software requirements specification 13
2.1. Conceptual Database Model 13
2.2. Functional Requirements 14
2.2.1. General description of functions 14
2.2.2. Functional breakdown by modules 17
2.2.3. User Management module 20
2.2.4. Apartment management module 24
2.2.5. Inhabitants management module 27
2.2.6. Data Submission module 31
2.2.7. Billing Module 36
2.2.8. Notifications module 48
2.3. Non-functional requirements 55
3. Software design description 56
3.1. Database design 56
3.1.1. Users and corresponding entities 59
3.1.2. Inhabitants and corresponding entities 60
3.1.3. Apartments and corresponding entities 62
3.1.4. Meters and corresponding entities 63
3.1.5. Bills, Fines and corresponding entities 65
3.1.6. Notifications and corresponding entities 68
3.2. Partial Design of User Interfaces 70
3.3. Partial Functional Design 98
References and sources 107
Appendix 108

List of figures

Number	Title	Page
1	Data flow diagram (level 0)	12
2	Conceptual database model (Chen syntax)	13
3	Modules of the system	15
4	Data flow diagram (level 1)	19
5	User management module – data flow diagram (level 2)	20
6	Apartment management module – data flow diagram (level 2)	24
7	Inhabitants management module – data flow diagram (level 2)	27
8	Data submission module – data flow diagram (level 2)	31
9	Billing module – data flow diagram (level 2) part I	36
10	Billing module – data flow diagram (level 2) part II	43
11	Notifications module – data flow diagram (level 2)	48
12	Logical database model (crowfoot syntax)	57
13	Physical database model	58
14	Main page (Guest)	70
15	Registration form	71
16	Login form	71
17	Error message for Blocked user role	72
18	Main page (Inhabitant)	73
19	Apartment’s data	74
20	Bills	75
21	Fines	76
22	Meters	77
23	Meters data submission	77
24	Main page (Privileged user)	78
25	Main page (Admin)	79
26	Announcements (Privileged user)	80
27	Announcements (Admin)	80
28	New announcement	81
29	Announcement edit	82
30	User list (Admin)	82
31	User role change (Admin)	83

32	Linking inhabitant to user account (Admin)	84
33	List of all apartments (Admin)	84
34	Edit apartment’s data (Admin)	85
35	List of all inhabitants (Admin)	86
36	Linking inhabitant to property (Admin)	86
37	New inhabitant (Admin)	87
38	Fines (Admin)	88
39	New fine (Admin)	88
40	Edit fine (Admin)	89
41	Change fine status (Admin)	90
42	Bills (Admin)	91
43	New bill (Admin)	91
44	Edit bill (Admin)	92
45	Change bill status (Admin)	93
46	Meters (Admin)	94
47	Edit meter’s data (Admin)	95
48	User interface diagram (overall)	96
49	User interface diagram (inhabitant route)	96
50	User interface diagram (privileged user route)	97
51	User interface diagram (admin route)	97
52	Billing process (UML activity diagram)	99
53	Bill statuses (UML state transition diagram)	100
54	Configuring working account (UML activity diagram)	101
55	Bill controller – MVC approach (UML class diagram)	102
56	Main parts of the system (UML package diagram)	103
57	Log in process (UML sequence diagram)	104
58	Register process (UML sequence diagram)	105
59	Fines functions (UML use case diagram)	106

Glossary

Acronym	Definition
CSRF	Cross-site request forgery – a type of web attacks
CSS	Cascading Style Sheets – a style sheet language used for describing the presentation of a document written in a markup language
DBMS	Database management system – a software package designed to define, manipulate, retrieve and manage data in a database
GUI	Graphical User Interface – a user interface that includes graphical elements, such as windows, icons and buttons, etc.
HTLM	Hypertext Markup Language – the standard markup language for documents designed to be displayed in a web browser
IEC	International Electrotechnical Commission – an international standards organization that prepares and publishes international standards for all electrical, electronic and related technologies
IEEE	Institute of Electrical and Electronics Engineers – a professional association for electronic engineering and electrical engineering
ISO	International Organization for Standardization – an international standard-setting body composed of representatives from various national standards organizations
MVC	Model–view–controller – a software design pattern
PDF	Portable Document Format (PDF) is a file format developed by Adobe in 1993 to present documents
SQL	Structured Query Language – a domain-specific language used in programming and designed for managing data held in a relational database management system
UML	Unified Modeling Language – a general-purpose, developmental, modeling language in the field of software engineering that is intended to provide a standard way to visualize the design of a system
W3C	World Wide Web Consortium – the main international standards organization for the World Wide Web
XSS	Cross-site scripting – a type of security vulnerability typically found in web applications

Introduction

Purpose

The document describes “eHouse” web application including a general vision of the application, its users and clients, software requirements specification, restrictions as well as software design specification with partial user interface mockup. This document is a mutual agreement between the customer and the contractor.

Scope

The “eHouse” web application provides comfortable access to managing company services to inhabitants, who resides in an apartment house in order to reduce unsystematic errors rate and low expenses on interaction with inhabitants for managing company to be more competitive on the market and making access and interactions with managing company easier for inhabitants allowing to have all mandatory and optional services in one application. The application can be used by one managing company to manage apartment house including managing apartments and inhabitants, handling data submissions, billing as well as providing notifications. The payment process is not integrated and is out-of-the-scope for the system.

Links with the other documents

The document is based on the current version of the standard "Systems and software engineering – Life cycle processes – Requirements engineering" ISO/IEC/IEEE 29148:2018 [2].

Overview

The document consists of 3 chapters, list of pictures, definitions, acronyms, and abbreviations, references and sources and appendices.

- Chapter 1 “Overall description” showing the clients, product vision of the developers, business requirements, system users, explaining their roles and access to the web application as well as general restrictions and assumptions and dependencies.
- Chapter 2 “Software requirements specification” contains conceptual database model, both functional and non-functional requirements. In this section also described requirement for modules of the system (User Management module, Apartment Management module, Data Submission module, Billing module and Notification module) containing data flow diagrams and function description.

- Chapter 3 “Software design description” describes logical and physical database models, most crucial functional design and provides a mockup of the important parts of the user interface.
- Section “List of figures” contains the list of all pictures used in the document, their names, numbers and pages.
- Section “Glossary” contains explanation of used acronyms.
- Appendix contains information excluded from the main sections due to being additional.

1. Overall description

Many apartment houses are managed by companies that don't have any e-system for inhabitants, so it leads to troubles with interactions between inhabitants and managing company. Besides this problem absence of such system usually causes various unsystematic errors by the company which is hard to track.

The system is a web application which would allow users to access most of the functionality and services of a management company by web application and raise quality of services provided by the managing company.

1.1. AS IS

There are several similar offers at the market. The biggest downside of all of them is that they’re cloud-based applications (meaning that data will be stored somewhere remotely), providing services by subscription plans and have excessive extra functionality that is used generally.

eHouse application is deployed on the servers of the company who’ll use the application (making the company itself store information by thus making the application sensitive information, such as inhabitants data more secure compared to cloud-based version) without subscription fee (life-long license) and only with most crucial functionality making the application more flexible.

1.2. Client

The system is developed by the group of students as assignment for the course DatZ2072: Software Engineering. The real client is a managing company, providing services for an apartment house. The system is used by one managing company to manage and provide services to one or more apartment houses.

1.3. Product vision

The system is a web application in form of a website, personalized by the needs and inner procedures in the managing company to fit it needs and services. All managing functions and services are in scope of the web-application, so there is no connection with external systems required.

1.4. Business requirements

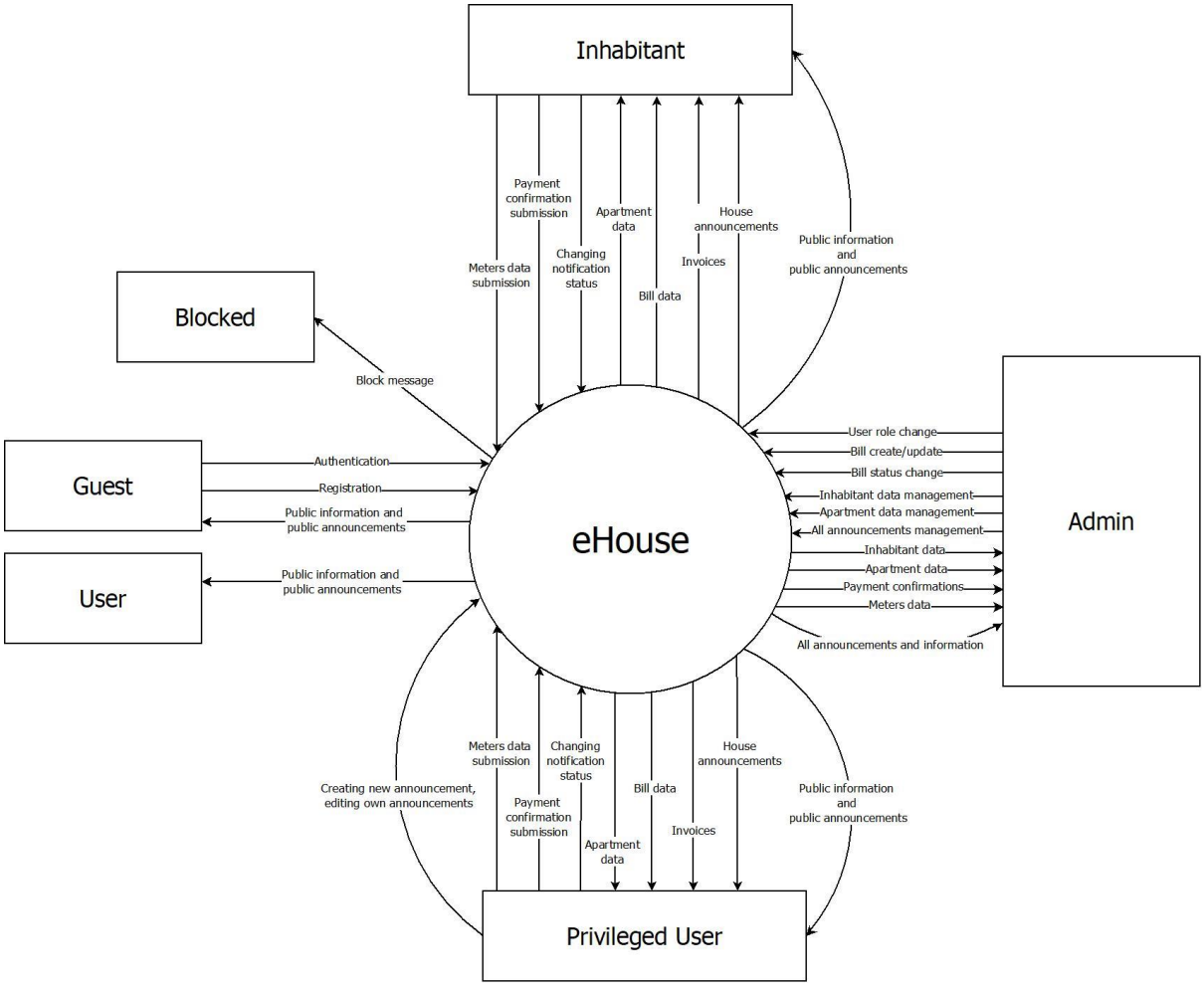
The application is reducing unsystematic errors rate and low expenses on interaction with inhabitants and providing services for managing company, which is making the company more competitive on the market, as the cost of service could be lowered for all inhabitants.

1.5. System Users

There are 6 basic user roles provided by the application, so each user would have one of the following roles:

- Guest – the basic role of non-registered visitors with limitation to see public information, public announcements and possibility of registering and authentication.
- User – registered person, but not confirmed by admin or former inhabitant of the house who no longer owns the apartment. Can only see public information and public announcements.
- Admin – the most privileged user of the application, who is a staff-member from managing company, responsible for all of the activities for company side. Responsible for changing roles of the users, bill creation and management, management of inhabitant and apartment data, as well as announcements managing.
- Inhabitant – basically the person who lives in this house. In general – the owner of the apartment, the one is responsible for payments. Has a functional of submitting meters data and a payment confirmation. Can also change notifications status. Has access to apartment and billing data, invoice generation and both public and house announcements.
- Privileged user – person, who lives in the house, but by the meeting of the owners received extra responsibilities. The functionality is almost identical as for Inhabitant with a feature of creating new announcements and editing them.
- Blocked – the user that after authorization would not be able to access any functionality at all due to various reasons. The only response from the system is a block message.

All data flow between the application and user roles are showed by data flow diagram of 0 level (*pic. 1*).



Pic. 1. Data flow diagram (level 0)

1.6. General restrictions

As the system is web application it's mandatory for end-users to have a device with connection to the internet with installed web browser. Managing company also must have a delegated internet domain as well as server for storing the application data.

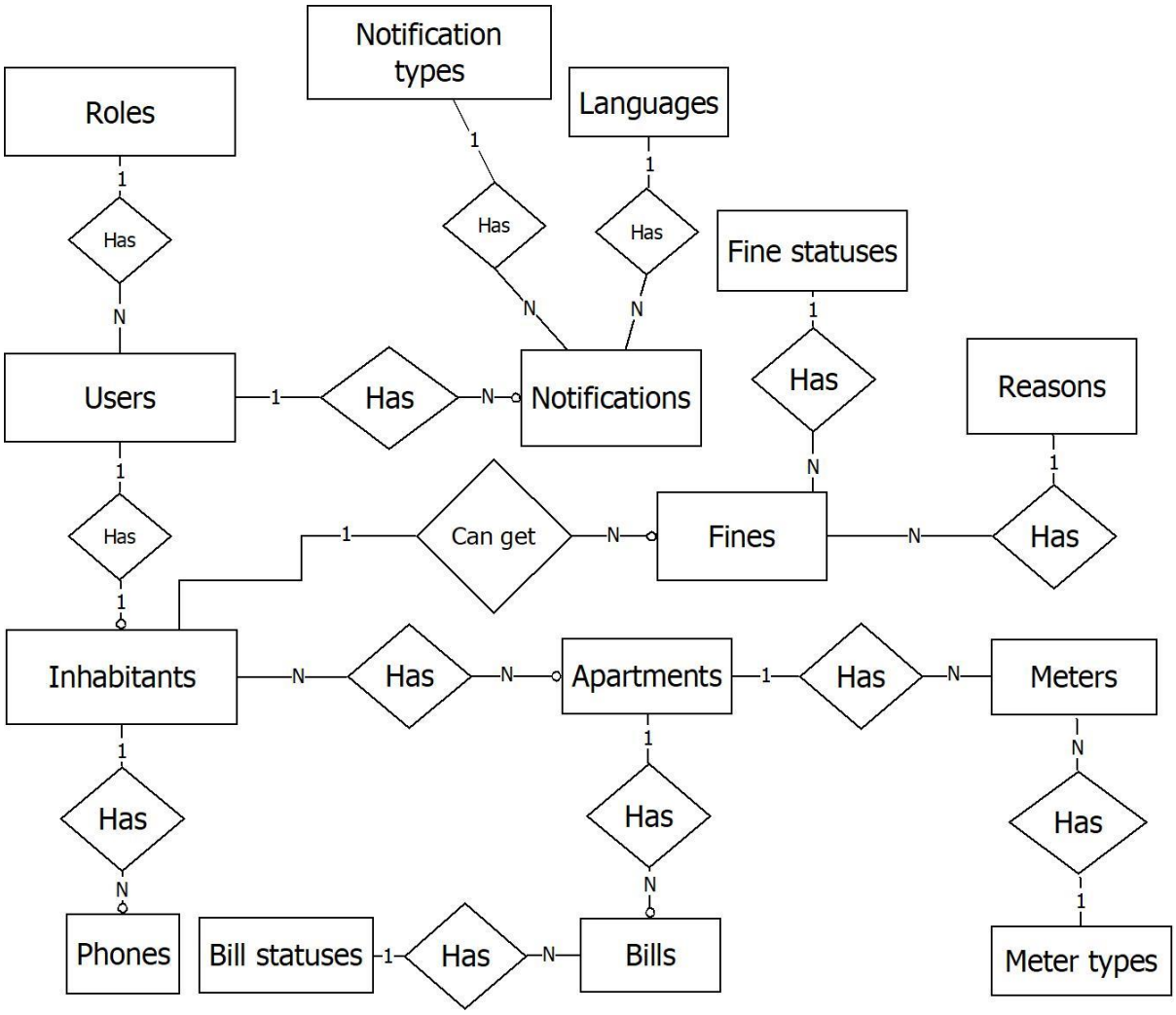
1.7. Assumptions and dependencies

It's assumed that during the installation of the system on a server besides creation of the database it will be filled with basic apartment data, such as apartment number, area in m^2 and apartments meters with their values and classifier database entities will be filled with necessary data. Also, all statuses tables have to be seeded with options. An example of data seeds is shown in the *Appendix*.

2. Software requirements specification

2.1. Conceptual Database Model

The conceptual model of the database shows database entities and their connections without attributes and supplementary conjunction table (*pic. 2*).



Pic. 2. Conceptual database model (Chen syntax)

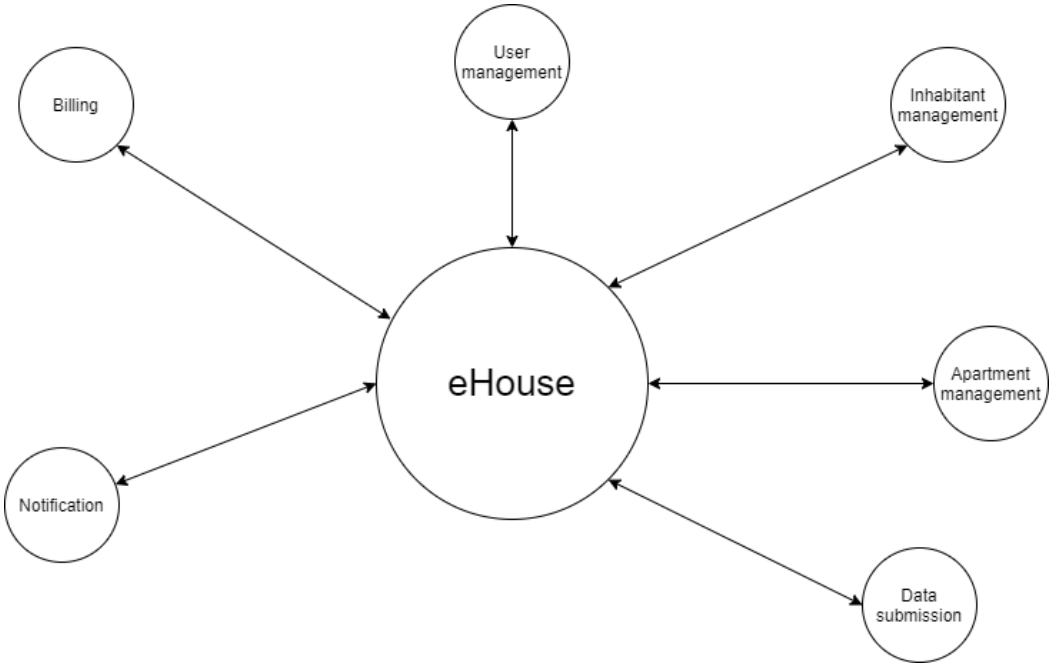
- Entity “Users” has information about user nickname, password, email, user role and email notifications status. Entity “Users” has one-to-one relation with “Inhabitants” entity, meaning that each user account can be connected with at most one inhabitant of the house.
- Entity “Roles” is a codifier entity for “Users” and contains the list of user roles and their descriptions.
- Entity “Notifications” contains information about the messages, users, who created them and language mark.

- Entity “Notification types” is a codifier table containing list of possible notification types.
- Entity “Languages” is codifier table containing list of possible languages that can be used for notifications.
- Entity “Inhabitants” contains inhabitant’s personal data such as full name and personal code. “Inhabitants” entity has N-to-N relation meaning that apartment can be owned by many inhabitants as well as one inhabitant can own multiple apartments by thus requiring a conjunction table to proper representation of such relation.
- Entity “Phones” is explicitly shown as an additional entity as due to the nature of the one-to-N relation it could not be represented as an attribute of “Inhabitant” entity as otherwise it will lead to the denormalization of the database.
- Entity “Apartments” contains all data related to the apartments such as number, area and floor.
- Entity “Bills” contains all bills with its fields for all apartments (as bill is issued per apartment).
- Entity “Bill statuses” is a codifier table, which stores list of possible bill statuses.
- Entity “Fines” contains all issued fines data for inhabitants who got fines (as fine issued per inhabitant).
- Entity “Reasons” is a codifier table, which stores list of possible fine reasons.
- Entity “Fine statuses” is a codifier table, which stores list of possible fine statuses.
- Entity “Meters” has information about meters data and verification dates.
- Entity “Meter types” is a codifier table containing list of meters types, that are used in the house.

2.2. Functional Requirements

2.2.1. General description of functions

The system consists of 6 modules: User Management module, Apartment management module, Inhabitants management module, Data Submission module, Billing module and Notification module. High-level conceptual modules structure is shown on the *pic. 3*. Description of their functions and corresponding user groups, who has access to these functions is described in *table 1*.



Pic. 3. Modules of the system

Table 1. Modules functions and corresponding user groups

Module	Function	User group
User Management module	Registration	Guest
	Authentication	Guest
	View user list	Admin
	Change user role	Admin
	Logout	User, Inhabitant, Privileged user, Admin, Blocked
Apartment management module	View list of all apartments	Admin
	View information about apartment	Inhabitant, Privileged user
	Edit apartment information	Admin
Inhabitants management module	Add new inhabitant	Admin
	View list of all inhabitants	Admin

	Linking inhabitant with property owned	Admin
	Linking inhabitant with user account	Admin
Data Submission module	View meters information	Inhabitant, Privileged user
	View all meters data	Admin
	Submit meters data	Inhabitant, Privileged user
	Upload meters verification	Inhabitant, Privileged user, Admin
	Edit meters data	Admin
Billing Module	View list of all bills	Admin
	Mass bill creation	Admin
	Edit bill	Admin
	Delete bill	Admin
	View bills for apartment	Inhabitant, Privileged user
	Bill Invoice download	Inhabitant, Privileged user
	Bill payment confirmation upload	Inhabitant, Privileged user
	Payment confirmation download	Admin
	Change bill status	Admin
	Create individual fine	Admin
	Update fine	Admin
	Delete fine	Admin
	View list of all fines	Admin

	View fine details	Inhabitant, Privileged user
	Fine Invoice download	Inhabitant, Privileged user
	Change fine status	Admin
Notification module	View public announcements	Guest, User, Inhabitant, Privileged user, Admin
	View house announcements	Inhabitant, Privileged user, Admin
	Create announcement	Privileged user, Admin
	Delete announcement	Admin
	Delete own announcement	Privileged user, Admin
	Edit own announcement	Privileged user, Admin
	Edit all announcement	Admin
	Receive email notifications	Inhabitant, Privileged user, Admin
	Disable/enable email notifications	Inhabitant, Privileged user

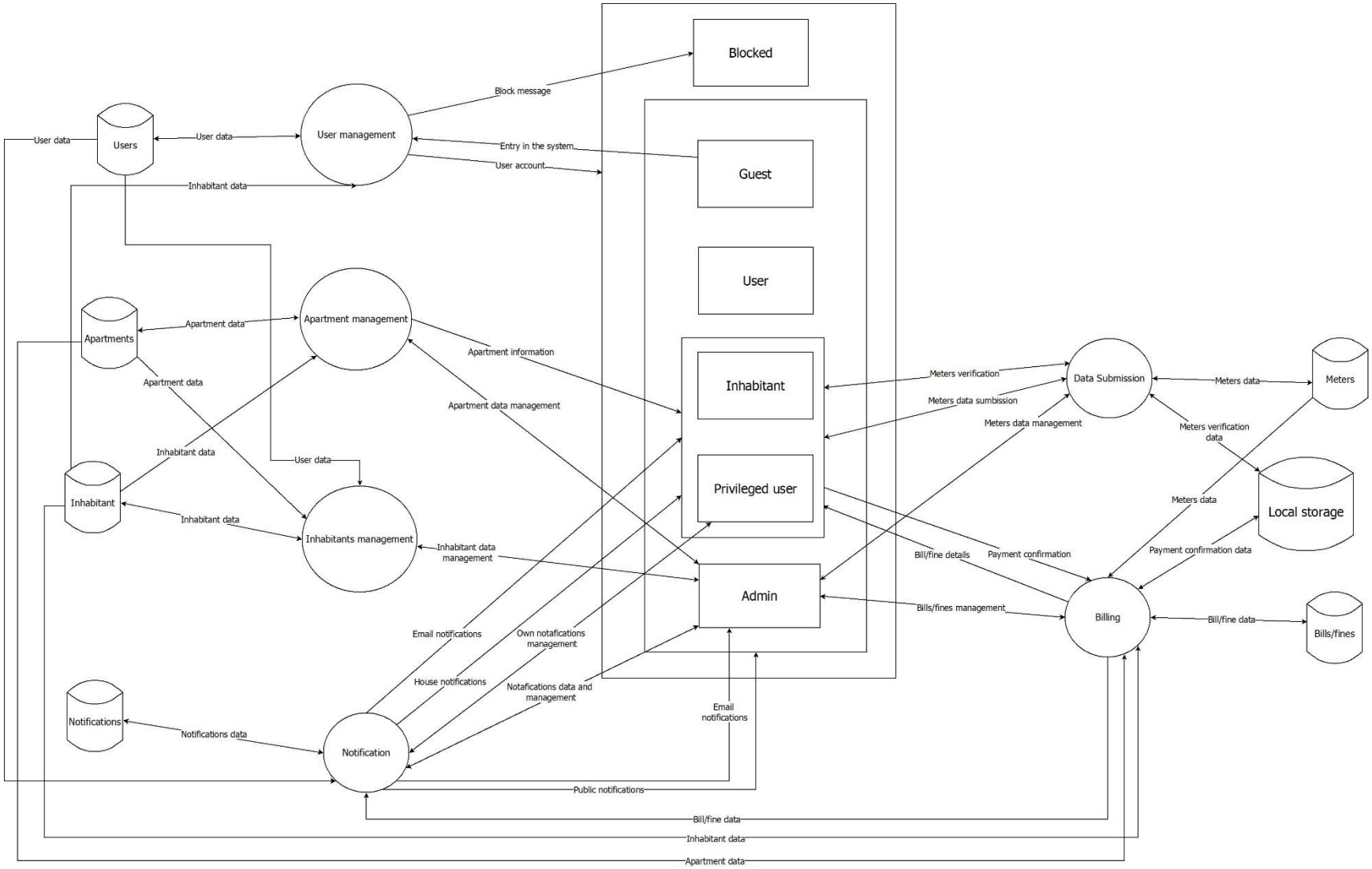
2.2.2. Functional breakdown by modules

The data flow diagram (*pic. 4*), showing functional breakdown is based on the description of the functions in the *table 1* and module division in the *section 2.2.1*.

Several modules are interconnected with each other and require data from another module to be able to proceed request. These modules are:

- User management module requires data about linked inhabitants to reflect data in the user list.
- Inhabitant management module requires data about user and data about apartment for linking inhabitant with user account and property owned.

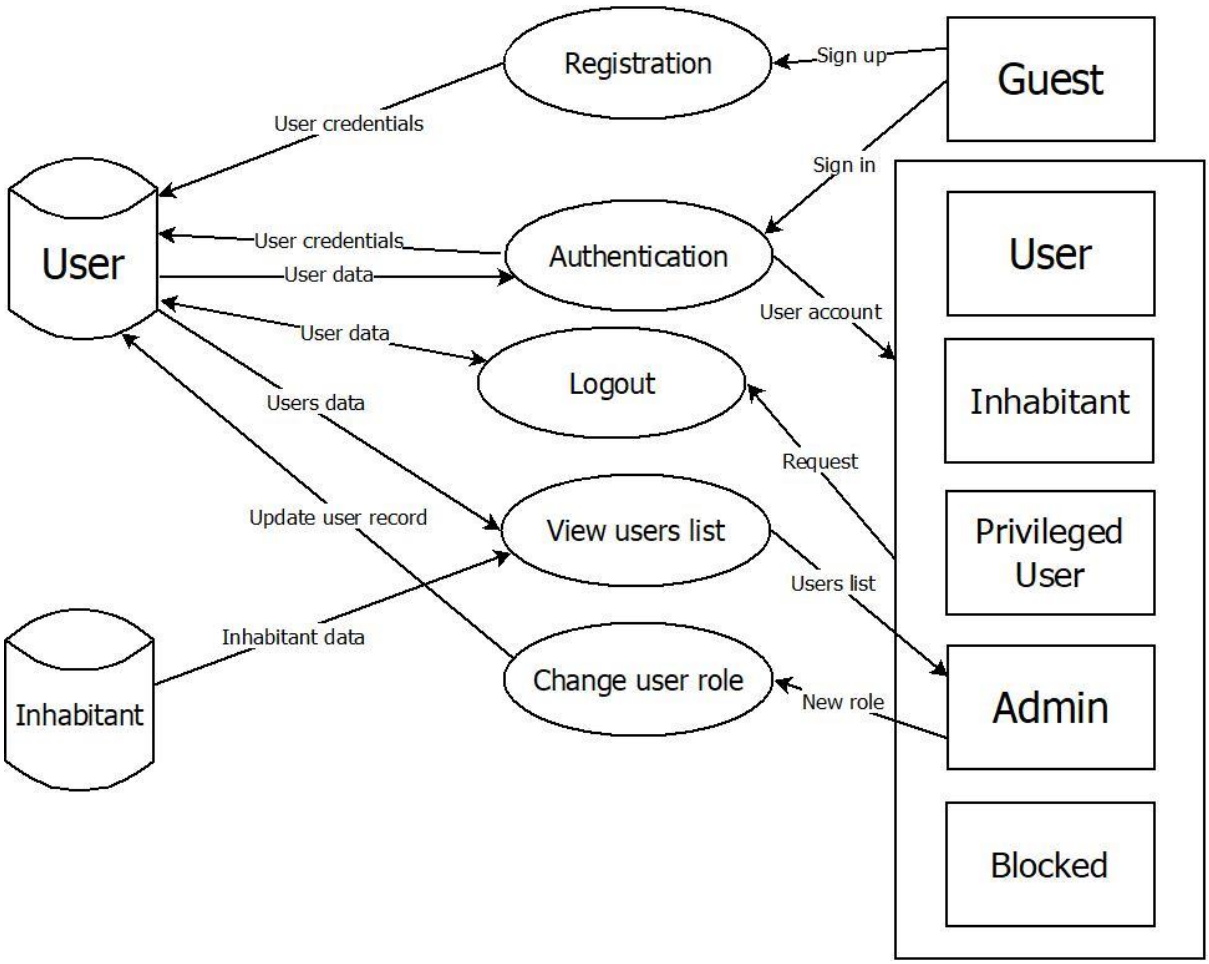
- Apartment management module requires data from Inhabitant module to prepare information for all owned apartment for inhabitant.
- Billing module requires data from Data submission module to calculate price on consumed heat and water. Also billing module requires data about apartment from Apartment management module to calculate managing cost based on apartment area, as well as data from Inhabitant management module to get payer's credentials.
- Notification module requires data from Billing module to send notifications when new bill/fine is available.



Pic. 4. Data flow diagram (level 1)

2.2.3. User Management module

The data flow diagram of the User management module (*pic. 5*) is based on the functions described in the *table 1*. Each function separately described in *tables 2-5*.



Pic. 5. User management module – data flow diagram (level 2)

Table 2. User management module – registration function

ID	User group
USR REGISTRATION	Guest
Name	
Registration	
Description	
Registering new user by a guest	
Pre-conditions	

1) Not logged to the system
Input
● User's name – up to 255 characters ● User's email – up to 255 characters ● User's password – minimum 8 characters
Processing
1. Checks email to have @ in it a. if don't conform, then error message (1) 2. Checks length of user's name a. if don't conform, then error message (3) 3. Checks length of email a. if don't conform, then error message (4) 4. Checks whether user with such email already exists. a. if exists, then error message (5) 5. Checks length of password a. if exists, then error message (2) 6. Check password confirmation field a. if not matches, then error message (6) 7. Adds new user to database.
Post-conditions
1) Logged to the system
Output
If user is successfully added to the system, he will be automatically logged in the system.
Error messages
1. Please include '@' in email address. 2. The password must be at least 8 characters. 3. The name may not be greater than 255 characters. 4. The email may not be greater than 255 characters. 5. The email has already been taken. 6. The password confirmation does not match.

Table 3. User management module – authentication function

ID	User group
USR LOGIN	Guest
Name	
Authentication	

Description
Login to the system by a guest
Pre-conditions
1) Not logged to the system
Input
● User's email – up to 255 characters ● User's password – minimum 8 characters ● Remember me (optional)
Processing
1. Checks email to have @ in it a. if don't conform, then error message (1) 2. Checks email and password to match the record a. if don't conform, then error message (2)
Post-conditions
1) Logged to the system 2) If user role (according *section 1.5*) is Blocked – show error message
Output
If user is successfully logged in, he sees the message saying "Welcome, username".
Error messages
1. Please include '@' in email address. 2. These credentials do not match our records.

Table 3a. User management module – logout function

ID	User group
LOGOUT	User, Inhabitant, Privileged user, Admin, Blocked
Name	
Logout	
Description	
Logout from the system by any non-guest user	
Pre-conditions	

2) Logged in the system
Processing
1. Logs out from the system by manipulations with the session
Post-conditions
1) Logged out of the system
Output
The main page of the application

Table 4. User management module – view user list function

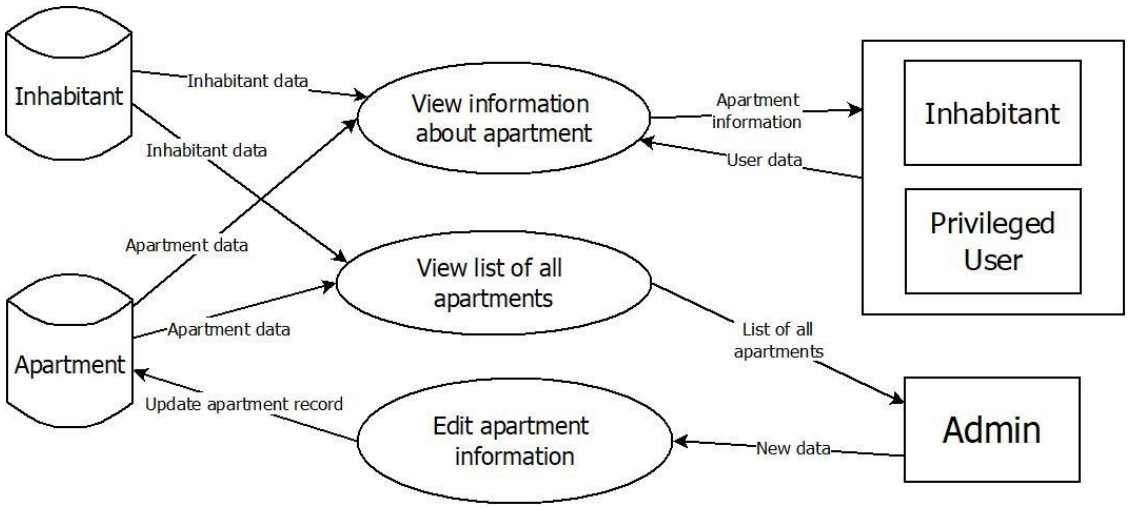
ID	User group
USR LIST	Admin
Name	
View user list	
Description	
Proceed and shows the list of all users of the system with corresponding user role and actions to be taken with users	
Pre-conditions	
1) Logged to the system as Admin	
Processing	
1. Selects all users from the database (name, email, role) 2. Checks if the user is linked to inhabitant a. if linked selects his full name b. if not linked adds function to link in the list (*Linking inhabitant with user account from the Inhabitant management module*) 3. Adds Change user role (*table 5*) function to each user	
Output	
Returns a formatted list of all users of the system, consisting of username, email, role, full name, linking to inhabitant (*table 12*) and change role function (*table 5*). At least there always will be 1 user – admin itself.	

Table 5. User management module – change user role function

ID	User group
USR ROLE	Admin
Name	
Change user role	
Description	
Changes the role of the selected user to different to change the user's rights	
Pre-conditions	
1) Logged to the system as Admin 2) Opened list of all users by View user list function (*table 4*). 3) Select function "change user role" for a specific user	
Processing	
1. Updates record field "user role" of the selected user in the database based on the new role chosen from the drop-down list.	
Output	
Returns a formatted list of all users of the system (by View user list function, *table 4*) with changed user role.	

2.2.4. Apartment management module

The data flow diagram of the Apartment management module (*pic. 6*) is based on the functions described in the *table 1*. Each function separately described in *tables 6-8*.



Pic. 6. Apartment management module – data flow diagram (level 2)

Table 6. Apartment management module – View information about apartment function

ID	User group
APT INFO	Inhabitant, Privileged User
Name	
View information about apartment	
Description	
Allows to see basic information about own apartment(s)	
Pre-conditions	
1) Logged to the system as Inhabitant or Privileged User	
Processing	
1. Checks if inhabitant has an apartment a. if not, apartment not found for a user, then error message (1) b. if apartment(s) is found – selects necessary fields from the database for all found apartments	
Output	
Returns a formatted data about all apartment owned by inhabitant of the house with checkbox to enable/disable email notifications (*table 42*)	
Error messages	
(1) You do not have an apartment	

Table 7. Apartment management module – View list of all apartments function

ID	User group
APT LIST	Admin
Name	
View information about all apartments in the house	
Description	
Allows to see all information about all apartments in the house	
Pre-conditions	
1) Logged to the system as Admin	

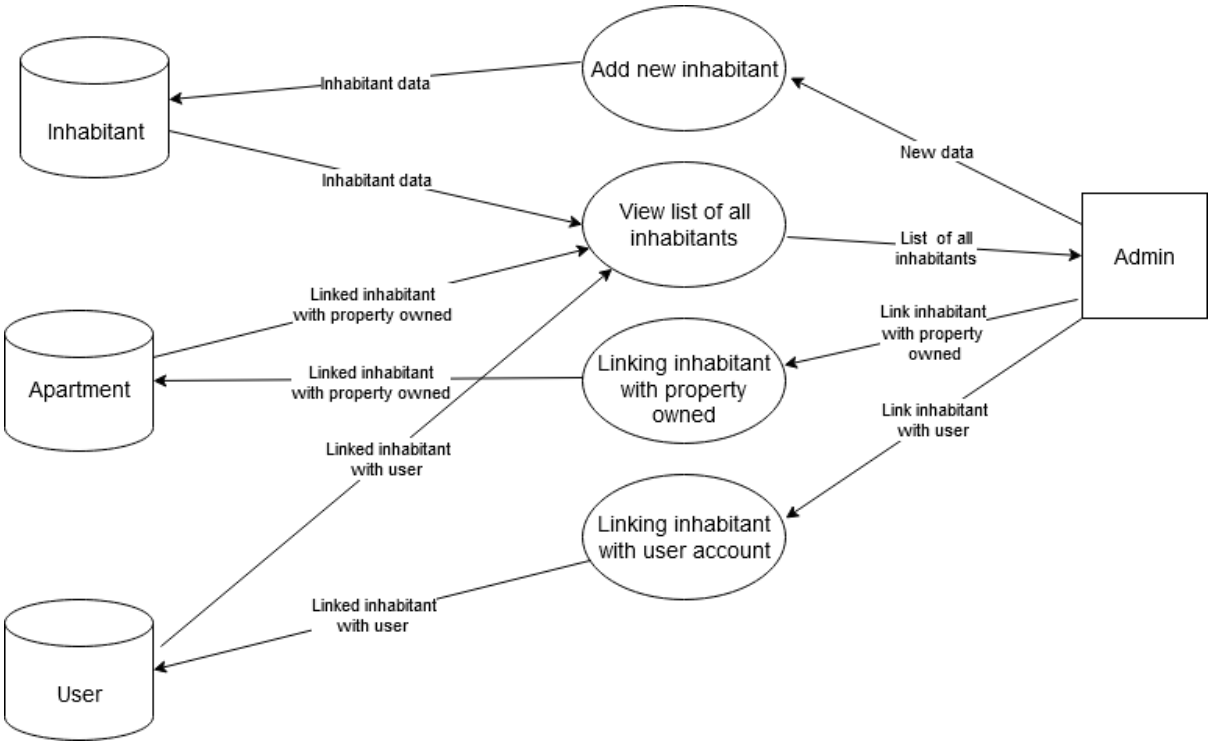
Processing
1. Selects all data about all apartments and their owners 2. Adds “Edit apartment information” function to all of them (*table 8*)
Output
Returns a formatted data about all apartments owned by inhabitant of the house and function to edit their data (*table 8*).

Table 8. Apartment management module – Edit apartment information function

ID	User group
APT EDIT	Admin
Name	
Edit apartment information	
Description	
Allows to edit information for specific apartment	
Pre-conditions	
1. Logged to the system as Admin 2. Open list of all apartments by View list of all apartments function (*table 7*)	
Input	
• All fields are pre-filled with database data Data can be changed in: • Number – integer • Area – float • Floor – integer	
Processing	
1. Fields data validation a. if wrong data type or blank, then error message (1) b. Updates record fields of selected apartment based on the new data from the fields.	
Output	
Returns a formatted data about all apartments (*table 7*) owned by inhabitants of the house with changed data.	
Error messages	
(1) Wrong data input	

2.2.5. Inhabitants management module

The data flow diagram of the Inhabitants management module (*pic. 7*) is based on the functions described in the *table 1*. Each function separately described in *tables 9-13*.



Pic. 7. Inhabitants management module – data flow diagram (level 2)

Table 9. Inhabitants management module – Add new inhabitant

ID	User group
ADD INH	Admin
Name	
Add new inhabitant	
Description	
Allows to add new inhabitant to system	
Pre-conditions	
1) Logged to the system as the Admin	
Input	

• Personal code (12 chars) • Name (up to 255 chars) • Surname (up to 255 chars) • Phone(s) (according to the custom phone validation format)
Processing
1. Fields data validation a. if wrong format of personal code, then error message (1) b. if personal code already exists in the database, then error message (2) c. if name has more than 255 chars, then error message (3) d. if surname has more than 255 chars, then error message (4) e. if phone format is incorrect, then error message (5) 2. Adds record with inserted data to the database.
Output
Returns list of all inhabitants (*table 10*) with newly added inhabitant
Error messages
(1) Personal code does not fit the format. (2) Such entry already exists. (3) The name may not be greater than 255 characters. (4) The surname may not be greater than 255 characters. (5) Phone is in incorrect format

Table 10. Inhabitants management module – View list of all inhabitants

ID	User group
INH LIST	Admin
Name	
View list of all inhabitants	
Description	
Allows to see all information about all inhabitants in the house	
Pre-conditions	
1. Logged to the system as Admin	
Processing	
1. Selects all data about all inhabitants in the system 2. Adds function “Linking inhabitant with property owned” (*table 11*) to each entry in the list	

Output
Returns a formatted data about all inhabitants of the house

Table 11. Inhabitant management module – Linking inhabitant with property owned

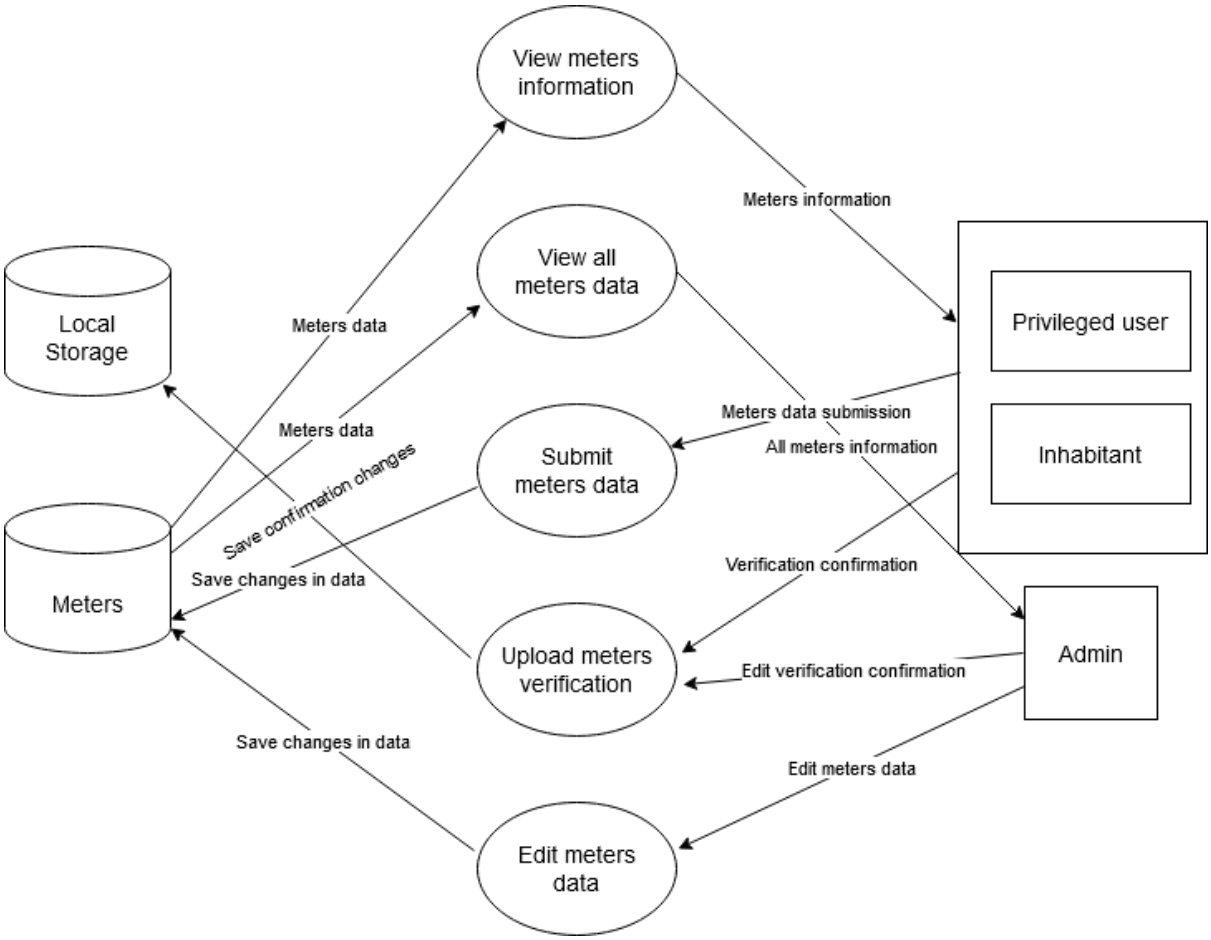
ID	User group
LINK INH_PROP	Admin
Name	
Linking inhabitant with property owned	
Description	
Allows to link inhabitant(s) with their property(ies) (apartment(s) owned)	
Pre-conditions	
1. Logged to the system as Admin 2. Opened “list of all inhabitants” (*table 10*)	
Input	
1) • Selected “Linking inhabitant with property owned” in “list of all inhabitants” (*table 10*) • Selected number of apartment(s) to be linked to 2) • Selected “Linking inhabitant with property owned” in “list of all inhabitants” (*table 10*) • Selected “delete” at the according linking entry • Deletes record with corresponding data from the database.	
Processing	
1. Fields data validation a. if apartment number not chosen, then error message (1) 2. Adds/deletes record to/from database.	
Output	
Stays on the same page, displaying new changes	
Error messages	
(1) Apartment has to be chosen	

Table 12. Inhabitant management module – Linking inhabitant with user account

ID	User group
LINK INH_USR ACC	Admin
Name	
Linking inhabitant with user account	
Description	
Allows to link inhabitant(s) with their user account(s)	
Pre-conditions	
1. Logged to the system as Admin 2. Open “list of all users” (*table 4*)	
Input	
• Selected “Linking inhabitant with user account” in “list of all inhabitants” (*table 10*) • Selected user account to be linked to	
Processing	
1. Fields data validation a. if blank, then error message (1) 2. Updates record fields of selected inhabitant and user account based on the new data	
Output	
Returns list of all users (*table 4*) without function “Linking inhabitant with user account” for linked inhabitant	
Error messages	
(1) Wrong data input.	

2.2.6. Data Submission module

The data flow diagram of the Data Submission module (*pic. 8*) is based on the functions described in the *table 1*. Each function separately described in *tables 13-17*.



Pic. 8. Data submission module – data flow diagram (level 2)

Table 13. Data Submission module – View meters information

ID	User group
METR INFO	Inhabitant, Privileged user
Name	
View meters information	
Description	
Allow to inhabitants and privileged users to view information about meters	
Pre-conditions	

1. Logged to the system as Inhabitant or Privileged User
Processing
1. Checks if inhabitant has an apartment a. if not, apartment not found for a user, then error message (1) 2. If apartment(s) is found – selects necessary fields from the database for all found apartments
Output
Returns a formatted data about apartment's meters information owned by inhabitant of the house with function of submitting meters data (*table 15*) and upload meters verification (*table 16*) for each of meters.
Error messages
(1) You do not have an apartment

Table 14. Data Submission module – View all meters data

ID	User group
ALL METR INFO	Admin
Name	
View all meters data	
Description	
Allows admin to view all information about all apartment's meters	
Pre-conditions	
1. Logged to the system as Admin	
Processing	
1. Retrieves all meters data submissions for selected time period (default is current month) from database associated with apartments and owners (if there are such)	
Output	
Returns a formatted data about meters information associated with apartments and owners (if there are such) with functionality of editing data (*table 17*) and uploading verification report with each meters entry (*table 16*).	

Table 15. Data Submission Module – Submit meters data

ID	User group
SUBM METR INFO	Inhabitant, Privileged user
Name	
Submit meters data	
Description	
Allow to inhabitants and privileged users to submit data collected from their apartment meters	
Pre-conditions	
1. Logged to the system as Inhabitant or Privileged User 2. Opened “view meters information” (*table 13*)	
Input	
• All fields are filled with data collected from meters	
Processing	
1. Checks if inhabitant has an apartment a. if not, apartment not found for a user, then error message (1) 2. If apartment(s) is found – collect present necessary data from meters a. if wrong data type or blank, then error message (2) b. if submitted data is lower than previous submitted, then error message (3)	
Output	
Submits a formatted data about apartment’s meters information owned by inhabitant of the house to the database and returns “view meters information” (*table 13*) with new data	
Error messages	
(1) You do not have an apartment (2) Wrong entry type (3) New data could not be lower than previous	

Table 16. Data Submission Module – Upload meters verification

ID	User group
UPD METR VERIF	Inhabitant, Privileged user, Admin
Name	
Upload meters verification	

Description
Allow to inhabitants, privileged users and admins to upload meters verification report
Pre-conditions
1. Logged to the system as Inhabitant or Privileged User 2. Opened “view meters information” (*table 13*) or 1. Logged to the system as Admin 2. Opened “view all meters data” (*table 14*)
Input
• Verification report upload
Processing
If Inhabitant or Privileged User: 1. Checks if inhabitant has an apartment a. if not, apartment not found for a user, then error message (1) 2. If apartment(s) is found – select meter to which upload verification 3. Uploads verification report to local storage a. if left blank or wrong document type, then error message (2) If Admin: 1. Selects “upload verification” for meter in “view all meters data” (*table 14*) 2. Uploads verification report to local storage a. if left blank or wrong document type, then error message (2)
Output
If Inhabitant or Privileged User: • Returns “View meters information” (*table 13*) If Admin: • Returns “View all meters data” (*table 14*)
Error messages
(1) You do not have an apartment (2) Wrong upload

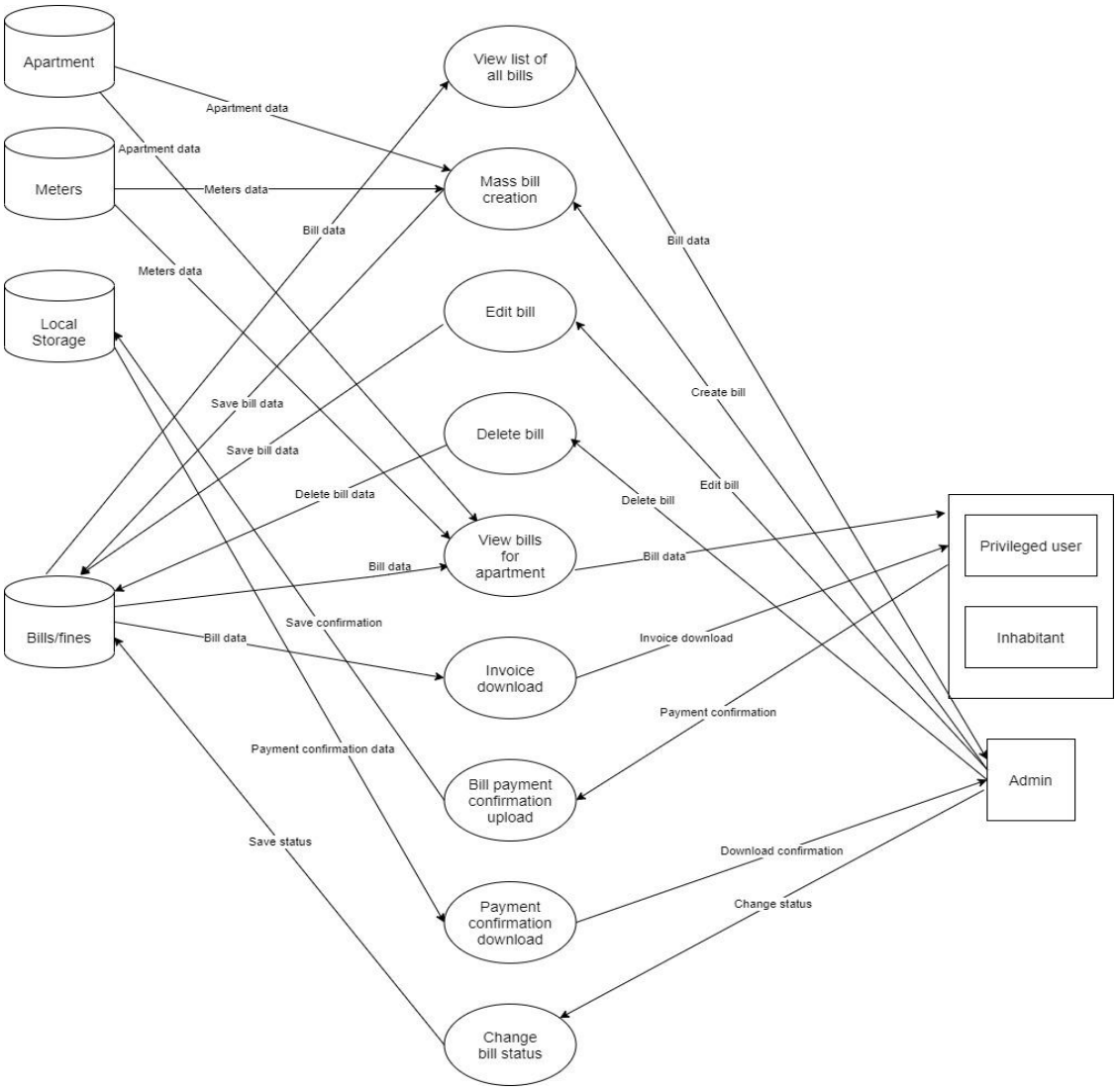
Table 17. Data Submission Module – Edit meters data

ID	User group
EDIT METR DATA	Admin
Name	
Edit meters data	

Description
Edition of meters data
Pre-conditions
1. Logged to the system as Admin 2. Opened list of all apartment's meters by "view all meter's data" (*table 14*). 3. Select function "Edit meters data" for a specific apartment's meter
Processing
1. Validates all input fields (meter data, verified till) a. if wrong data type, blank or incorrect format, then error message (1) 2. Updates record field "apartment's meters" of the selected apartment in the database
Output
Returns a formatted list of meters data (by "view all meters data" function, *table 14*).
Error messages
(1) Wrong data

2.2.7. Billing Module

The data flow diagram of the Billing module (*part 1*) (*pic. 9*) is based on the functions described in the *table 1*. Each function separately described in *tables 18-26*.



Pic. 9. Billing module – data flow diagram (level 2) part I

Table 18. Billing module – View list of all bills

ID	User group
BILLS LIST	Admin
Name	
View list of all bills	

Description
Proceed and shows the list of all bills of the system with corresponding to which apartment that bills refer.
Pre-conditions
1. Logged to the system as Admin
Processing
1. Selects list of bills from the database.
Output
Returns a formatted list of all bills in the system with functions of bills deleting (*table 21*) and editing (*table 20*) with each bill entry, confirmation download (*table 25*) (if there is such) and change bill status (*table 26*). Also utilized mass bill creation function (*table 19*).

Table 19. Billing module – Mass bill creation

ID	User group
BILL CRT	Admin
Name	
Mass bill creation	
Description	
Mass bill creation process for users.	
Pre-conditions	
1. Logged to the system as Admin 2. Open "view list of all bills" (*table 18*)	
Input	
• Select month for billing • Filled all cost fields	
Processing	
1. Validates all input fields a. if wrong data type or blank, then error message (1) 2. Created and adds bill entries to database	
Output	
Returns list of all bills (*table 18*) with newly added bills	

Error messages
(1) Wrong data

Table 20. Billing module – Edit bill

ID	User group
EDIT BILL	Admin
Name	
Edit bill	
Description	
Provides edition for bills. Changing details in case if small mistake was made in accounts department.	
Pre-conditions	
1. Logged to the system as Admin 2. Open “view list of all bills” (*table 18*)	
Input	
• Filled bill costs fields	
Processing	
1. Validates all input fields a. if wrong data type or blank, then error message (1) 2. Selects list of bills from the database according to period 3. Adds change to the bills	
Output	
Returns list of all bills with made changes (*table 18*)	
Error messages	
(1) Wrong data	

Table 21. Billing module – Delete bill

ID	User group
DEL BILL	Admin
Name	
Delete bill	

Description
Deletes bills from the database of the system.
Pre-conditions
1. Logged to the system as Admin 2. Opened “view list of all bills” (*table 18*)
Input
• Select bill to delete
Processing
1. Deletes bill entry from the database
Output
Returns list of all bills with made changes (*table 18*)

Table 22. Billing module – View bill for apartment

ID	User group
USR LIST	Inhabitant, Privileged user
Name	
View bill for apartment	
Description	
Proceed and shows the list of all bills referred for particular inhabitant of the system.	
Pre-conditions	
1. Logged to the system as Inhabitant or as Privileged user	
Processing	
1. Checks if the user is linked to inhabitant and has an apartment(s) a. if no, then error (1) 2. Proceed and collects and create a list of all bills which refers to him.	
Output	
Returns a formatted list of all bills of the apartment(s) to the inhabitant who made request to view with functions of upload payment confirmation (*table 24*) and download invoice (*table 23*) for each bill entry depending on status of the bill.	
Error messages	

(1) You do not have any bills to select

Table 23. Billing module – Bill invoice download

ID	User group
BIL INVC DOWNLOAD	Inhabitant, Privileged user
Name	
Bill invoice download	
Description	
Conducts processing and creating of bill for particular user and provides functionality to download invoice in the end for making regular payment.	
Pre-conditions	
1. Logged to the system as Inhabitant or as Privileged user 2. Opened “view bill for apartment” and selected (*table 22*)	
Processing	
1. Generates invoice in form of PDF for selected invoice	
Output	
Returns invoice document in PDF format and stays on the same page.	

Table 24. Billing module – Bill payment confirmation upload

ID	User group
PYMNT CONFN UPL	Inhabitant, Privileged user
Name	
Bill payment confirmation upload	
Description	
Allow to inhabitants, privileged users to confirm bill payment and saves confirmation of bill payment to local storage	
Pre-conditions	
1. Logged to the system as Inhabitant or Privileged User 2. Opened “view bill for apartment” and selected (*table 22*)	
Input	

• Payment confirmation in PDF format
Processing
1. Validates upload a. if left blank or wrong file format, then error (1) 2. Save invoice in PDF format to local storage 3. Changes bill status in the database
Output
Returns “view bill for apartment” (*table 22*) with made changes
Error messages
(1) Wrong upload

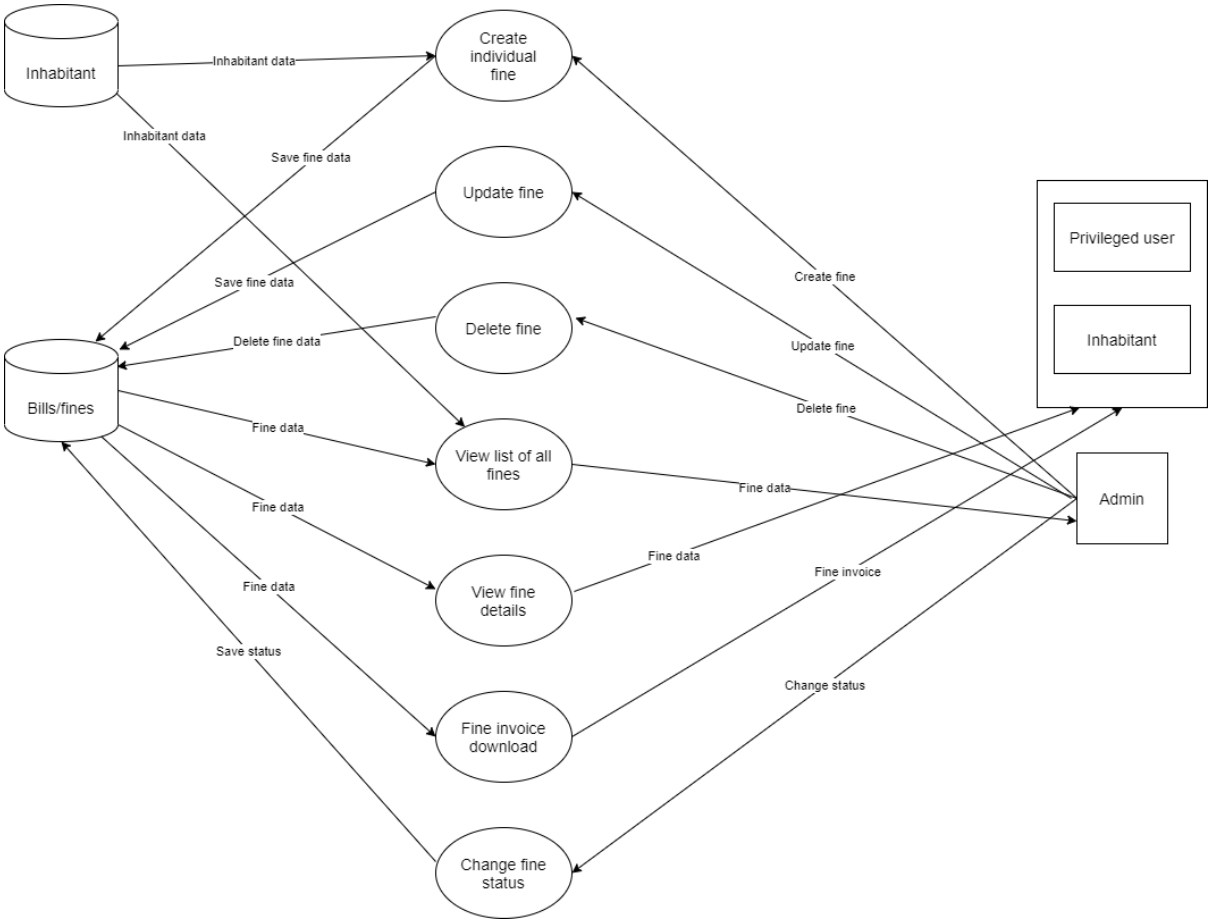
Table 25. Billing module – Payment confirmation download

ID	User group
PYMNT CONFN DWD	Admin
Name	
Payment confirmation download	
Description	
Allows admin to retrieve data from local storage and download it	
Pre-conditions	
1. Logged to the system as admin 2. Opened “view list of all bills” (*table 18*)	
Input	
• Select download confirmation for necessary bill entry	
Processing	
1. Downloads payment confirmation from local storage to device	
Output	
Payment confirmation in PDF format and stays on the same page.	

Table 26. Billing module – Change bill status

ID	User group
BILL STS CHG	Admin
Name	
Change bill status	
Description	
Allows admin to edit and change bill status after the answer from account department have payment processed or not.	
Pre-conditions	
1. Logged to the system as admin 2. Opened “view list of all bills” (*table 18*)	
Input	
• New status from list of all possible statuses	
Processing	
1. Checks if new status is selected a. if no or blank, then error (1) 2. Save changes to the database	
Output	
Returns “view list of all bills” (*table 18*) with made changes	
Error messages	
(1) Status has to be filled and to be new	

The data flow diagram of the Billing module (*part 2*) (*pic. 10*) is based on the functions described in the *table 1*. Each function separately described in *tables 27-33*.



Pic. 10. Billing module – data flow diagram (level 2) part II

Table 27. Billing Module (*part II*) – Create individual fine

ID	User group
CRT IND FINE	Admin
Name	
Create individual fine	
Description	
Allows admin to create individual fine for particular user/inhabitant.	
Pre-conditions	
1. Logged to the system as admin 2. Open “view list of all fines” (*table 30*)	
Input	
• Select inhabitant to whom fine is issued • Fine sum (can be left blank)	

• Fine reason
Processing
1. Validates all input fields a. if wrong data type or blank, then error message (1) 2. Adds new entry with fine data to database
Output
Returns “view list of all fines” (*table 30*) with new data
Error messages
(1) Wrong input data

Table 28. Billing module (*part II*) – Update fine

ID	User group
UPT FINE	Admin
Name	
Update fine	
Description	
Allows admin to update fine data for particular fine.	
Pre-conditions	
1. Logged to the system as admin 2. Opened “view list of all fines” (*table 30*)	
Input	
• New fine sum and/or new fine reason	
Processing	
1. Validates all input fields a. if wrong data type or blank, then error message (1) 2. Updates entry data in database with new fine data	
Output	
Returns “view list of all fines” (*table 30*) with new data	
Error messages	
(1) Wrong input data	

Table 29. Billing module (*part II*) – Delete fine

ID	User group
DLT FINE	Admin
Name	
Delete fine	
Description	
Allows admin to delete particular fine	
Pre-conditions	
1. Logged to the system as admin 2. Opened "view list of all fines" (*table 30*)	
Input	
• Select fine to delete	
Processing	
1. Deletes corresponding fine entry from the database	
Output	
Returns "view list of all fines" (*table 30*) with new data	

Table 30. Billing module (*part II*) – View list of all fines

ID	User group
LST ALL FINES	Admin
Name	
View list of all fines	
Description	
Allows admin to see list of all fines.	
Pre-conditions	
1. Logged to the system as admin	
Processing	
1. Selects all fines from the database.	

Output
Returns preformatted list of all fines from the database, adding to each fine entry functionality to delete (*table 29*), update fine data (*table 28*) and change fine status (*table 33*). Also utilizes function of creating a new fine (*table 27*)

Table 31. Billing module (*part II*) – View fine details

ID	User group
FINE DETAILS	Inhabitant, Privileged user
Name	
View fine details	
Description	
Allows inhabitant/privileged user to view fine lists for all fines that was issued to them	
Pre-conditions	
1. Logged to the system as Inhabitant or Privileged user.	
Processing	
1. Selects all fines issued to a particular inhabitant	
Output	
Returns preformatted list of all fines with function of downloading fine invoice (*table 32*)	

Table 32. Billing module (*part II*) - Fine invoice download

ID	User group
FINE INVC DOWNLOAD	Inhabitant, Privileged user
Name	
Fine invoice download	
Description	
Conducts processing and creating of fine for particular user and provides functionality to download invoice in the end for making regular payment.	
Pre-conditions	
1. Logged to the system as Inhabitant or Privileged user. 2. Opened “view fine details” (*table 31*)	

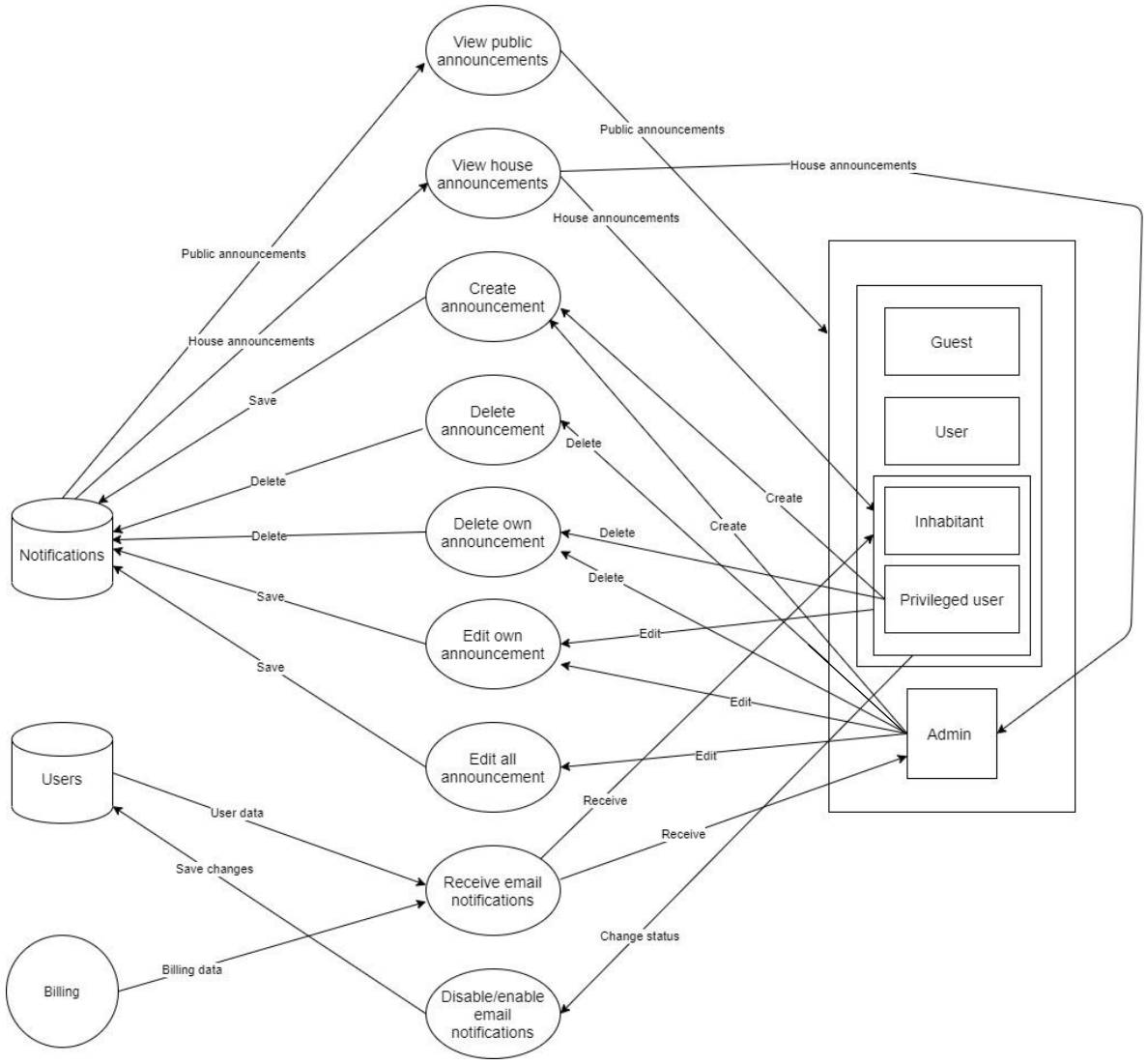
Input
• Select download invoice for necessary fine entry
Processing
1. Generates invoice in form of PDF for selected invoice
Output
Returns invoice document in PDF format and stays on the same page

Table 33. Billing module (*part II*) – Change fine status

ID	User group
CHG FINE STS	Admin
Name	
Change fine status	
Description	
Allows admin to change the status of individuals' fines after the verification and revision from accounts department.	
Pre-conditions	
1. Logged to the system as Admin. 2. Opened "view list of all fines" (*table 30*)	
Input	
• New status from list of all possible statuses	
Processing	
1. Checks if new status is selected a. if no or blank, then error (1) 2. Save changes to the database	
Output	
Returns "view list of all fines" (*table 30*) with made changes	
Error messages	
(1) Status has to be filled and to be new	

2.2.8. Notifications module

The data flow diagram of the Notifications module (*pic. 11*) is based on the functions described in the *table 1*. Each function separately described in *tables 34-42*.



Pic. 11. Notifications module – data flow diagram (level 2)

Table 34. Notifications module – View public announcements

ID	User group
PBL ANNOUN	Guest, User, Inhabitant, Privileged user, Admin
Name	
View public announcements	

Description
Allows to see all public announcements made in the system
Pre-conditions
1. Logged to the system as User, Inhabitant, Privileged user, Admin or being Guest (not logged in).
Processing
1. Selects all public announcements from the database
Output
Returns all public announcements in preformatted way (basically forming the main page for Guest and User and part of the main page for Inhabitant, Privileged user and Admin and part of list of all announcements for Admin and list of own announcements for Privileged user)

Table 35. Notifications module – View house announcements

ID	User group
PRVT ANNOUN	Inhabitant, Privileged user, Admin
Name	
View house announcements	
Description	
Allows to see all public announcements made in the system	
Pre-conditions	
1. Logged to the system as Inhabitant, Privileged user or Admin	
Processing	
1. Selects all house announcements from the database	
Output	
Returns all house announcements in preformatted way. Together with public announcements (*table 34*) forming the main page for Inhabitant, Privileged user and Admin and list of all announcements for Admin and list of own announcements for Privileged user	

Table 36. Notifications module – Create announcement

ID	User group
CRT ANNOUN	Privileged user, Admin
Name	
Create announcement	
Description	
Allows to see create new announcements in the system	
Pre-conditions	
1. Logged to the system as Privileged user or Admin 2. Open a list of all announcements (combination of functions from *table 34, 35*) 3. Select "create new announcement"	
Input	
• Header text • Message text • Announcement type • Announcement language	
Processing	
1. Validates all fields a. if filed left blank, then error (1) 2. Adds new notification to the database	
Output	
Returns the main page of application (combination of functions from *table 34, 35*)	
Error messages	
(1) All fields have to be filled	

Table 37. Notifications module – Delete announcement

ID	User group
DLT ANY ANNOUN	Admin
Name	
Delete announcement	

Description
Allows to delete any announcement in the system
Pre-conditions
1. Logged to the system as Admin 2. Open a list of all announcements (combination of functions from *table 34, 35*)
Input
• Select announcement to delete
Processing
1. Deletes selected announcement from the database
Output
Returns the main page of application (combination of functions from *table 34, 35*)

Table 38. Notifications module – Delete own announcement

ID	User group
DLT OWN ANNOUN	Privileged user, Admin
Name	
Delete own announcement	
Description	
Allows to delete own announcement in the system	
Pre-conditions	
1. Logged to the system as Privileged user or Admin 2. Open a list of all announcements (combination of functions from *table 34, 35*)	
Input	
• Select announcement to delete	
Processing	
1. Deletes selected announcement from the database	
Output	
Returns the main page of application (combination of functions from *table 34, 35*)	

Table 39. Notifications module – Edit own announcement

ID	User group
EDT OWN ANNOUN	Privileged user, Admin
Name	
Edit own announcement	
Description	
Allows to edit own announcement in the system	
Pre-conditions	
1. Logged to the system as Privileged user or Admin 2. Open a list of all announcements (combination of functions from *table 34, 35*)	
Input	
• Select announcement to edit • Change data of fields (header text, message text, announcement type)	
Processing	
1. Validates all fields a. if filed left blank, then error (1) 2. Updates selected announcement from the database	
Output	
Returns the main page of application (combination of functions from *table 34, 35*)	
Error messages	
(1) All fields have to be filled	

Table 40. Notifications module – Edit all announcements

ID	User group
EDT ALL ANNOUN	Admin
Name	
Edit all announcement	
Description	
Allows to edit all announcements in the system	
Pre-conditions	

1. Logged to the system as Admin 2. Open a list of all announcements (combination of functions from *table 34, 35*)
Input
• Select announcement to edit • Change data of fields (header text, message text, announcement type)
Processing
1. Validates all fields a. if filed left blank, then error (1) 2. Updates selected announcement from the database
Output
Returns the main page of application (combination of functions from *table 34, 35*)
Error messages
(1) All fields have to be filled

Table 41. Notifications module – Receive email notification

ID	User group
RCV EMAIL NTF	Inhabitant, Privileged user, Admin
Name	
Receive email notification	
Description	
Sends email notification based on contents got either from the Billing module or new announcements	
Pre-conditions	
1. Be Inhabitant, Privileged user, Admin 2. Has email notifications turned on (*table 42*)	
Input	
• Data for message from Billing module or new announcements data	
Processing	
1. Checks if email notifications turned on a. if no, then terminates 2. Composes email based on the necessary information 3. Sends email to corresponding user email address	

Output
Email to the user email address

Table 42. Notifications module – Disable/enable email notification

ID	User group
RCV EMAIL NTF	Inhabitant, Privileged user
Name	
Disable/enable email notification	
Description	
Allows to turn off and on email notification	
Pre-conditions	
1. Logged to the system as Inhabitant or Privileged 2. Opened “View information about apartment” function (*table 6*)	
Input	
• Select email notification status (on/off)	
Processing	
1. Updates data in the database according to selected option	
Output	
Success message on the same page	

2.3. Non-functional requirements

For the system formulated following non-functional requirements:

- The application should be accessible from the most popular browsers (Microsoft Internet Explorer, Google Chrome, Mozilla Firefox and Apple's Safari) without lack of functionality.
- The application design has to be responsible to provide the best experience both on desktop and mobile devices.
- The system has to utilize protection against Cross-site request forgery (CSRF), Cross-Site Scripting (XSS) and SQL injections in order to prevent data loss, corruption and stealing data from the database.
- The system must store user passwords in the encoded way.
- HTLM and CSS must be valid for the system by W3C Validation.
- Localization for two-language user interface should be provided.

3. Software design description

3.1. Database design

Based on the functional description and conceptual database model (*pic. 3*) logical database model was created (*pic. 12*). It includes one additional entity – conjunction table for N to N relation between Inhabitant and Apartment entities and showed on the picture as InhabitantApartment entity.

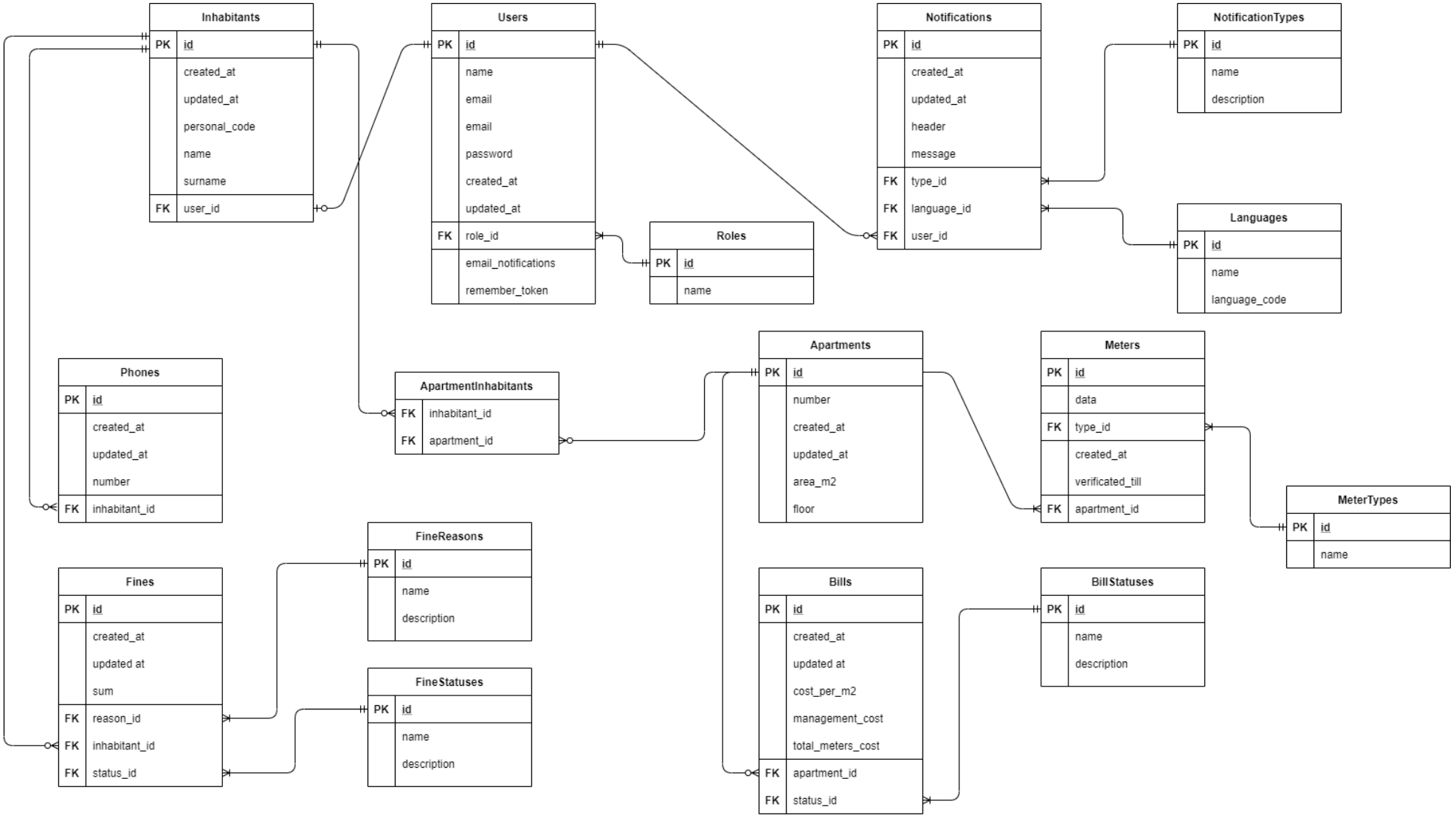
Based on the logical model – the physical databased model was created. It contains the same number of entities as logical model but includes main datatypes and nullable column *(pic. 13*).

The terms of data types and other applicable parameters to database content may vary depending on the used DBMS. Herein used terms of Microsoft SQL server.

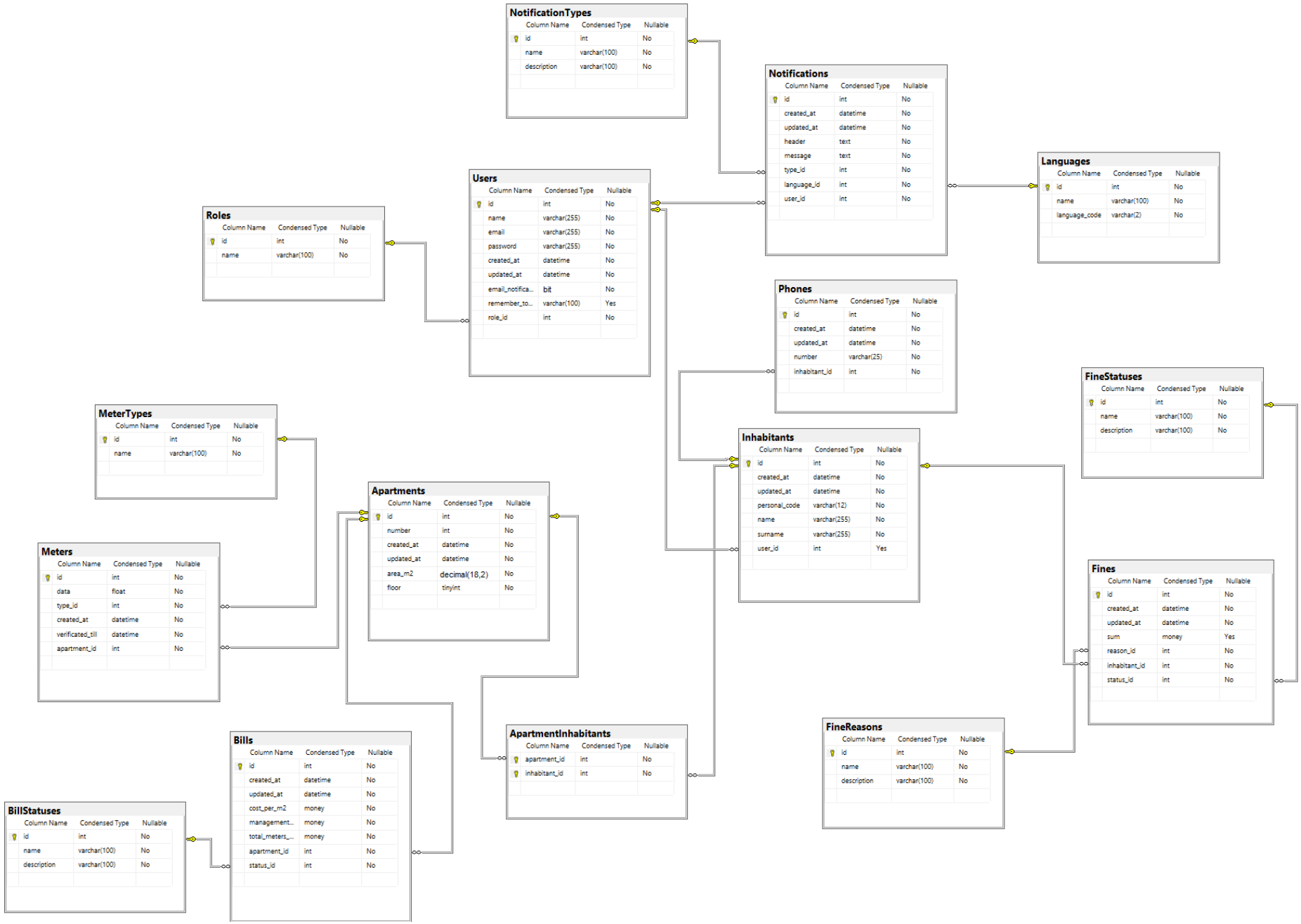
The description of database tables design is split in the subsections according to the modules breakdown shown in the *section 2.2.2*.

- In *subsection 3.1.1.* described design of Users and Roles entities.
- In *subsection 3.1.2.* described design of Inhabitants and Phones entities.
- In *subsection 3.1.3.* describes design of Apartments and ApartmentInhabitants entities.
- In *subsection 3.1.4.* described design of Meters and MeterTypes entities.
- In *subsection 3.1.5.* described design of Bills, BillStatus, Fines, FineStatus and FineReasons entities.
- In *subsection 3.1.6.* described design of Notifications, NotificationType and Languages entities.

Initial seedings examples are shown in the *Appendix* according to assumptions and dependencies described in the *section 1.7*.



Pic. 12. Logical database model (crowfoot syntax)



Pic. 13. Physical database model

3.1.1. Users and corresponding entities

This subsection contains detailed information about table design of Users entity (*table 43*) and related codifier table – Roles (*table 44*).

Mainly the data from the Users entity is used by User Management module functions (*section 2.2.3., tables 2-5*) as well as in Inhabitants managing module (*section 2.2.5., tables 10, 12*) and Notifications module (*tables 41, 42*).

Table 43. Users entity – description

Key	Column name	Data type	Nullable	Other parameters	Description
PK	id	int	No	IDENTITY (1, 1)	Artificial id for user identification
	name	varchar(255)	No		Username used in the system
	email	varchar(255)	No	UNIQUE	Unique email address belonging to the user
	password	varchar(255)	No		Hashed password
	created_at	datetime	No		Date of creation of the corresponding user account
	updated_at	datetime	No		Date of information update of the corresponding user account
	email_notifications	bit	No		Represents boolean datatype and shows whether email notifications turned on or off
	remember_token	varchar(100)	Yes		Token for “remember me” field of login. Can be NULL if was not selected

FK	role_id	int	No	FK referenced from entity Roles (*table 44*, id attribute)	The role of the user in the system

User roles are stored in the Roles table – are the roles showed in the *section 1.5*. of the document.

Table 44. Roles entity – description

Key	Column name	Data type	Nullable	Other parameters	Description
PK	id	int	No	IDENTITY (1, 1)	Artificial id for user role identification
	name	varchar(100)	No		Role name used in the system

3.1.2. Inhabitants and corresponding entities

This subsection contains detailed information about table design of Inhabitants entity (*table 45*) and related phone number table – Phones (*table 46*).

Mainly the data from the Inhabitants entity is used by Inhabitants Management module functions (*section 2.2.5., tables 9-13*) as well as in Apartment managing module (*section 2.2.4., tables 6-8*) and Notifications module (*tables 41, 42*).

Table 45. Inhabitants entity – description

Key	Column name	Data type	Nullable	Other parameters	Description
PK	id	int	No	IDENTITY (1, 1)	Artificial id for inhabitant identification

	created_at	datetime	No		Date of creation of the corresponding inhabitant entry
	updated_at	datetime	No		Date of information update of the corresponding inhabitant entry
	personal_code	varchar(12)	No	UNIQUE	Unique personal code belonging to inhabitant
	name	varchar(255)	No		Inhabitant's name
	surname	varchar(255)	No		Inhabitant's surname
FK	user_id	int	Yes	UNIQUE, FK referenced from entity Users (*table 43*, id attribute)	Connection between user's account and inhabitant. If NULL – inhabitant do not have user account

Telephone number of inhabitants are stored in the Phones entity.

Table 46. Phones entity – description

Key	**Column name**	**Data type**	**Nullable**	**Other parameters**	**Description**
PK	id	int	No	IDENTITY (1, 1)	Artificial id for phone identification
	created_at	datetime	No		Date of creation of the corresponding phone entry
	updated_at	datetime	No		Date of information update of the corresponding phone entry

	number	varchar(25)	No		Phone number
FK	inhabitant_id	int	No	FK referenced from entity Inhabitants (*table 45*, id attribute)	Inhabitant to whom the phone number belongs to

3.1.3. Apartments and corresponding entities

This subsection contains detailed information about table design of Apartments entity (*table 47*) and related table – Corresponding ApartmentInhabitant entities (*table 48*).

Mainly the data from the Apartments entity is used by Apartment Management module functions (*section 2.2.4., tables 6-8*) as well as in Inhabitants managing module (*section 2.2.5., tables 10, 12*) and Notifications module (*tables 41, 42*).

Table 47. Apartment entity – description

Key	**Column name**	**Data type**	**Nullable**	**Other parameters**	**Description**
PK	id	int	No	IDENTITY (1, 1)	Artificial id for apartment identification
	number	int	No		The number of apartment
	created_at	datetime	No		Date of creation of the corresponding apartment entry
	updated_at	datetime	No		Date of information update of the corresponding apartment entry
	area_m2	decimal(18, 2)	No		Area in m^2 of the apartment
	floor	tinyint	No		The floor of apartment

ApartmentInhabitants entity is used to solve problem of future denormalization of database structure. ApartmentInhabitant bridges two entities (Inhabitants and Apartments) between each other.

Table 48. ApartmentInhabitants entity – description

Key		Column name	Data type	Nullable	Other parameters	Description
PK	FK	inhabitant_id	int	No	FK referenced from entity Inhabitants (*table 45*, id attribute)	Contains inhabitant id, who currently owns property
	FK	apartment_id	int	No	FK referenced from entity Apartments (*table 47*, id attribute)	Contains apartment id, which currently owned by inhabitant

3.1.4. Meters and corresponding entities

The subsection contains detailed information about table design of Meters entity (*table 49*) and related table – Corresponding MetersTypes entity (*table 50*).

Meters entity mainly used in the Data submission module functions (*section 2.2.6., tables 13, 14, 15, 17*) and in the Billing module functions (*section 2.2.7., tables 19, 22*).

Table 49. Meters entity – description

Key	Column name	Data type	Nullable	Other parameters	Description
PK	id	int	No	IDENTITY (1, 1)	Artificial id for meters entry identification
	data	float	No		Data of the meter submitted

FK	type_id	int	No	FK referenced from entity MetersType (*table 50*, id attribute)	Type of the meter
	created_at	datetime	No		Date of creation of the corresponding data submission entry
	verificated_till	datetime	No		Date after which submitted data cannot be trusted and used for calculations
FK	apartment_id	int	No	FK referenced from entity Apartments (*table 47*, id attribute)	The apartment which data of meters was submitted

MeterTypes entity (*table 50*) is a codifier table to Meters entity (*table 49*), storing a type of meter.

Table 50. MeterTypes entity – description

Key	**Column name**	**Data type**	**Nullable**	**Other parameters**	**Description**
PK	id	int	No	IDENTITY (1, 1)	Artificial id for type of the meter identification
	name	varchar(100)	No		Name of the meter type (purpose)

3.1.5. Bills, Fines and corresponding entities

This subsection contains detailed information about table design of Bills, Fines and corresponding entities (*tables 51-55*).

Bills entity mainly used in Billing module (*section 2.2.7., tables 18-23*), also in Notifications module functions (*section 2.2.8., table 41*).

Table 51. Bills entity – description

Key	**Column name**	**Data type**	**Nullable**	**Other parameters**	**Description**
PK	id	int	No	IDENTITY (1, 1)	Artificial id for bill entry identification
	created_at	datetime	No		Date of creation of the corresponding bill entry
	updated_at	datetime	No		Date of information update of the corresponding bill entry
	cost_per_m2	money	No		Cost of services per m^2
	management_cost	money	No		Fixed management cost
	total_meters_cost	money	No		Total cost from meters
FK	apartment_id	int	No	FK referenced from entity Apartments (*table 47*, id attribute)	The apartment to bill is issued
FK	status_id	int	No	FK referenced from entity	Current bill status

				BillStatuses (*table 52*, id attribute)	

BillStatuses entity (*table 52*) is a codifier table to Bills entity (*table 51*), storing a list of possible bill statuses.

Table 52. BillStatuses entity – description

Key	Column name	Data type	Nullable	Other parameters	Description
PK	id	int	No	IDENTITY (1, 1)	Artificial id for type of the bill
	name	varchar(100)	No		Name of the status
	description	varchar(100)	No		Description of the status

Fines entity mainly used in Billing module (*section 2.2.7., tables 27, 28, 29, 30, 31, 33*), also in Notifications module functions (*section 2.2.8., table 41*).

Table 53. Fines entity – description

Key	Column name	Data type	Nullable	Other parameters	Description
PK	id	int	No	IDENTITY (1, 1)	Artificial id for fine entry identification
	created_at	datetime	No		Date of creation of the corresponding fine entry
	updated_at	datetime	No		Date of information update of the corresponding fine entry
	sum	money	Yes		The total sum of fine. If NULL, then it's warning.

FK	reason_id	int	No	FK referenced from entity FineReasons (*table 55*, id attribute)	The reason of issuing fine
FK	inhabitant_id	int	No	FK referenced from entity Inhabitants (*table 45*, id attribute)	To whom fine is issued
FK	status_id	int	No	FK referenced from entity FineStatuses (*table 54*, id attribute)	Current fine status

FineStatuses entity and FineReasons entity (*tables 54, 55*) are codifiers tables to Fines entity (*table 53*).

Table 54. FineStatuses entity – description

Key	**Column name**	**Data type**	**Nullable**	**Other parameters**	**Description**
PK	id	int	No	IDENTITY (1, 1)	Artificial id for status of the fine
	name	varchar(100)	No		Name of the status
	description	varchar(100)	No		Description of the status

Table 55. FineReasons entity – description

Key	Column name	Data type	Nullable	Other parameters	Description
PK	id	int	No	IDENTITY (1, 1)	Artificial id for reason of the fine
	name	varchar(100)	No		Name of the reason
	description	varchar(100)	No		Description of the reason

3.1.6. Notifications and corresponding entities

This subsection contains detailed information about table design of Bills, Fines and corresponding entities (*tables 56-58*).

Notifications entity is used in Notifications module functions (*section 2.2.8., tables 34-40*).

Table 56. Notifications entity – description

Key	Column name	Data type	Nullable	Other parameters	Description
PK	id	int	No	IDENTITY (1, 1)	Artificial id for notification entry identification
	created_at	datetime	No		Date of creation of the corresponding notification entry
	updated_at	datetime	No		Date of information update of the corresponding notification entry
	header	text	No		Header of notification
	message	text	No		Body message of notification

FK	type_id			FK referenced from entity NotificationTypes (*table 57*, id attribute)	Type of notification
FK	language_id			FK referenced from entity Languages (*table 58*, id attribute)	Language of notification
FK	user_id			FK referenced from entity Users (*table 43*, id attribute)	Creator of notification

NotificationTypes entity and Languages entity (*tables 57, 58*) are codifiers tables to Notifications entity (*table 56*).

Table 57. NotificationTypes entity – description

Key	Column name	Data type	Nullable	Other parameters	Description
PK	id	int	No	IDENTITY (1, 1)	Artificial id for type of the notification
	name	varchar(100)	No		Name of the type
	description	varchar(100)	No		Description of the type

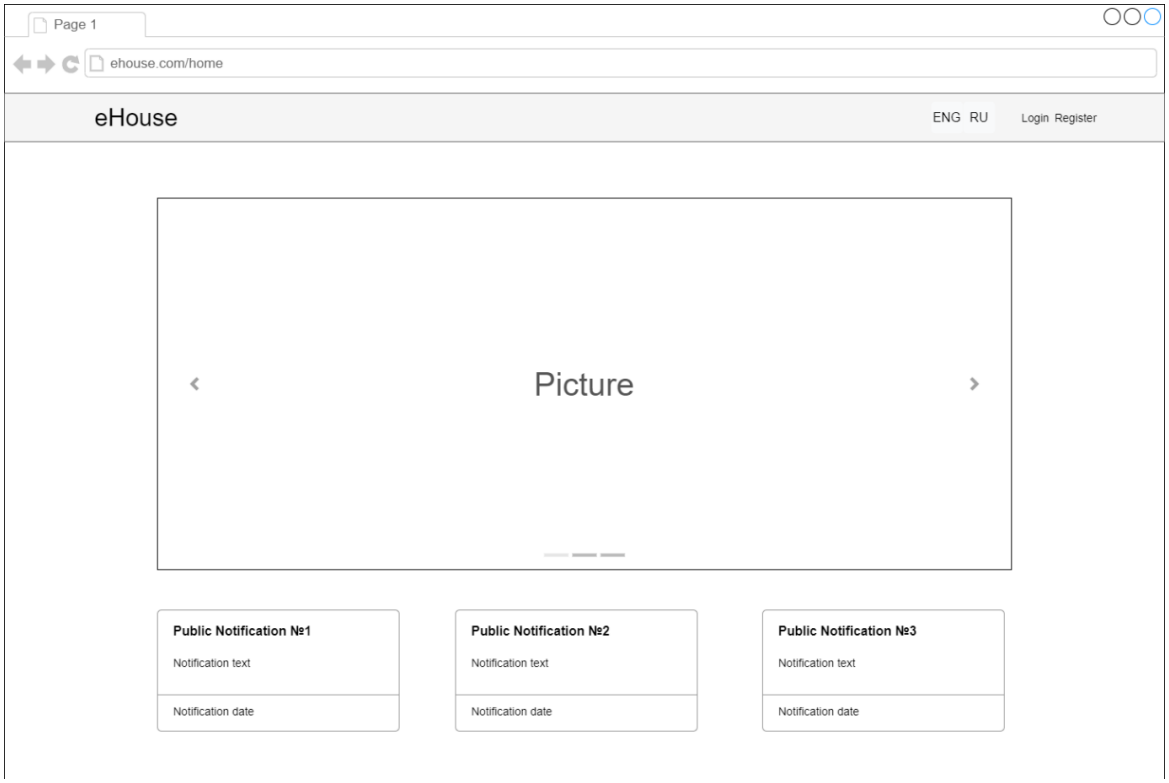
Table 58. Languages entity – description

Key	Column name	Data type	Nullable	Other parameters	Description
PK	id	int	No	IDENTITY (1, 1)	Artificial id for language identification

	name	varchar(100)	No		Name of the language
	language_code	varchar(2)	No		Language code according to ISO 639-1:2002 standard [1]

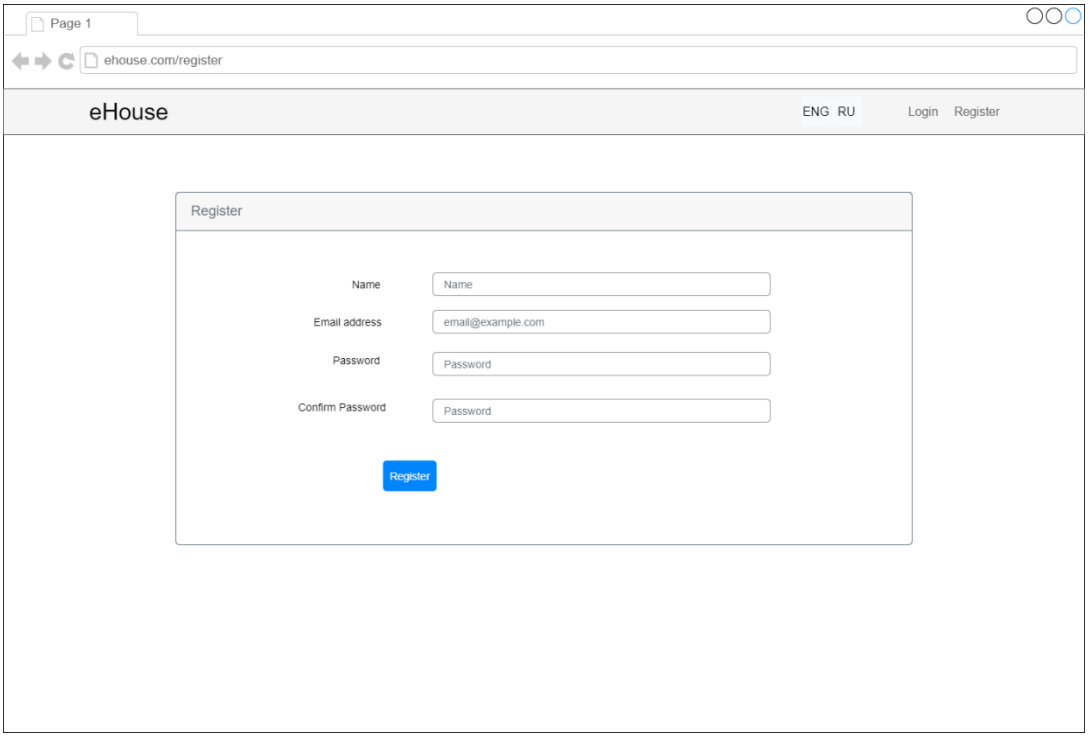
3.2. Partial Design of User Interfaces

The main page (homepage) (*pic. 14*) includes public notifications, generated by function in Notification module (*section 2.2.8., table 34*). From this page it's possible to either register or login into the system (*section 2.2.3., tables 2-3*). In each page there is a language change, which is non-functional requirement described in *section 2.3*. The very same page is available for User role (according to roles described in *section 1.5.*) after login to the system.



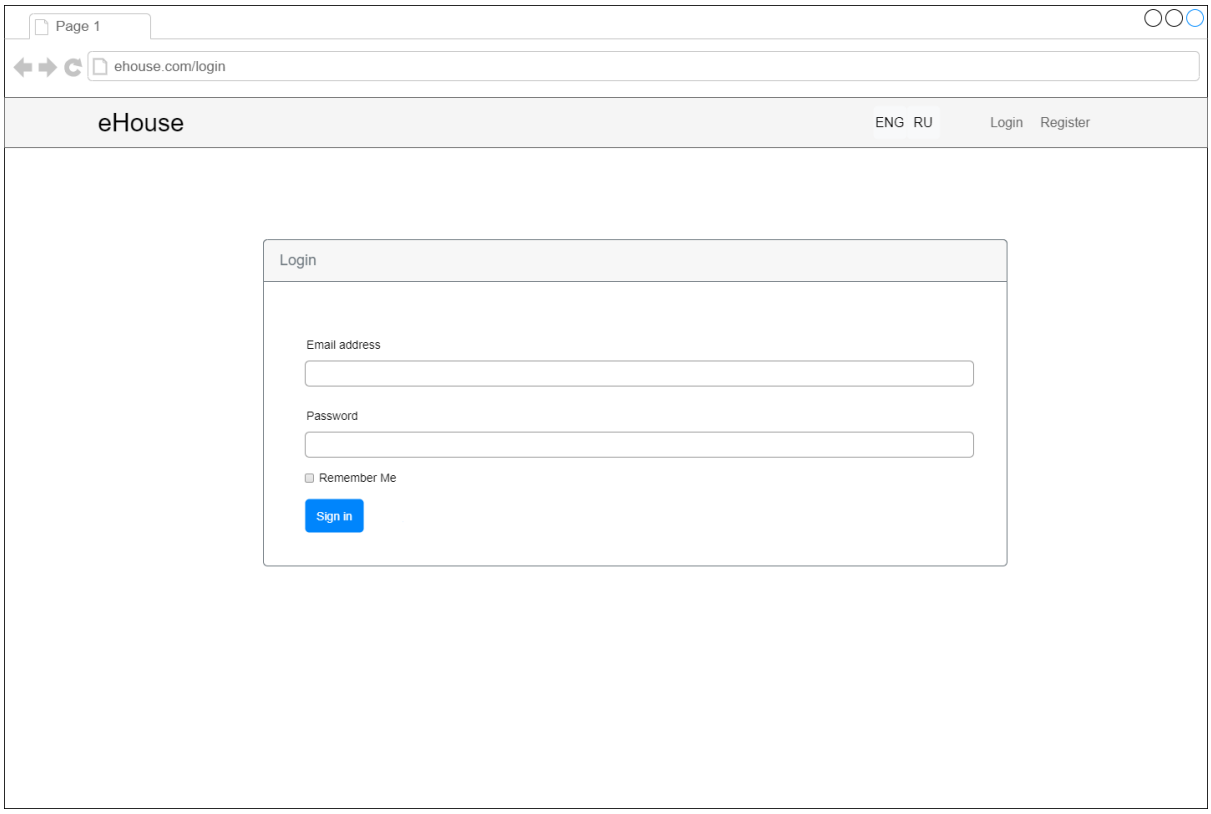
Pic. 14. Main page (Guest)

Registration form (*pic. 15*) includes fields required by functional requirement in *section 2.2.3., table 2.*



Pic. 15. Registration form

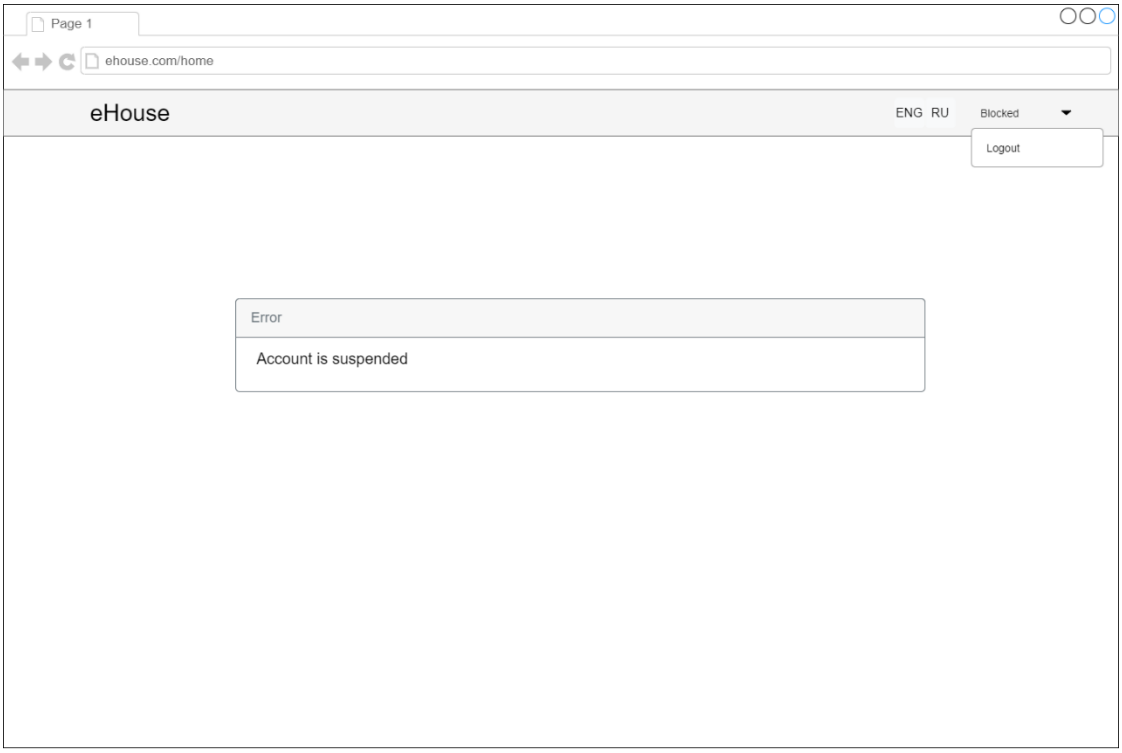
Login form (*pic. 16*) includes fields required by functional requirement in *section 2.2.3.*, *table 3.*



Pic. 16. Login form

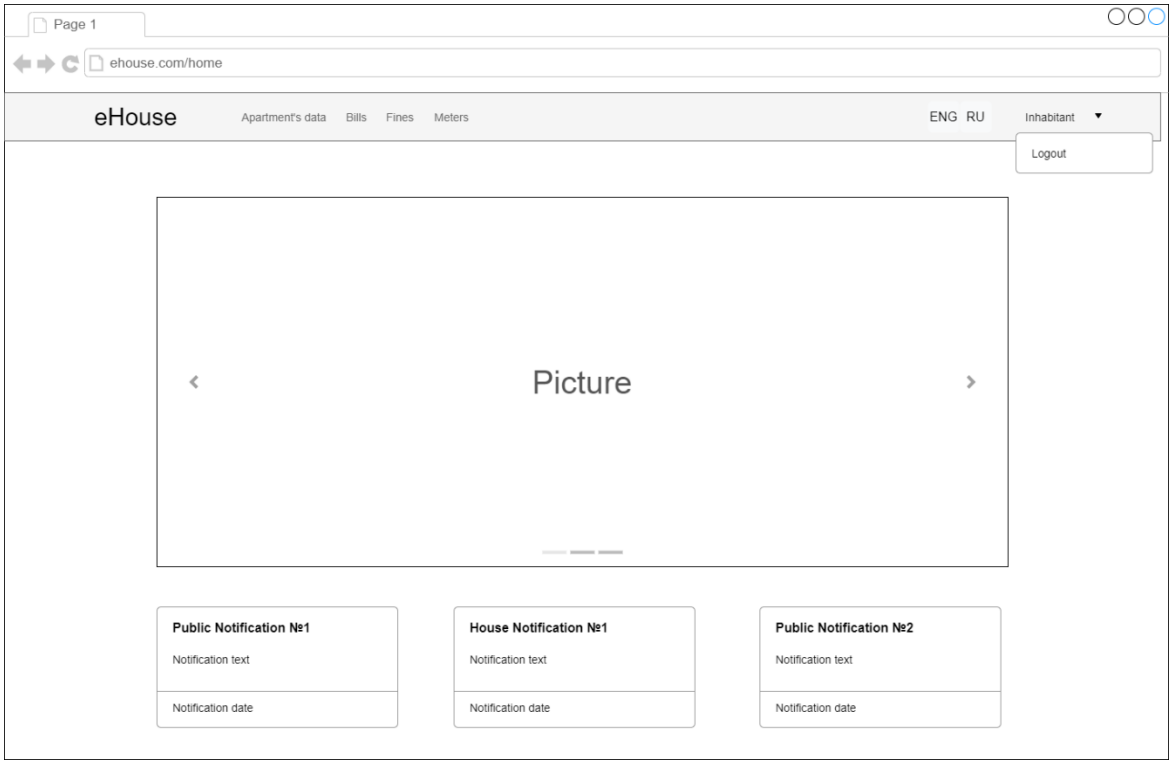
After login to the system Blocked user (according to roles described in *section 1.5*.) will get error message (*pic. 17*) based on functional requirement in *section 2.2.3.*, *table 3*.

Logout functional is processed by the function in *section 2.2.3.*, *table 3a* and is the same for all frames.



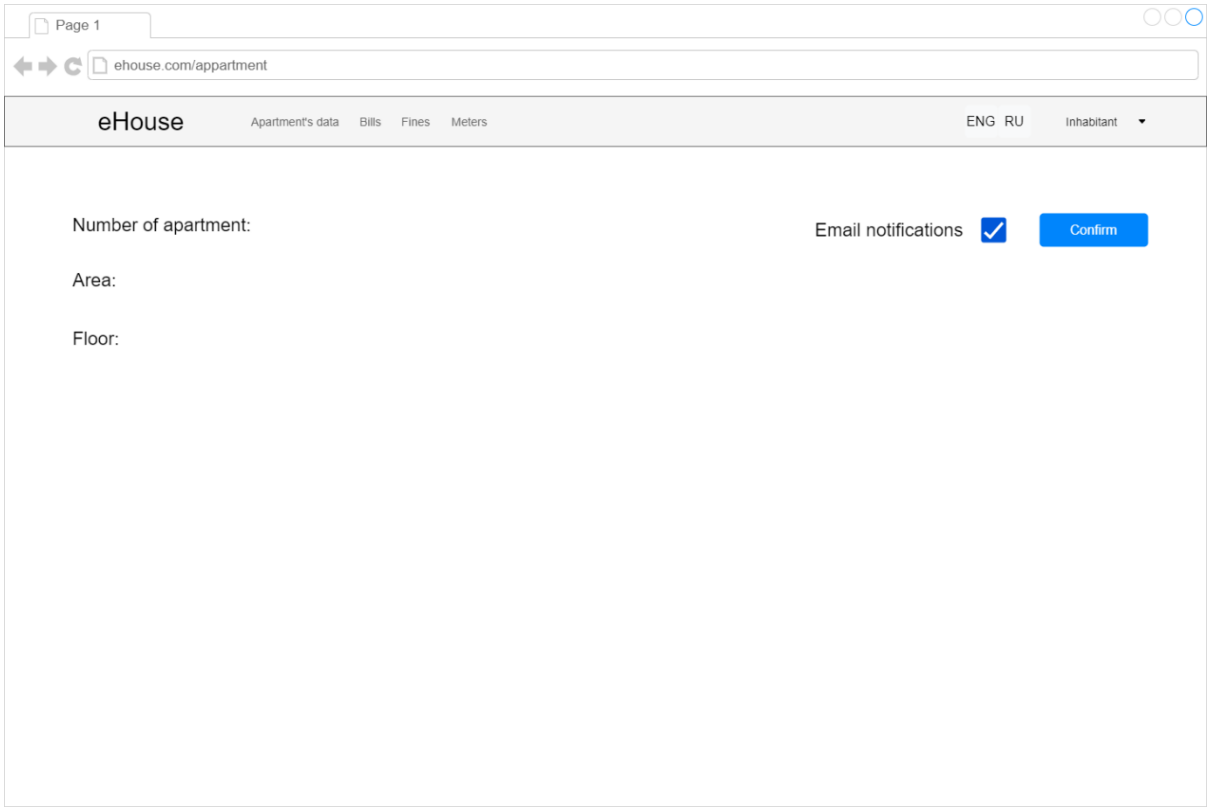
Pic. 17. Error message for Blocked user role

Main functionality of the application available via top menu bar, which is slightly differ for each user role. Main page for Inhabitant is shown at the *pic. 18*. It includes public notifications, generated by function in Notification module (*section 2.2.8.*, *table 34*) as well as house notifications (*section 2.2.8.*, *table 35*).



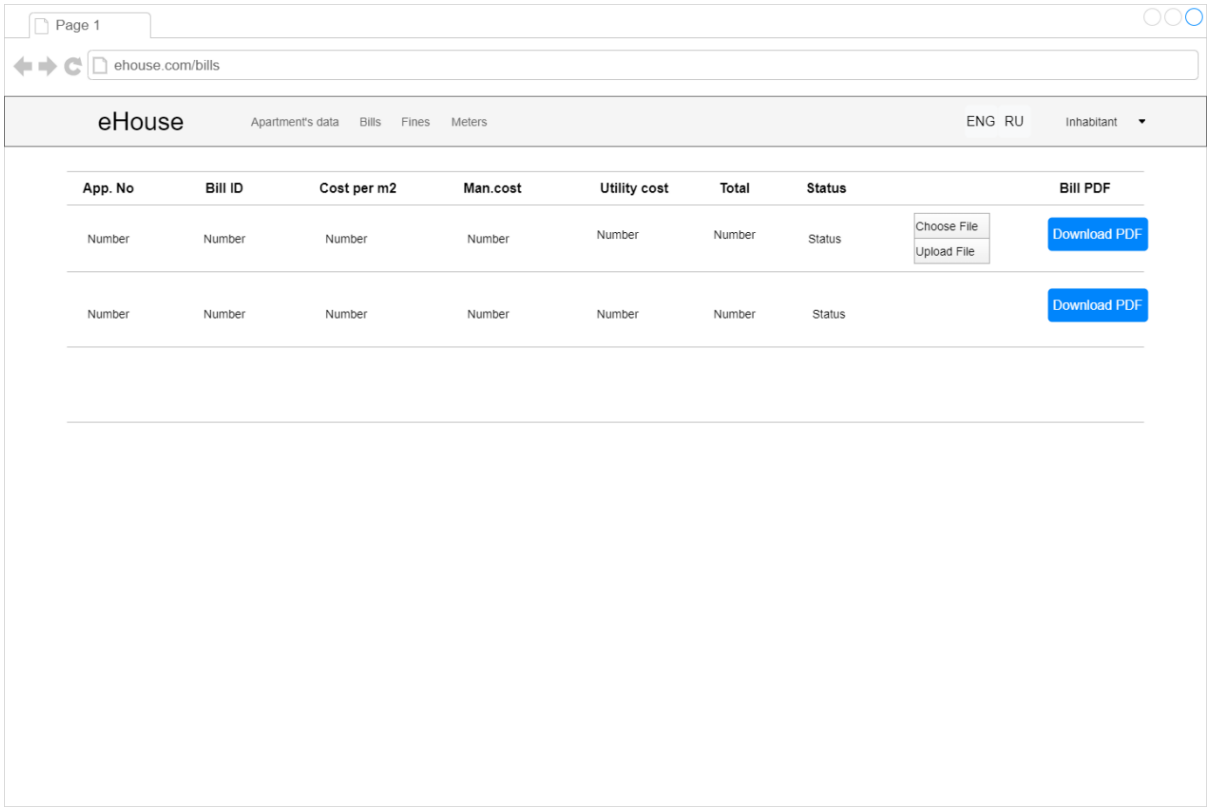
Pic. 18. Main page (Inhabitant)

Apartment data generated by function described in *section 2.2.4.*, *table 6* and email notification enabling/disabling function described in *section 2.2.8.*, *table 42* are accessible via “Apartment’s data” in the top menu (*pic. 19*).



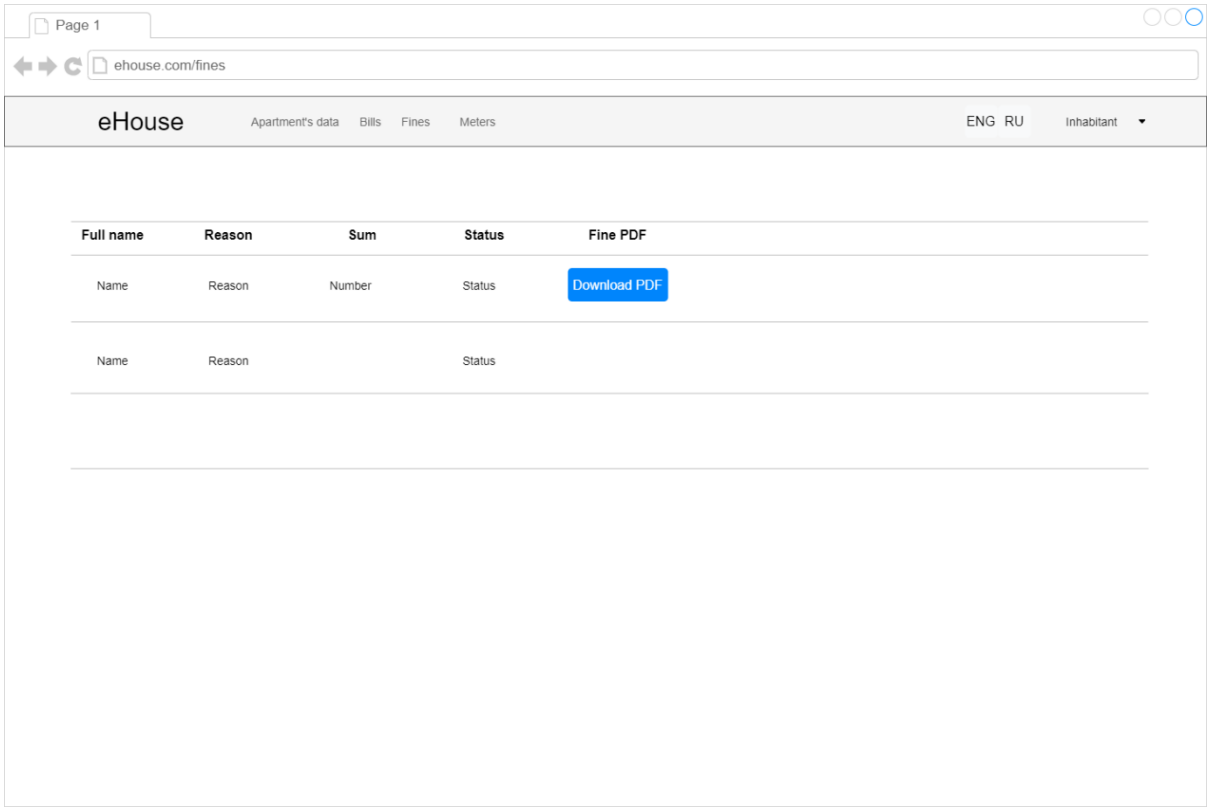
Pic. 19. Apartment's data

"Bills" in the top menu (*pic. 20*) shows information about bills, that tenant of the house has for his apartment(s) generated by function in *section 2.2.7*, *table 22*. It's also utilizes functions of invoice download (*section 2.2.7*, *table 23*) and payment confirmation upload (*section 2.2.7*, *table 24*).



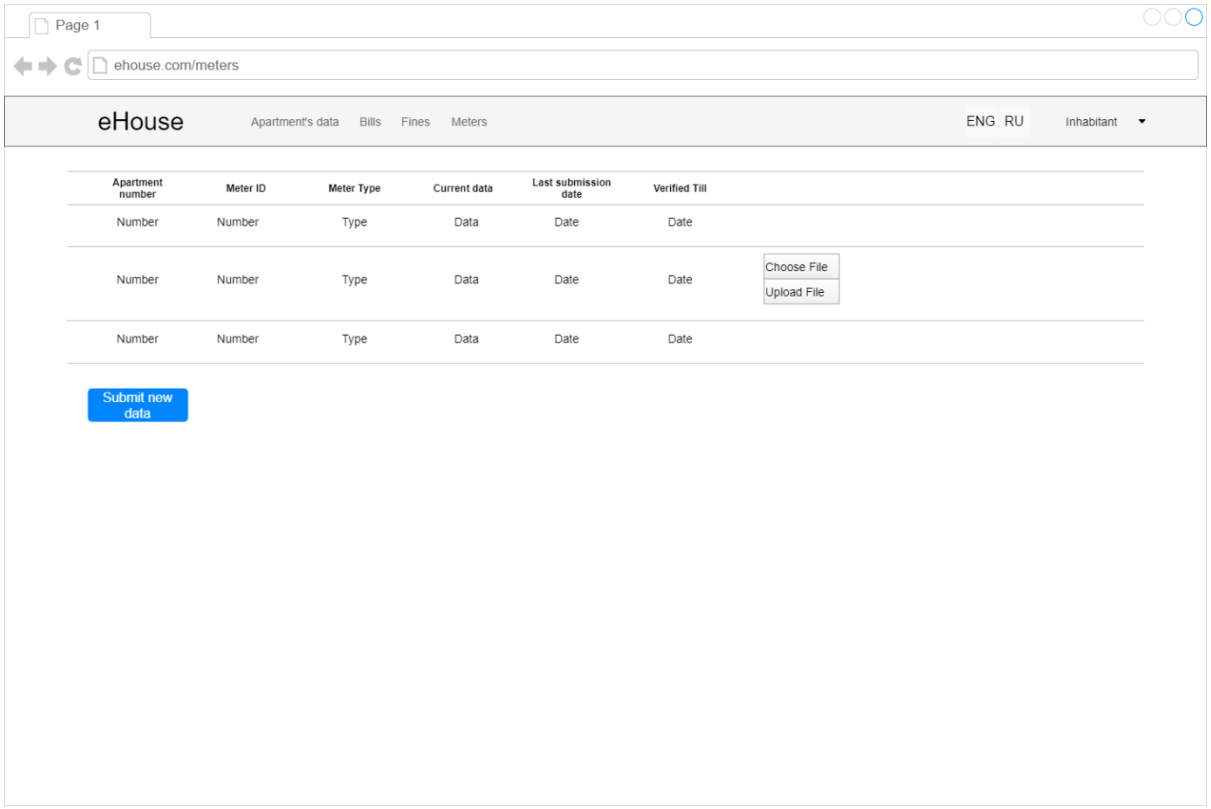
Pic. 20. Bills

"Fines" in the top menu (*pic. 21*) shows information about fines, that tenant of the house has for himself generated by function in *section 2.2.7*, *table 31*. It's also utilizes functions of invoice download (*section 2.2.7*, *table 32*).



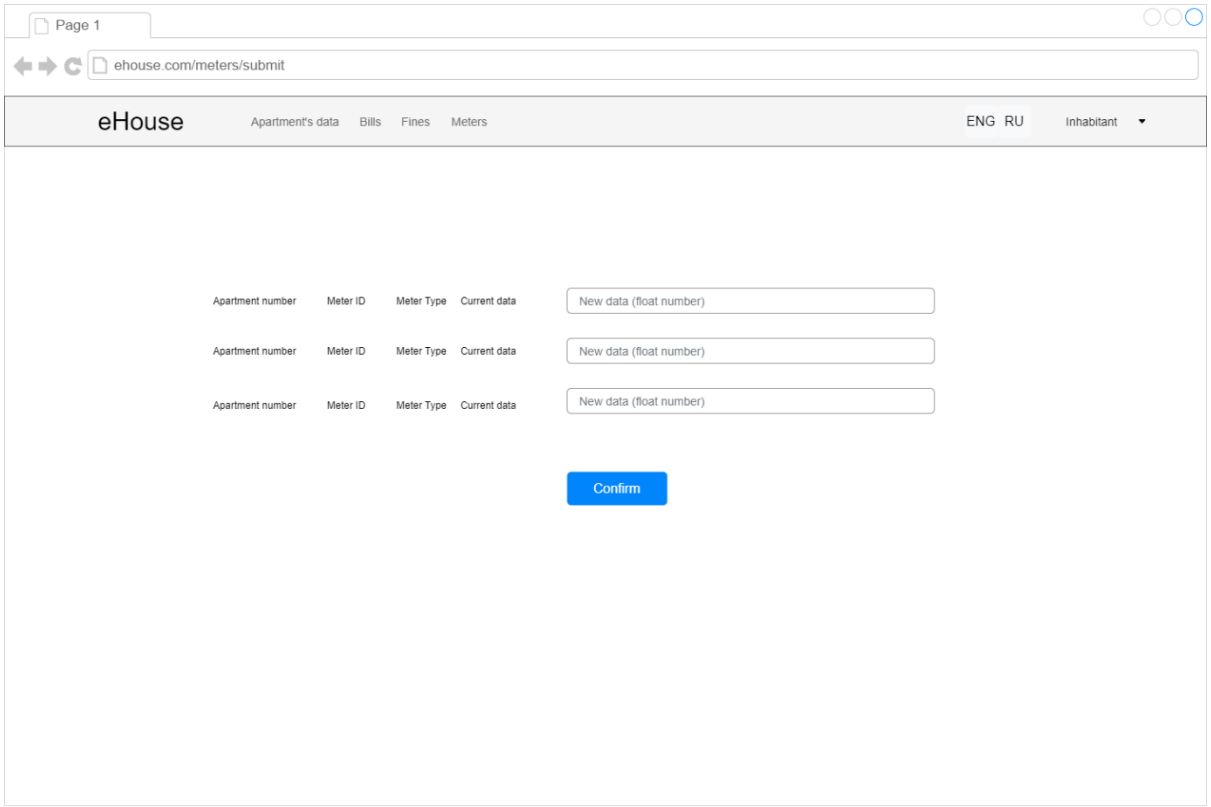
Pic. 21. Fines

"Meters" in the top menu (*pic. 22*) shows information about meters for all apartments that tenant has and generated by function in *section 2.2.6, table 13.* It's also utilizes functions of verification report upload (*section 2.2.6, table 16*).



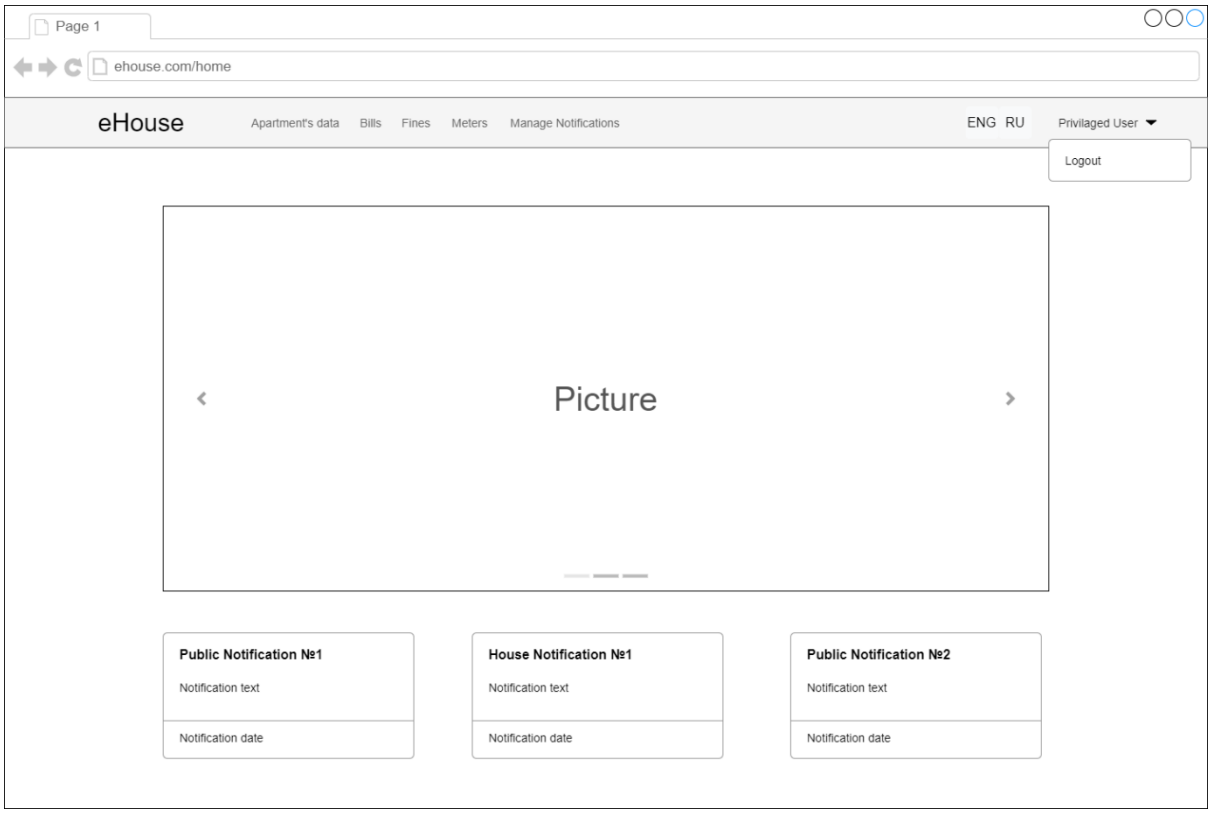
Pic. 22. Meters

After going to “Submit new data” in pic. 22 data submission page will be opened (pic. 23). The submission processing described in *section 2.2.6*, *table 15.*

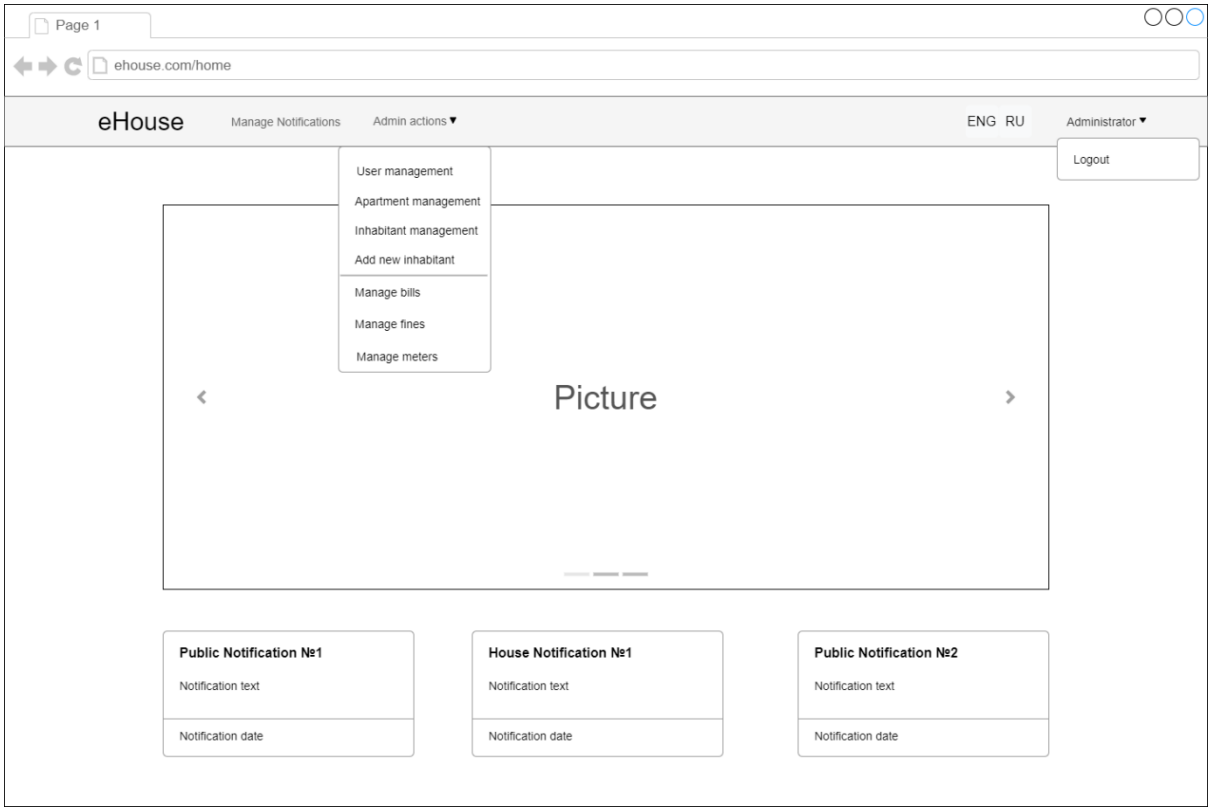


Pic. 23. Meters data submission

Privileged user has additional function in top menu bar “Manage notifications” (*pic. 24*, other functions are shared with inhabitant). Admin has “Manage notifications” and list of admin-only actions (*pic. 25*). They both showed as main pages.

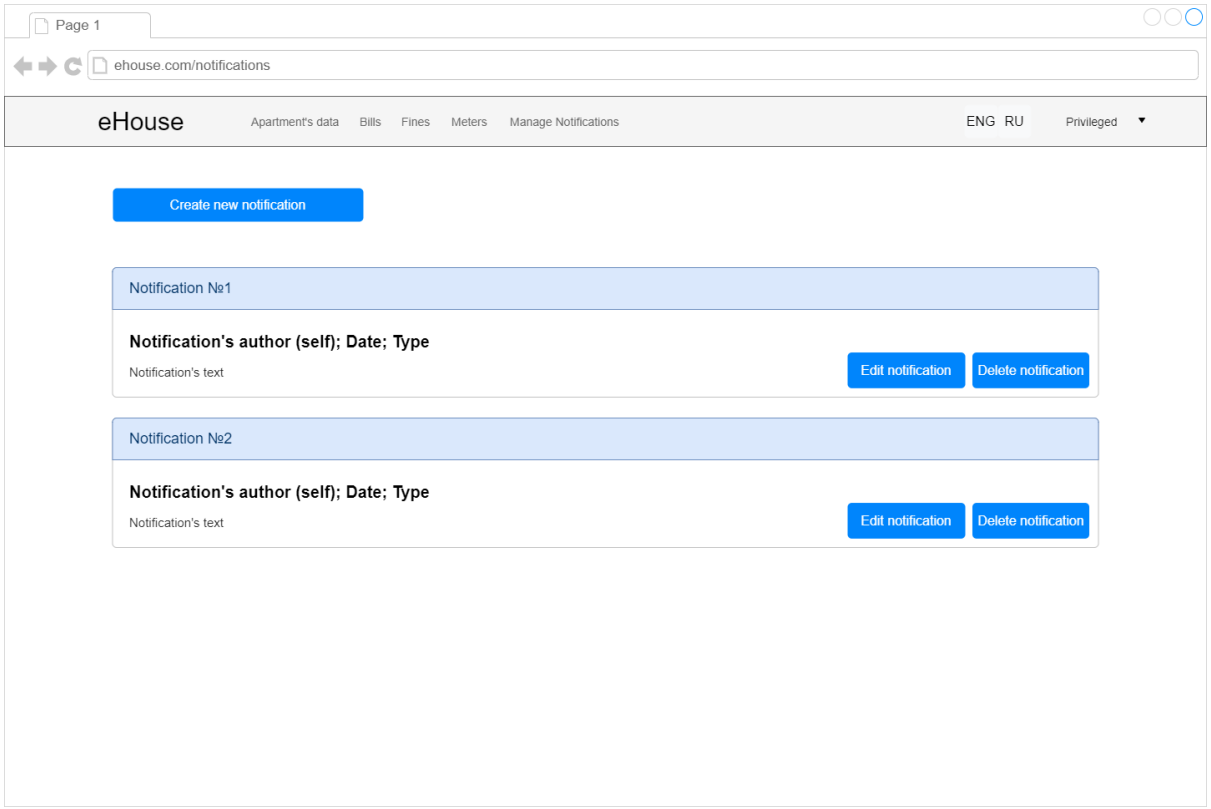


Pic. 24. Main page (Privileged user)

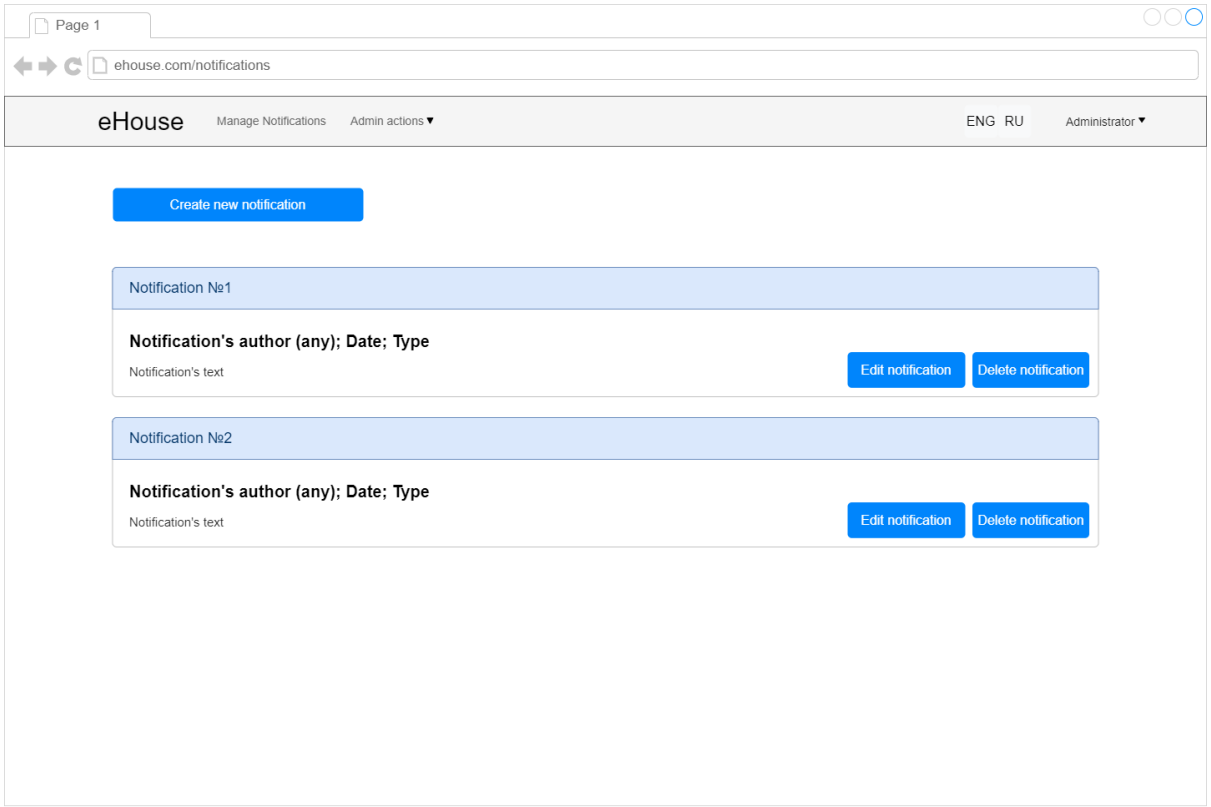


Pic. 25. Main page (Admin)

On *pics. 26, 27* shown pages in “Manage Notifications” for Privileged user (*pic. 26*) and Admin (*pic. 27*). Privileged user utilizes functions described in *section 2.2.8.*, *tables 38, 39*, while admin also uses functions described in *section 2.2.8.*, *table 37* and *table 40*. Lists of notifications built based on functions from *section 2.2.8.*, *table 34* and *table 35.*

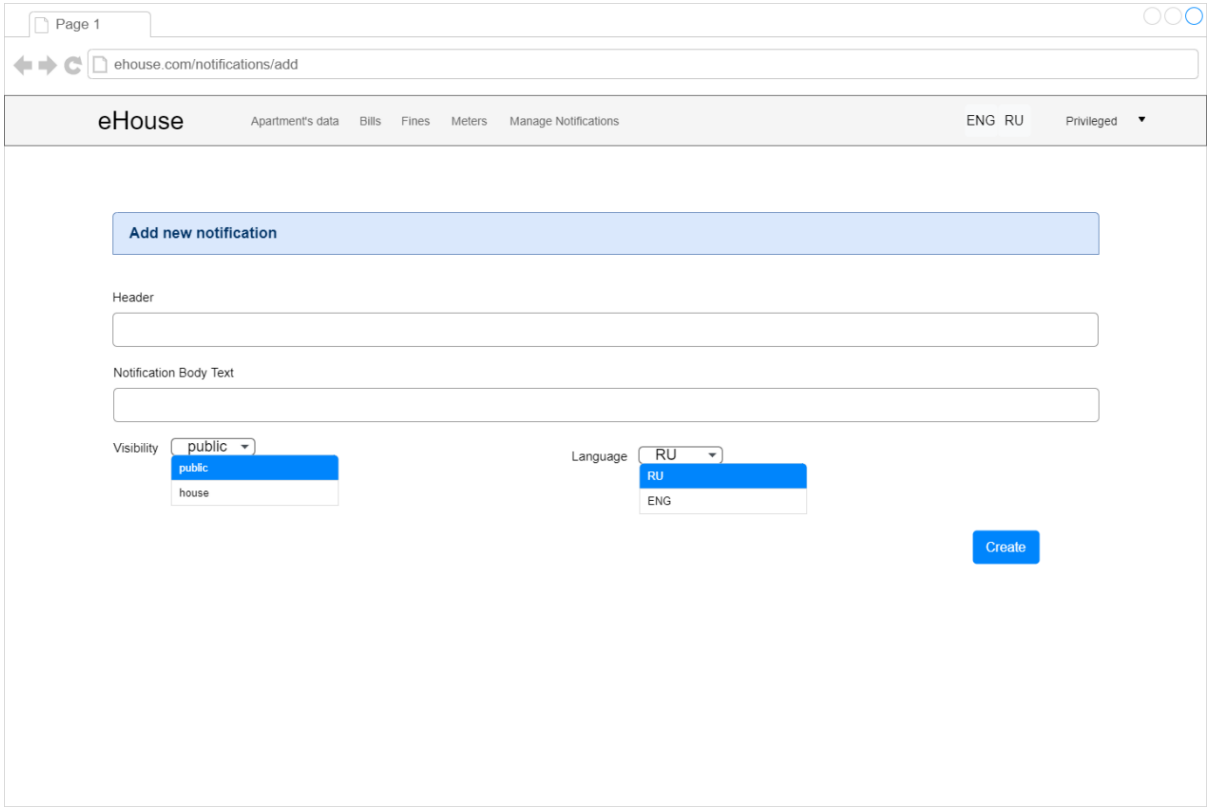


Pic. 26. Announcements (Privileged user)



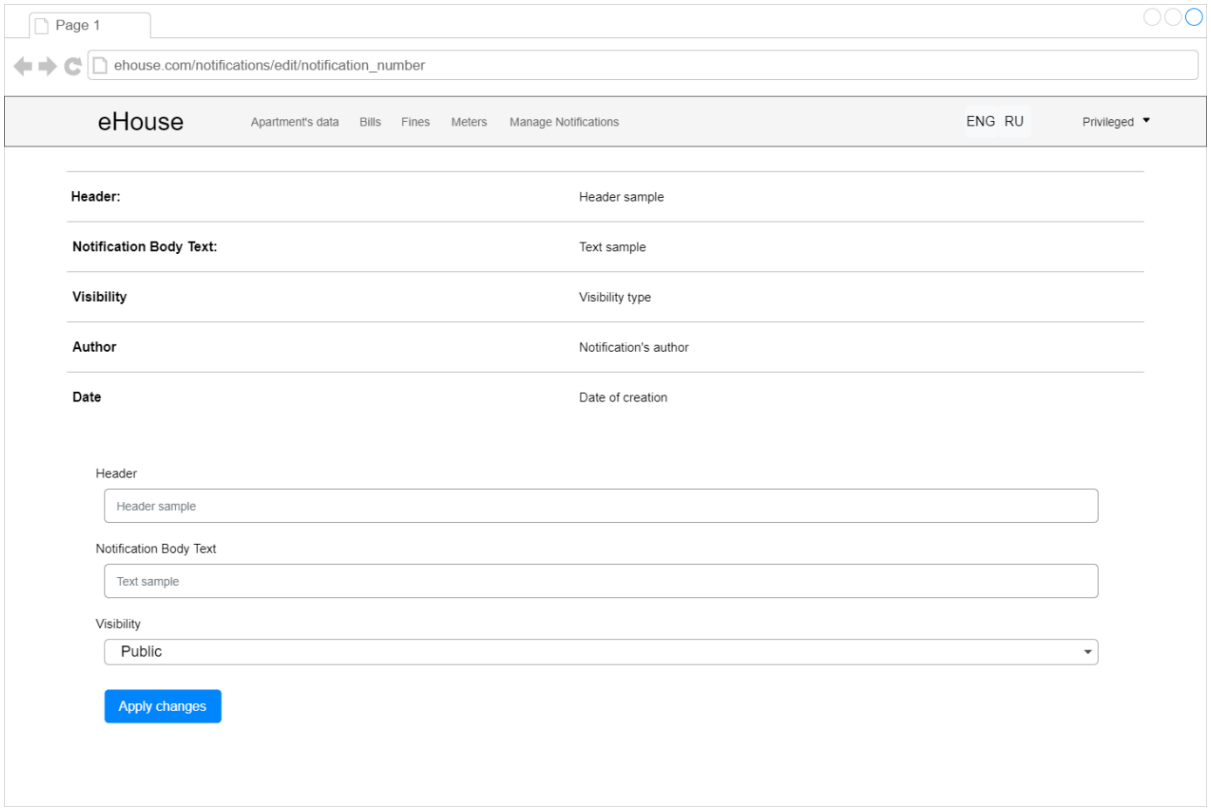
Pic. 27. Announcements (Admin)

New announcement created the same way for both Admin and Privileged user (*pic. 28*) and processed by functions described in *section 2.2.8.*, *table 36.*



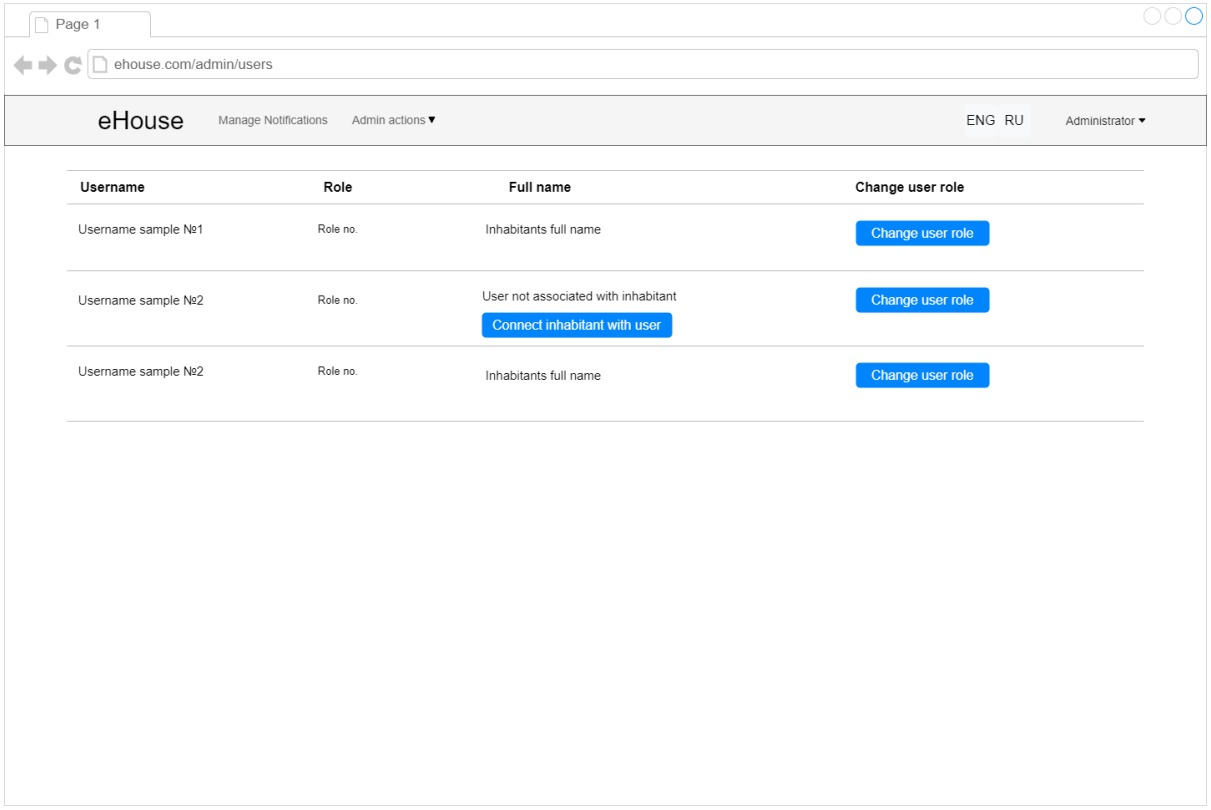
Pic. 28. New announcement

Edition of notifications (*pic. 29*) done by function described in *section 2.2.8.*, *table 39*, while admin also uses function described in *table 40.*



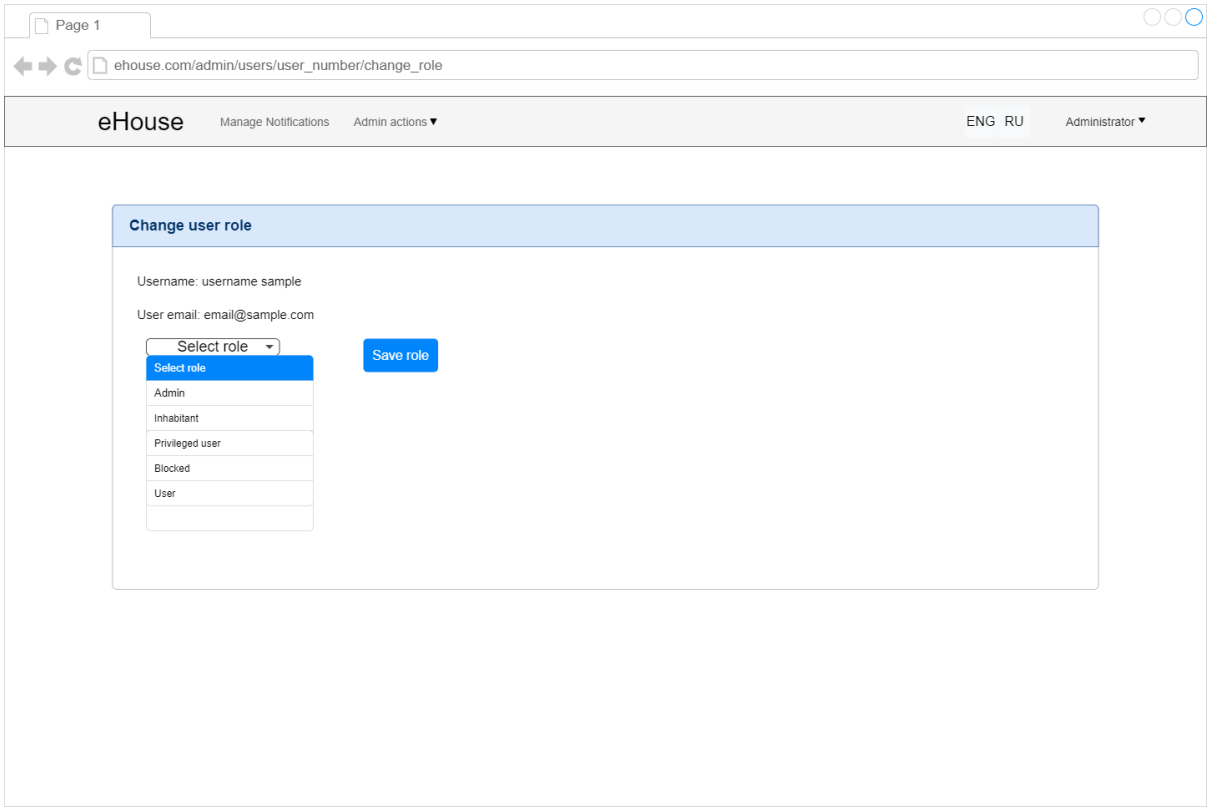
Pic. 29. Announcement edit

Admin can retrieve user list (*pic. 30*) based on function in *section 2.2.3.*, *table 4*.



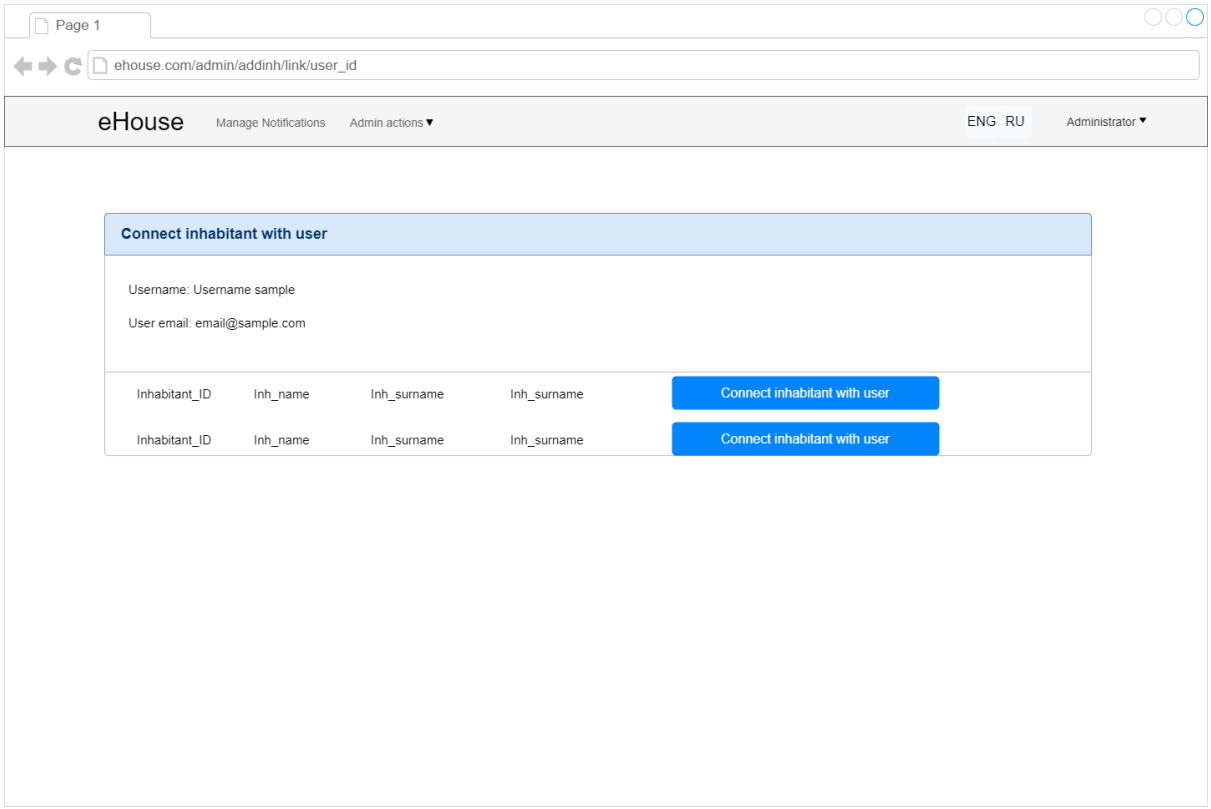
Pic. 30. User list (Admin)

User role change showed at the pic. 31 and processed by function in *section 2.2.3.*, *table 5.*



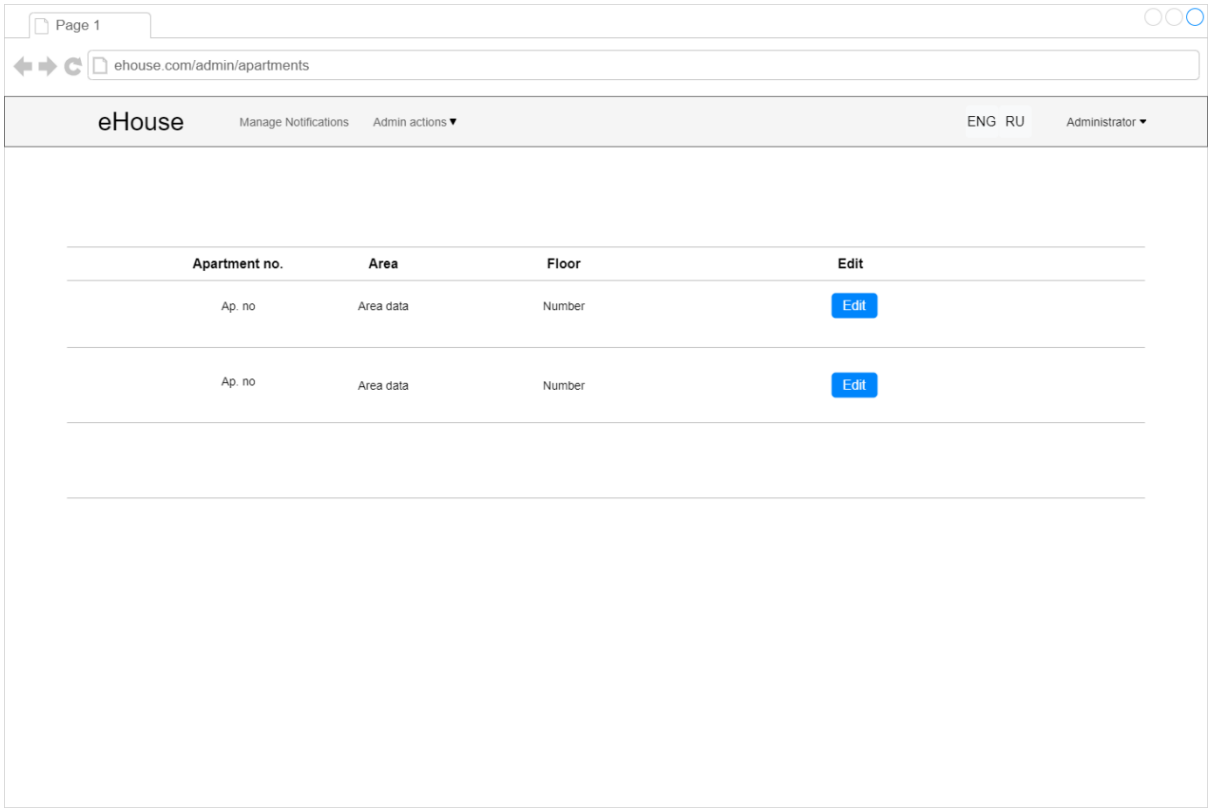
Pic. 31. User role change (Admin)

Page of connecting user and inhabitant showed the *pic. 32* and processed by function in *section 2.2.5.*, *table 12.*



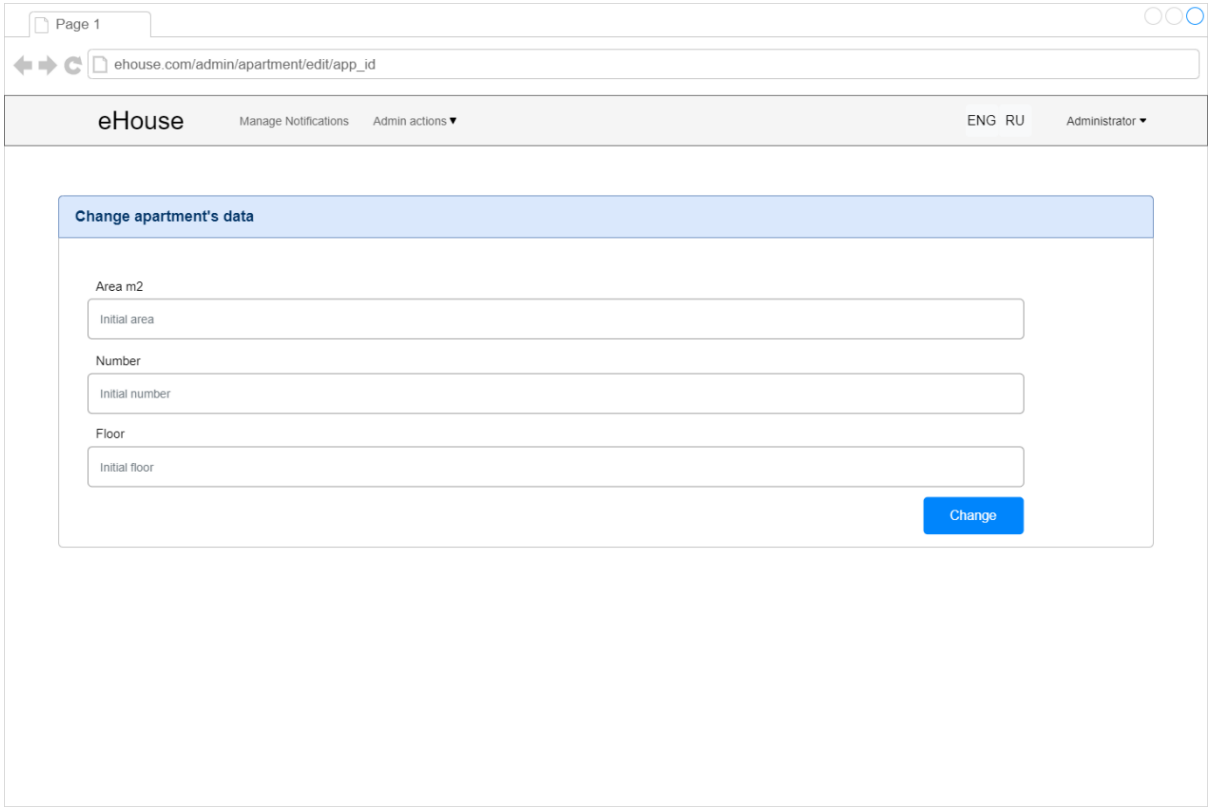
Pic. 32. Linking inhabitant to user account (Admin)

Admin can retrieve list of all apartments shown the *pic. 33* and processed by function in *section 2.2.4., table 7.*



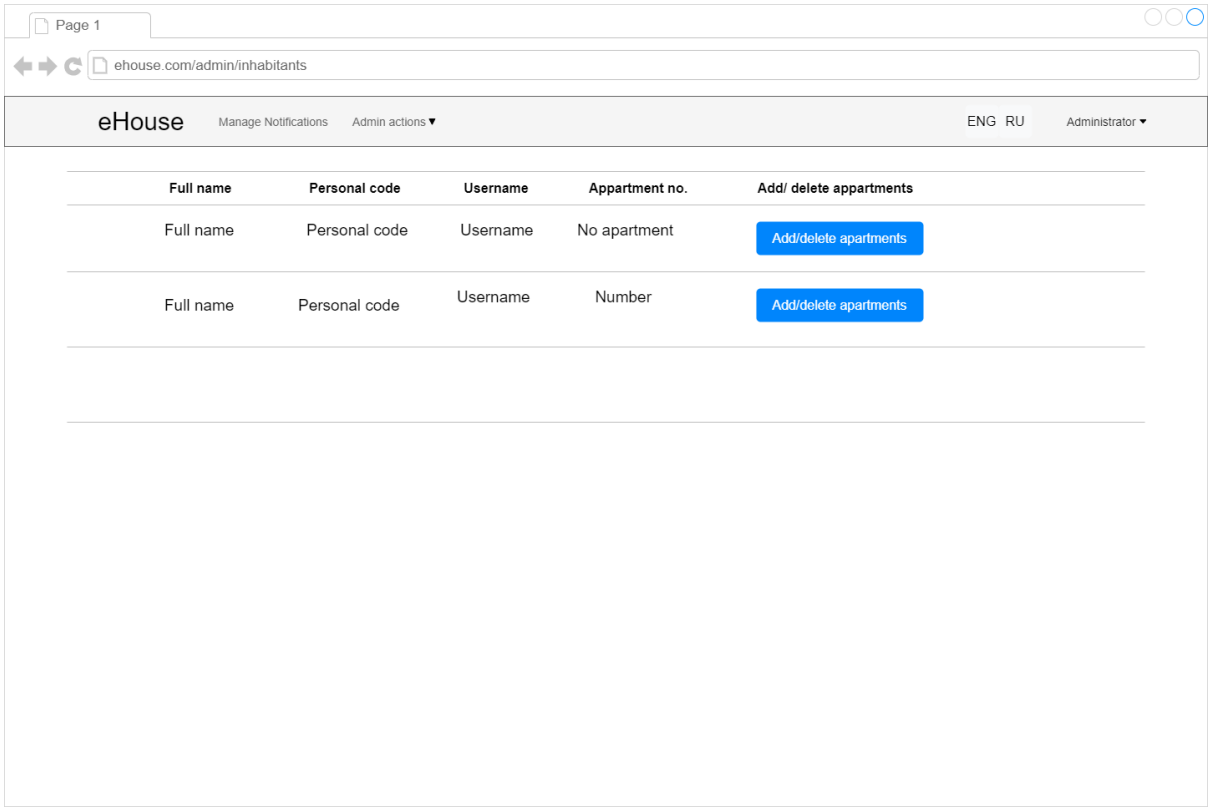
Pic. 33. List of all apartments (Admin)

Admin can edit data about apartment. It’s shown at the *pic. 34* and processed by function in *section 2.2.4., table 8.*



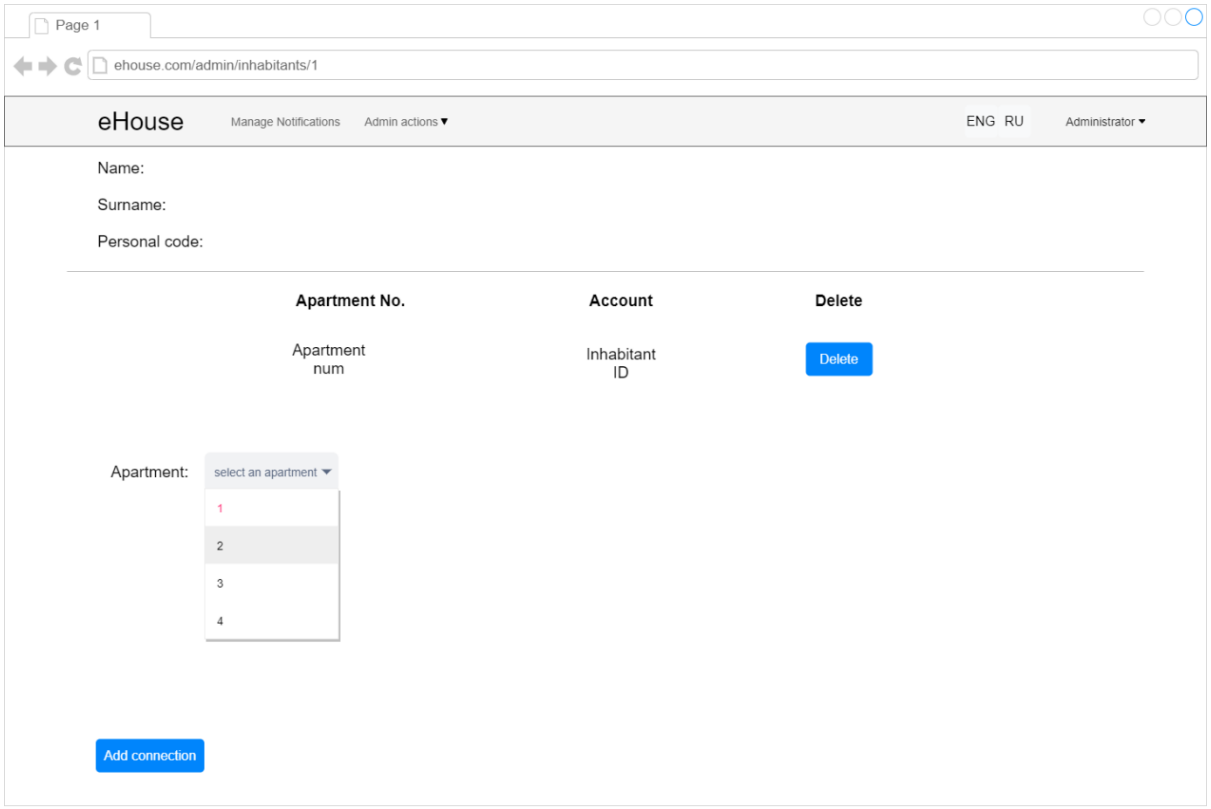
Pic. 34. Edit apartment’s data (Admin)

Admin can retrieve list of all inhabitants shown the *pic. 35* and processed by function in *section 2.2.5., table 10.*



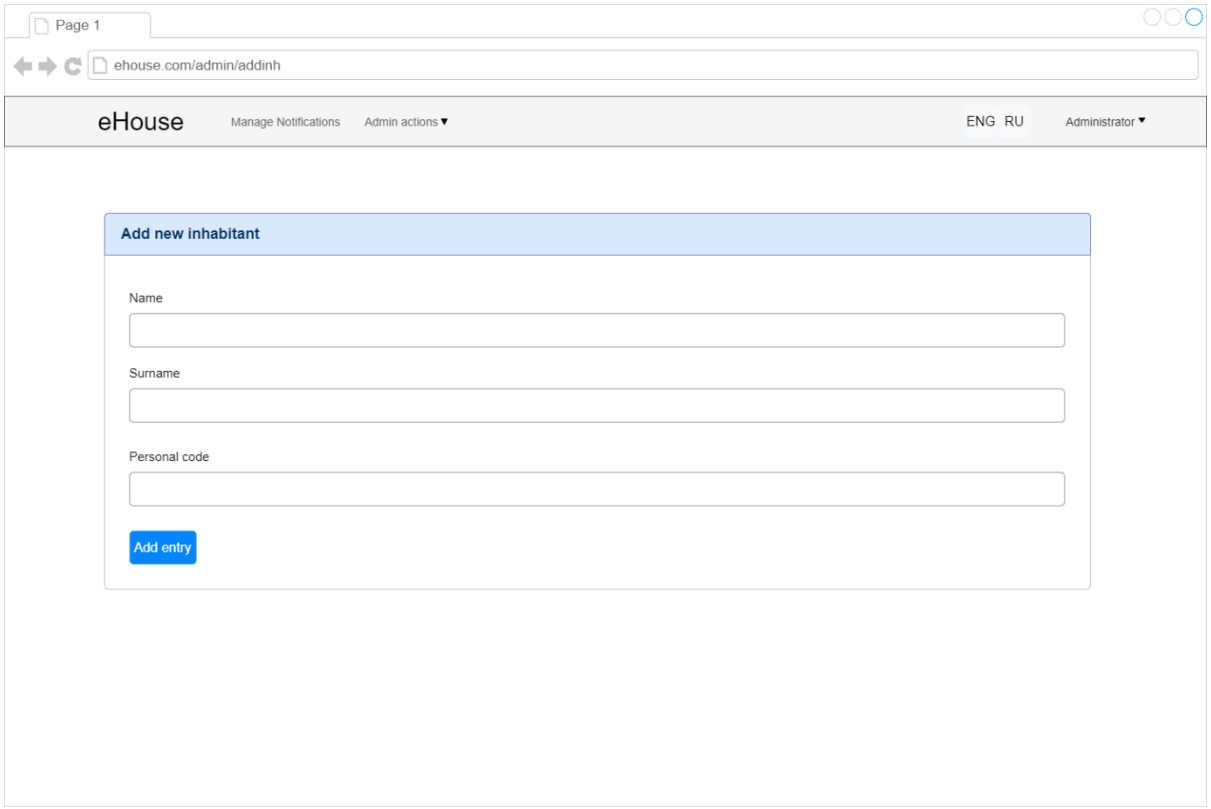
Pic. 35. List of all inhabitants (Admin)

Page of connecting inhabitant to apartment(s) shown at the *pic. 36* and processed by function in *section 2.2.5.*, *table 11*. Deleting functionality proceeded by the very same function.



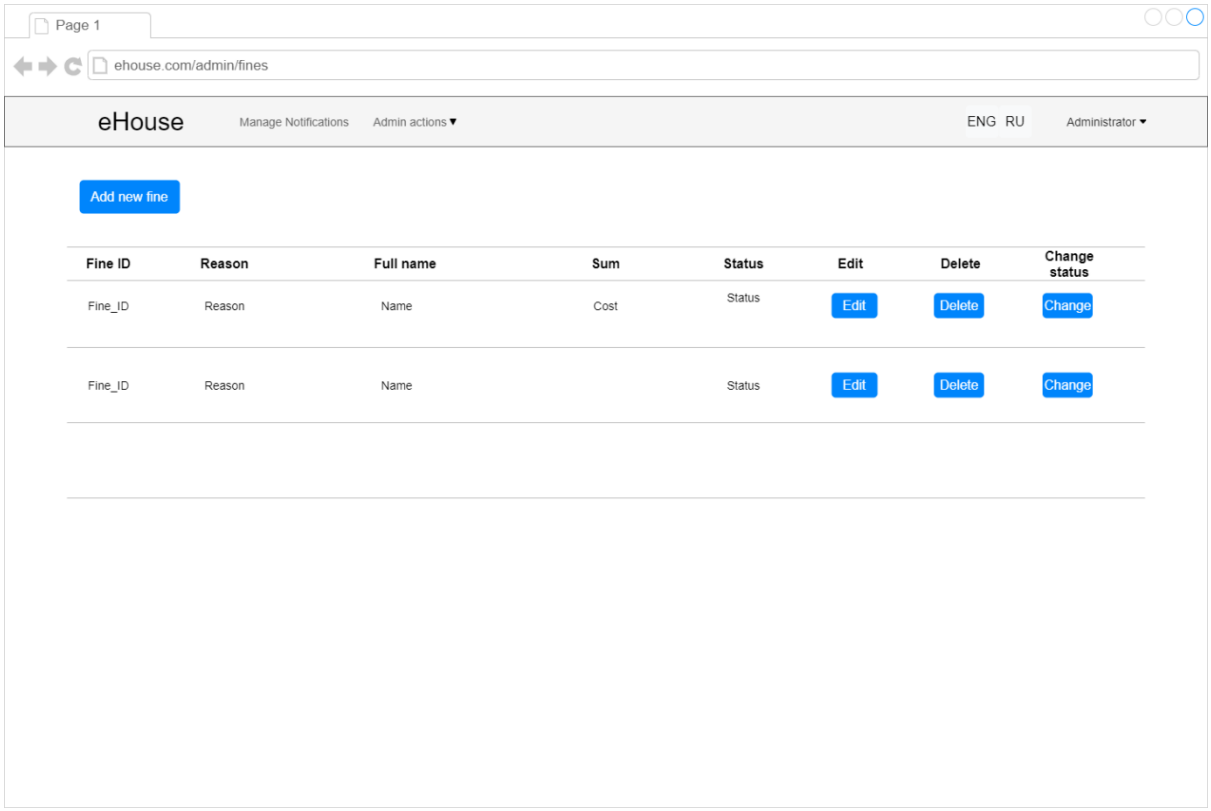
Pic. 36. Linking inhabitant to property (Admin)

Page of creating new inhabitant entry shown at the *pic. 37* and processed by function in *section 2.2.5.*, *table 9.* The process of adding phone number (according to *section 3.1.2.*, *table 46*) to inhabitant could be utilized by the same function (*section 2.2.5.*, *table 9*) and the same view (*pic. 37*). As being the same as other similar parts – not shown explicitly.



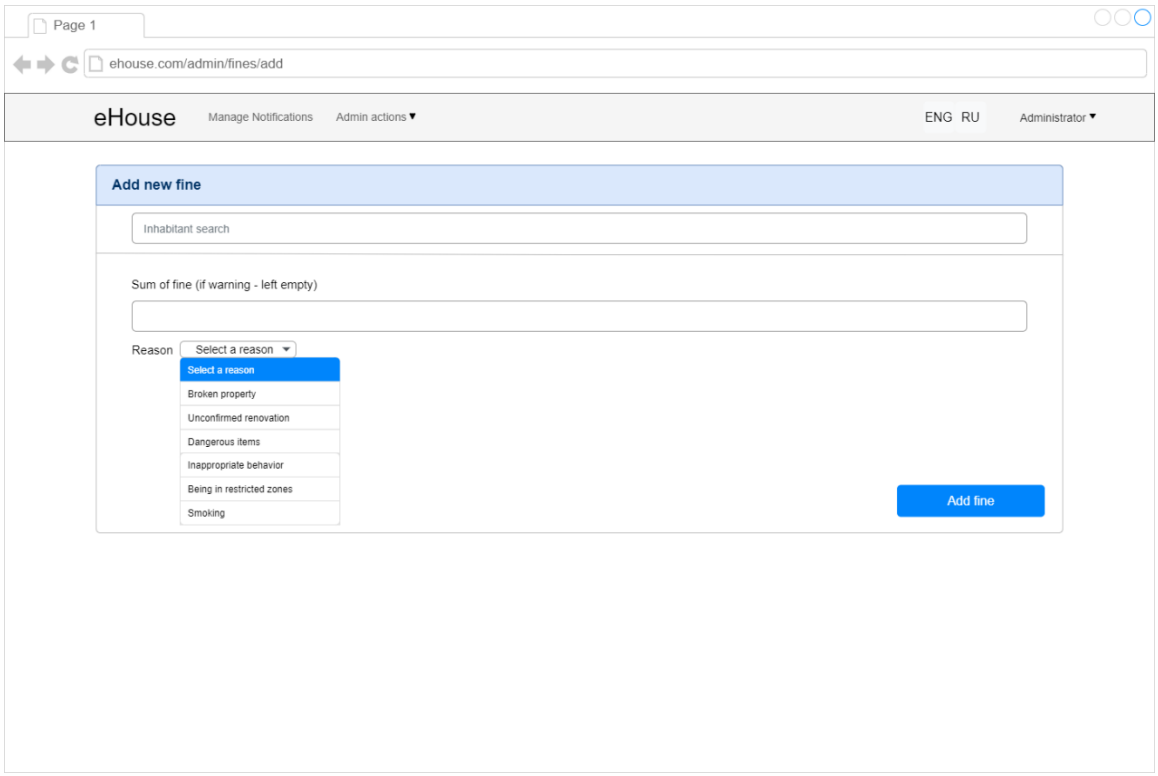
Pic. 37. New inhabitant (Admin)

Admin can retrieve list of all fines shown the *pic. 38* and processed by function in *section 2.2.7.*, *table 30.* Page also utilizes delete functionality described in the same section, *table 29.*



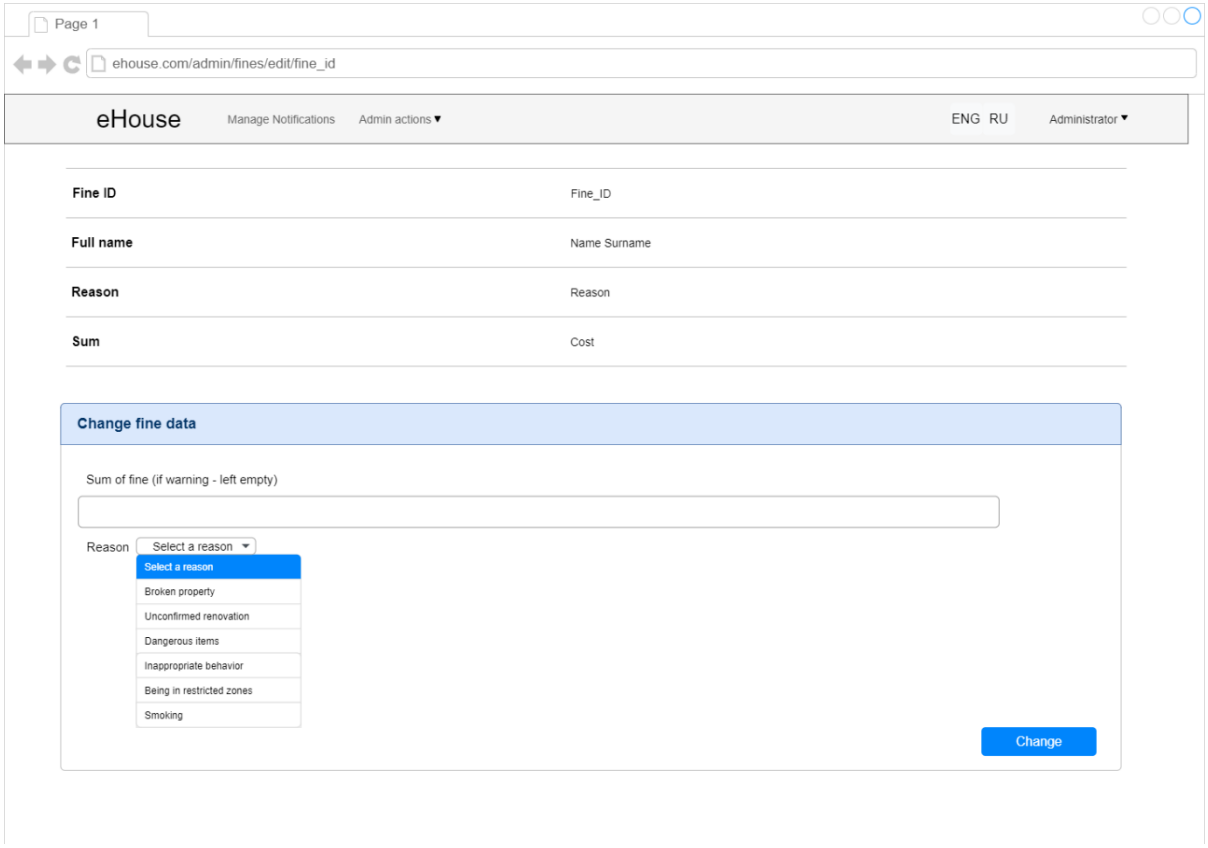
Pic. 38. Fines (Admin)

New fine can be created by admin (*pic. 39*) and proceeded by function in *section 2.2.7., table 27.*



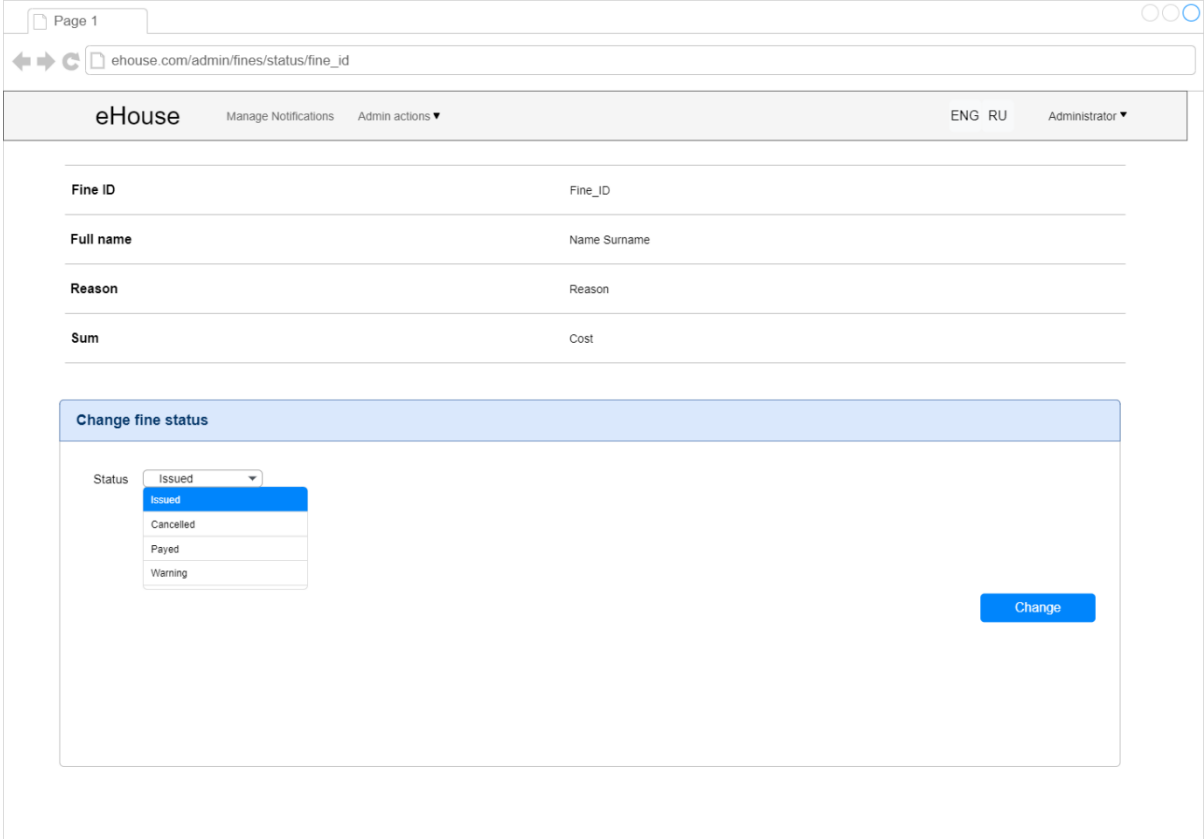
Pic. 39. New fine (Admin)

Admin can also edit already created fine (*pic. 40*). It's proceeded by function in *section 2.2.7., table 28.*



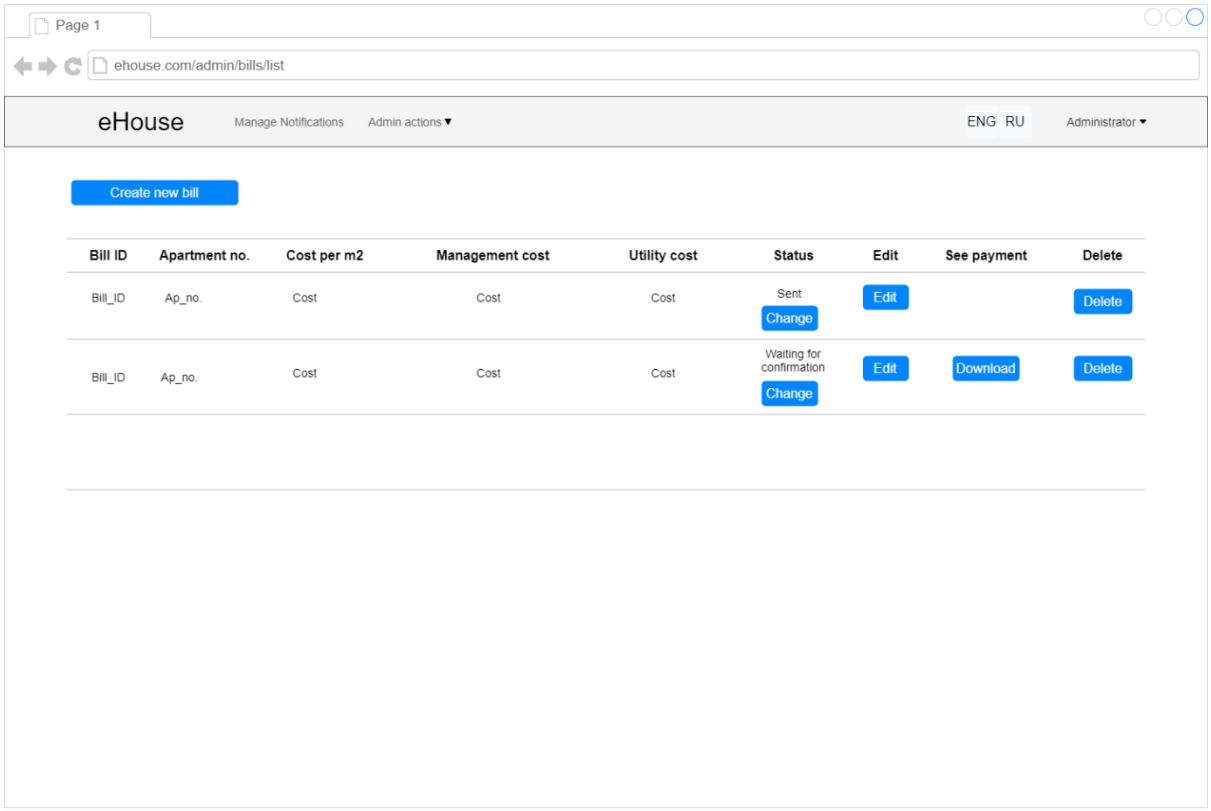
Pic. 40. Edit fine (Admin)

Admin can change status of the already created fine (*pic. 41*). It's proceeded by function in *section 2.2.7., table 33.*



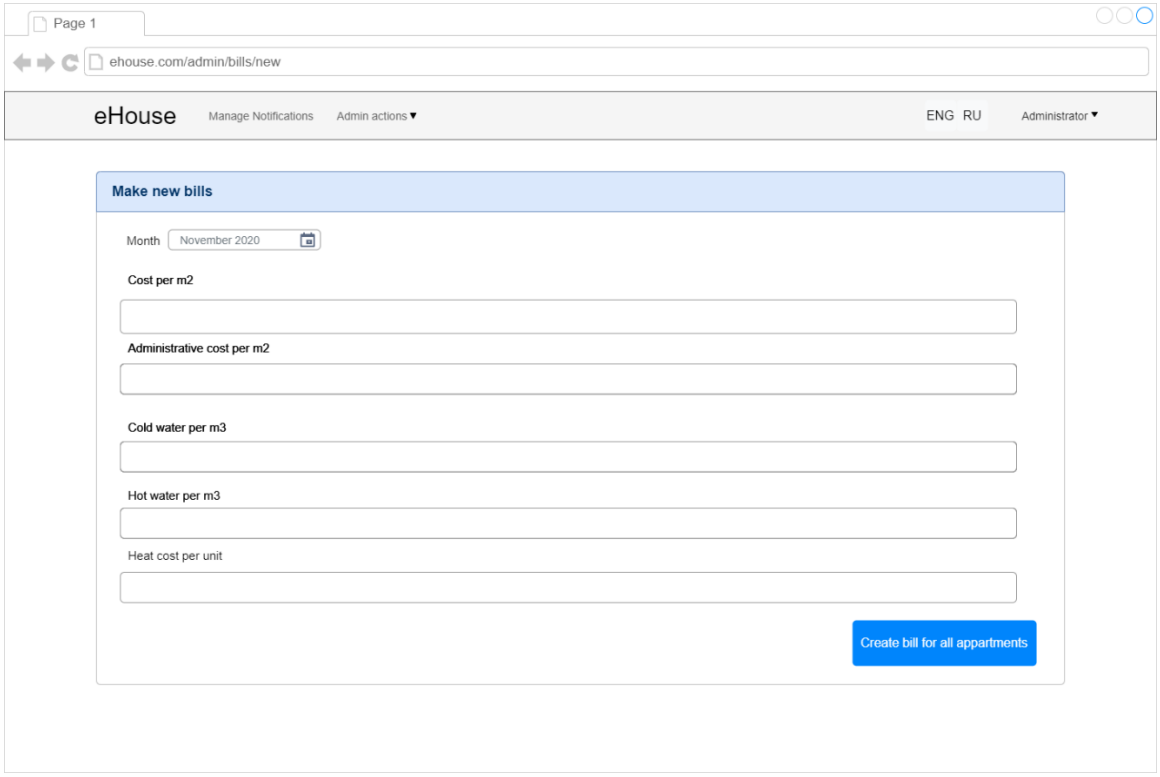
Pic. 41. Change fine status (Admin)

Admin can retrieve list of all bills shown the *pic. 42* and processed by function in *section 2.2.7.*, *table 18.* Page also utilizes delete functionality described in the same section, *table 21* and payment confirmation download (*section 2.2.7.*, *table 25*).



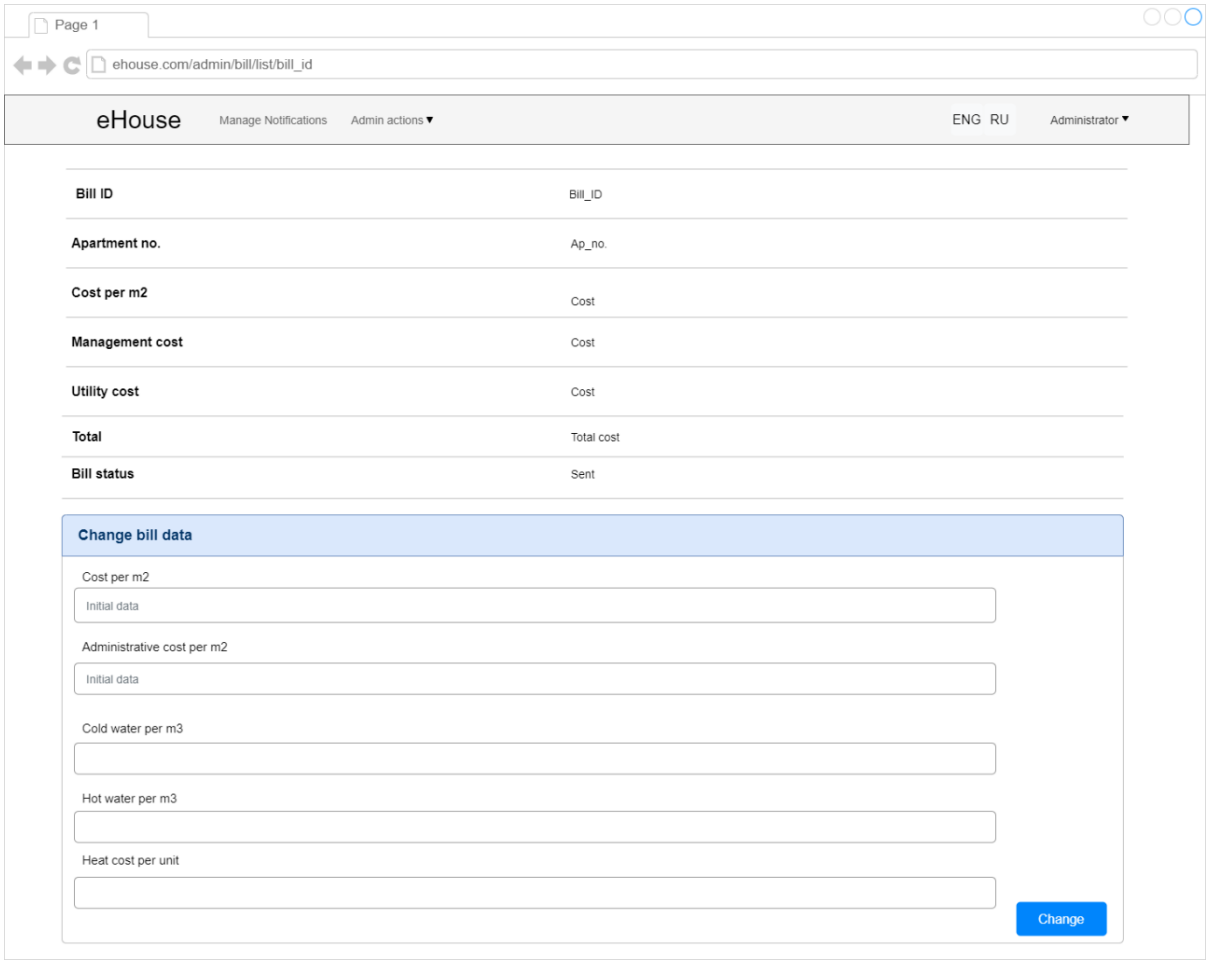
Pic. 42. Bills (Admin)

New bills can be created by admin (*pic. 43*) and proceeded by function in *section 2.2.7.*, *table 19.*



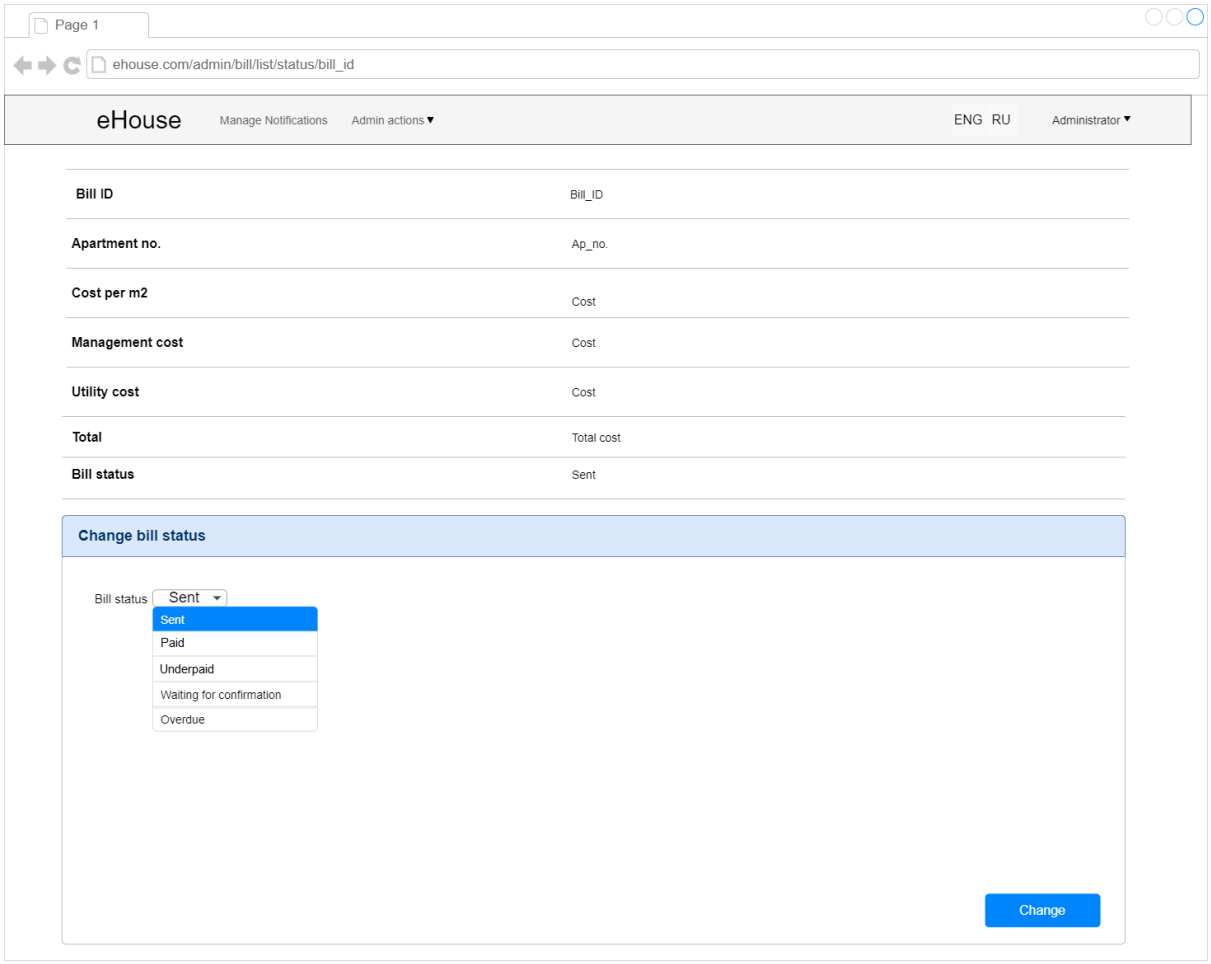
Pic. 43. New bill (Admin)

Admin can also edit already created bill (*pic. 44*). It's proceeded by function in *section 2.2.7., table 20.*



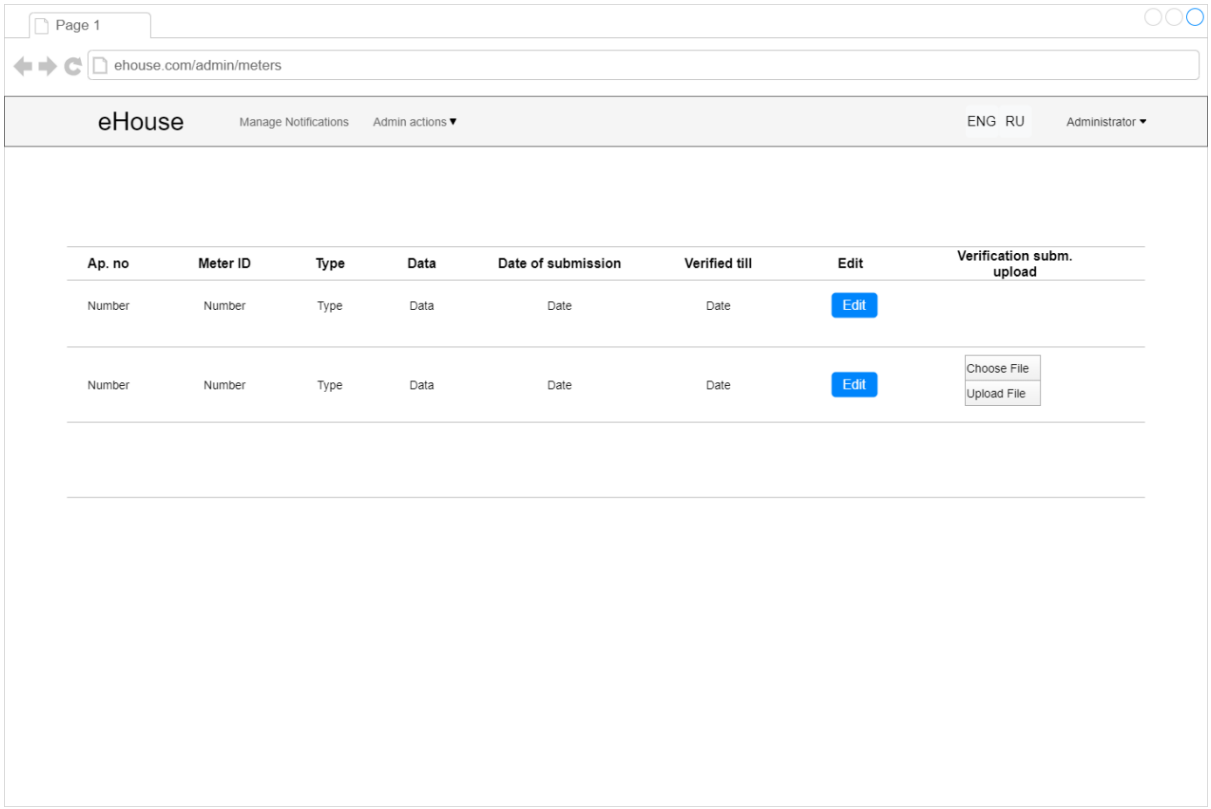
Pic. 44. Edit bill (Admin)

Admin can change status of the already created bill (*pic. 45*). It's proceeded by function in *section 2.2.7., table 26.*



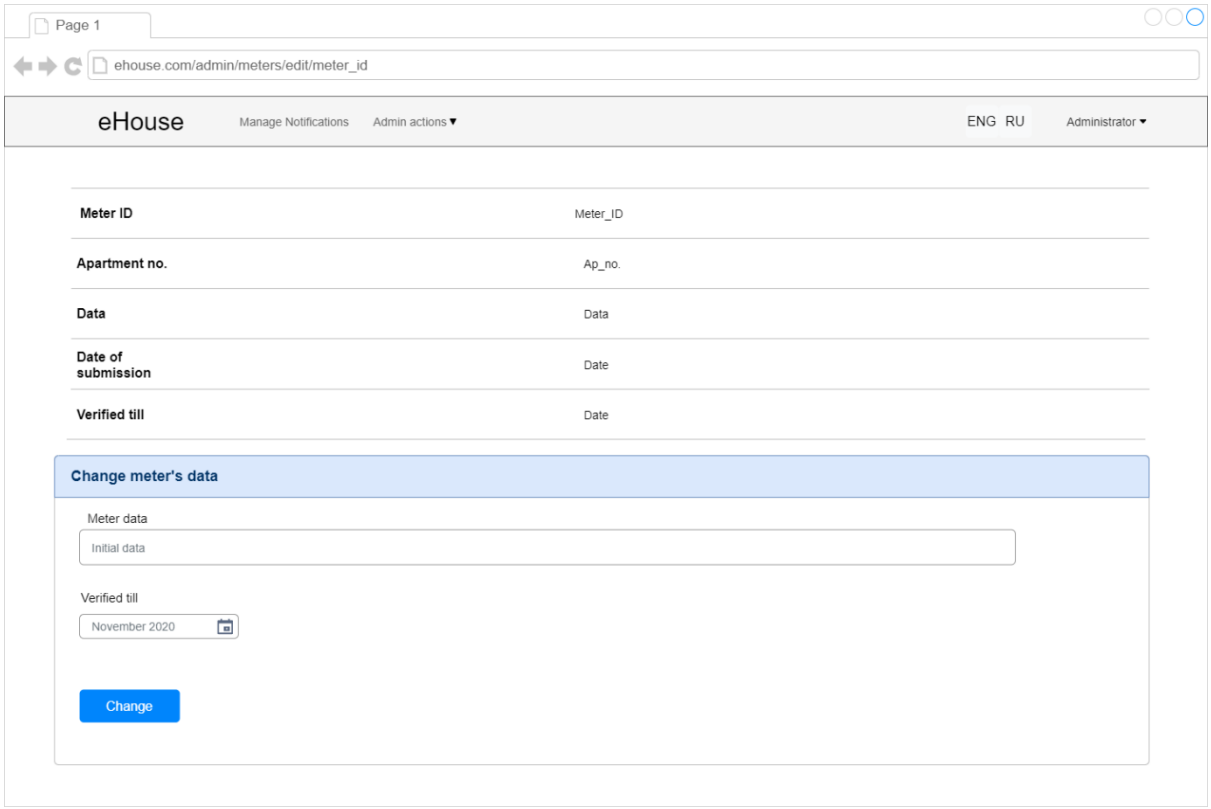
Pic. 45. Change bill status (Admin)

Admin can retrieve list of all meters and their data, it's shown on the *pic. 46* and processed by function in *section 2.2.6.*, *table 14.* Page also utilizes meters verification upload (*section 2.2.6.*, *table 16*).



Pic. 46. Meters (Admin)

Admin can edit meter's data, it's shown on the *pic. 47* and processed by function in *section 2.2.6., table 17.* Page also utilizes meters verification upload (*section 2.2.6., table 16*).

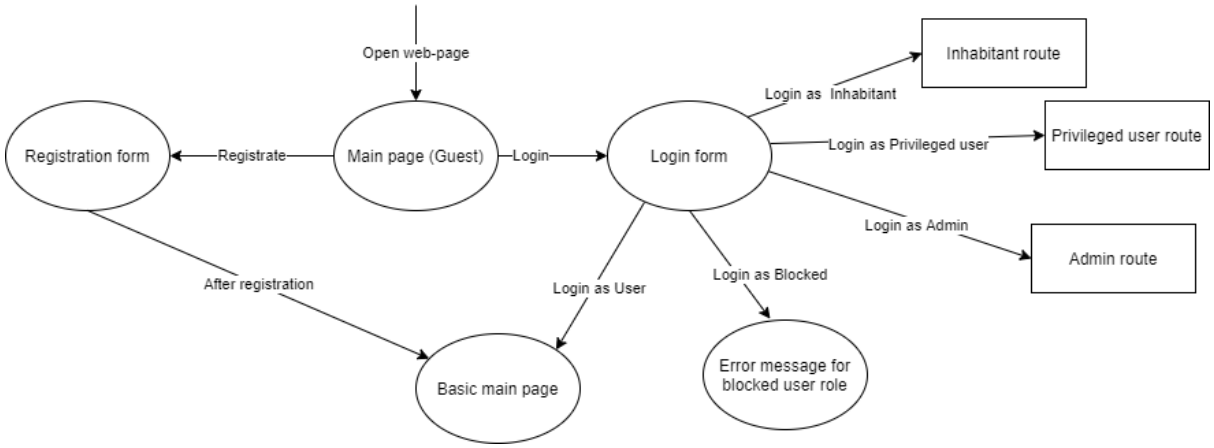


Pic. 47. Edit meter's data (Admin)

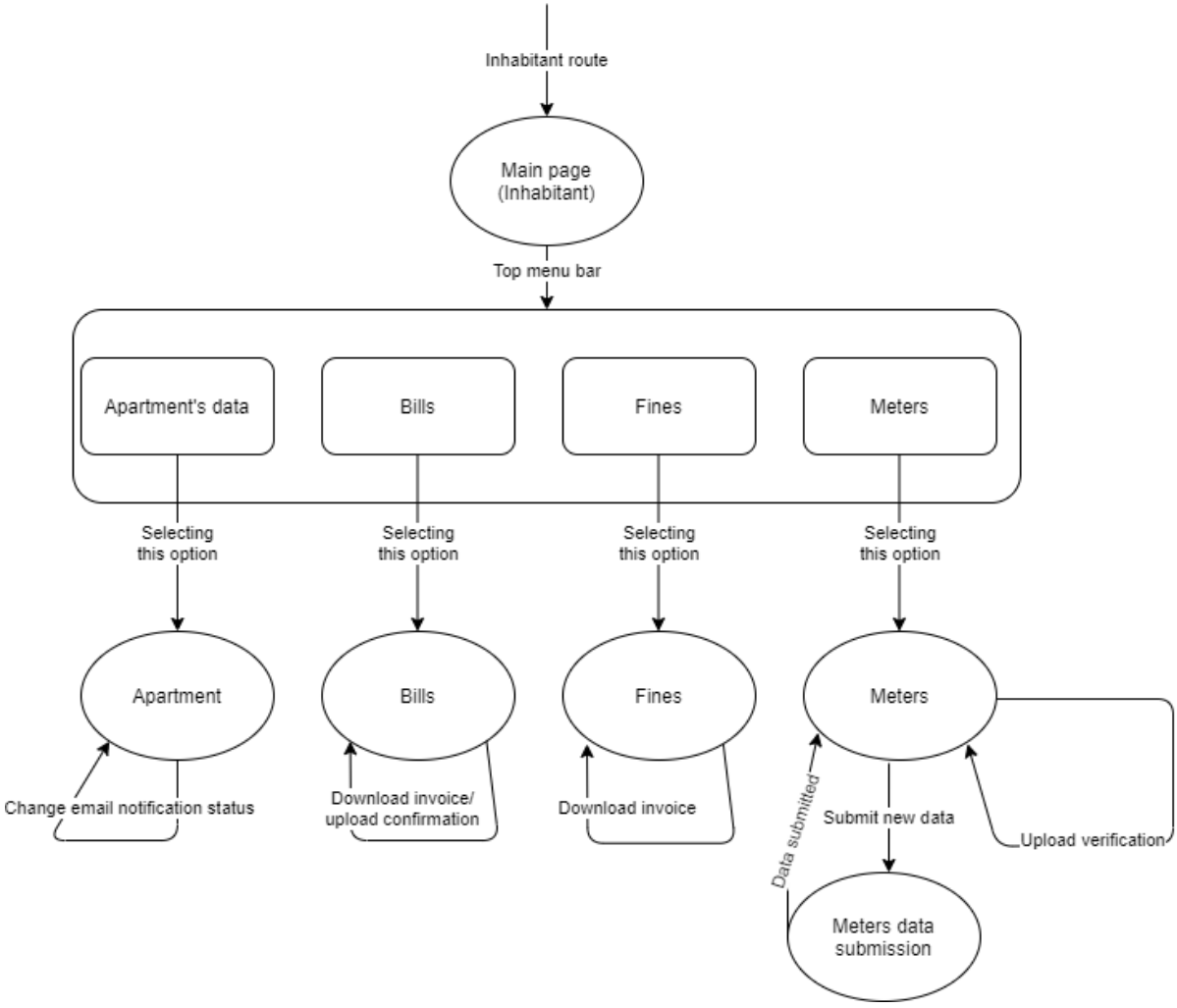
Based on described pages of web application user interface diagrams are created (*pics. 48-51*). Registration, login functions as well as results for guest, user and blocked roles are shown in *pic. 48*. In *pic. 49* shown Inhabitant route, in *pic. 50* – Privileged user route and in *pic. 51* – admin route. All menu bar functions are available via each page. Logout function is not shown at these diagrams due to simplification of the diagrams contents as it will always lead to the main page as the guest.

As the diagrams uses custom syntax the description of it shown below:

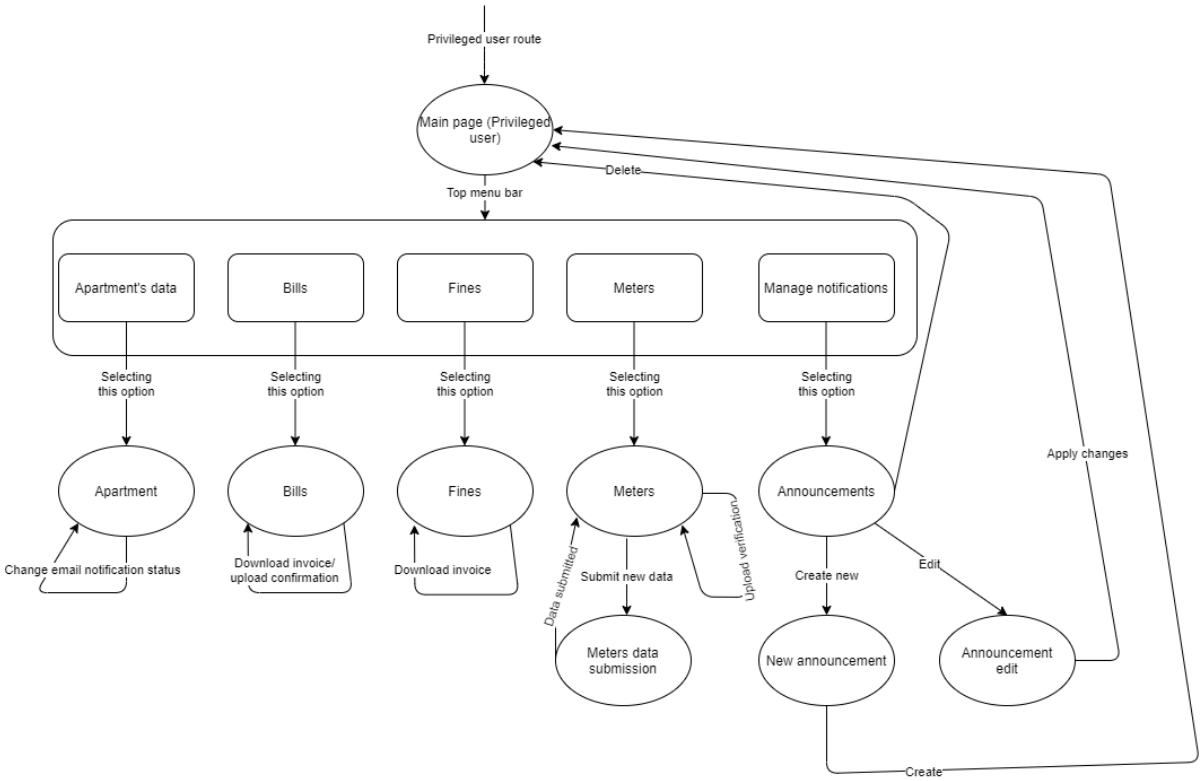
- *Oval* on the diagram represents a view (from the *section 3.2*).
- *Rectangular* on the diagram represents split-up to another diagram, showing a route explicitly.
- *Rectangular with rounded corners and its contents* represents top menu bar, which is accessible on any of views at the same diagram.
- *Arrow* represents action, that can be done in the particular view and it could either lead to another view or stay in the same view. All arrows are labeled with action.



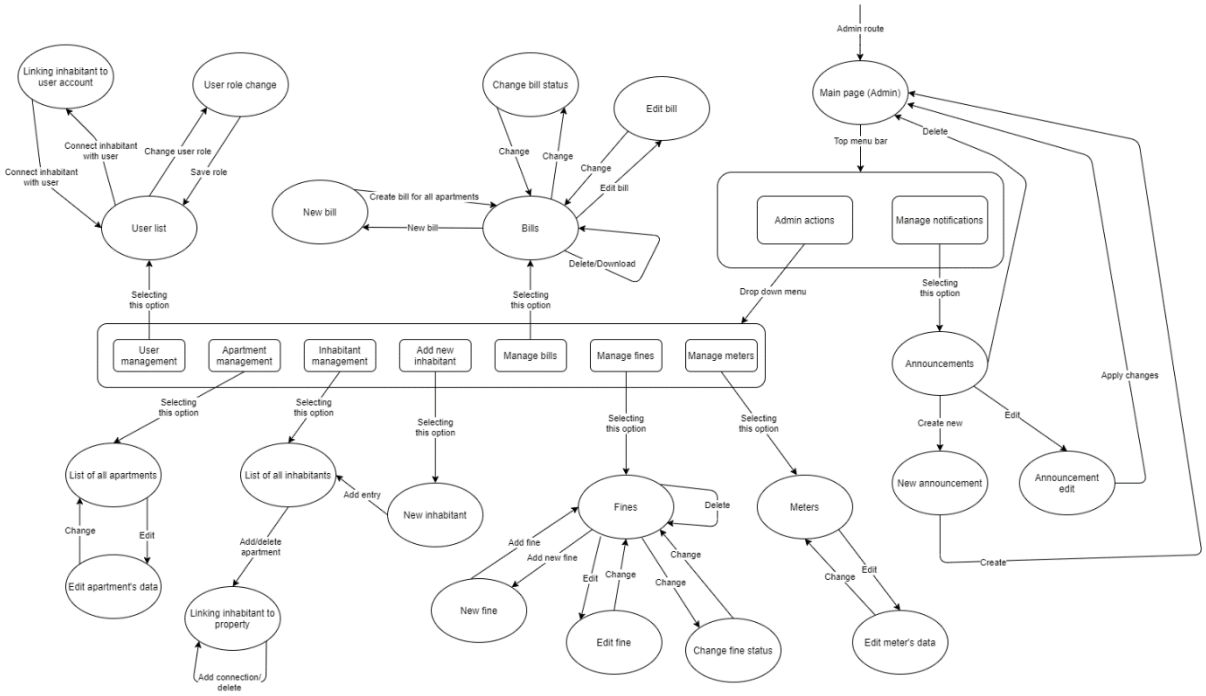
Pic. 48. User interface diagram (overall)



Pic. 49. User interface diagram (inhabitant route)



Pic. 50. User interface diagram (privileged user route)

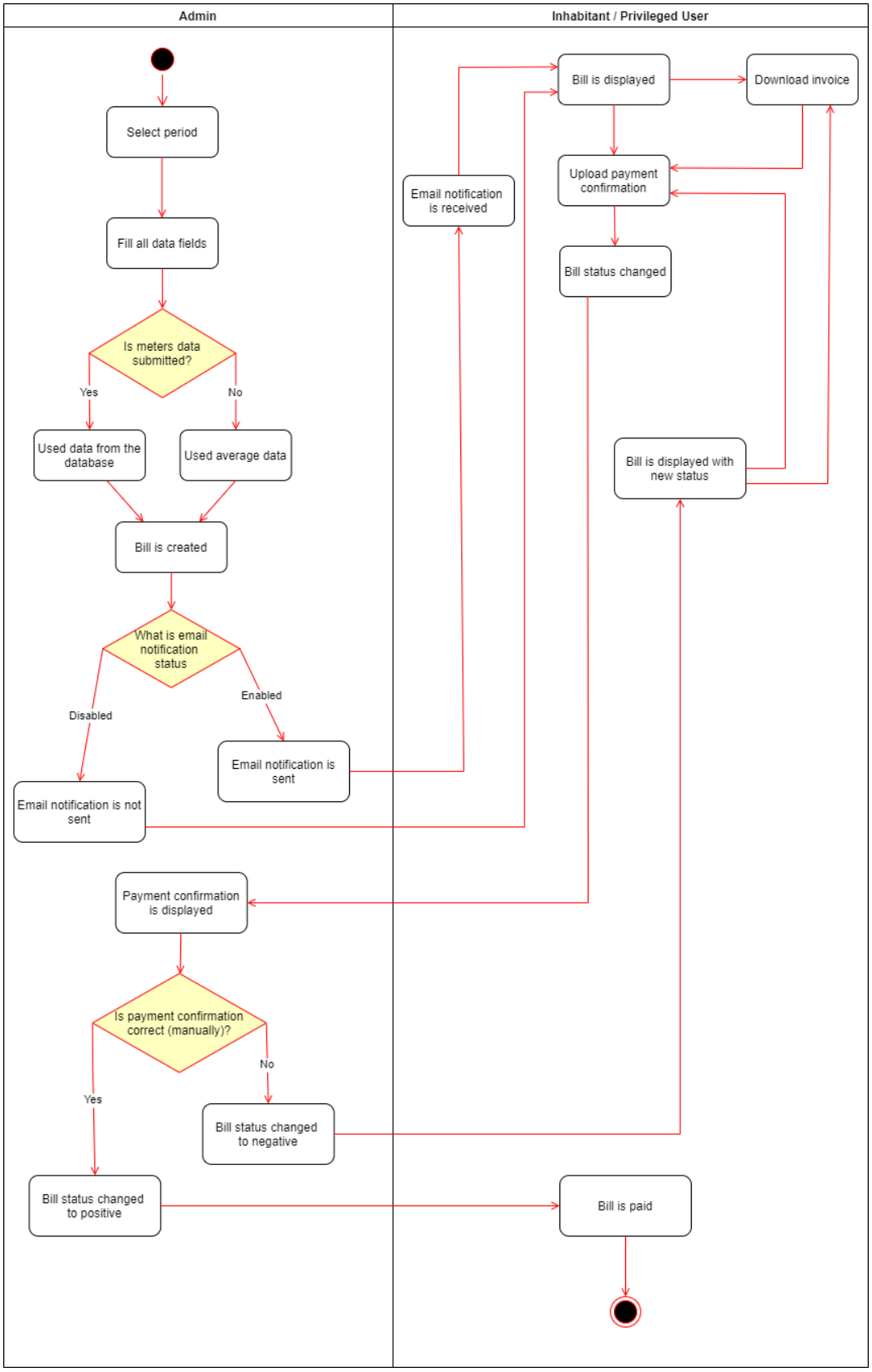


Pic. 51. User interface diagram (admin route)

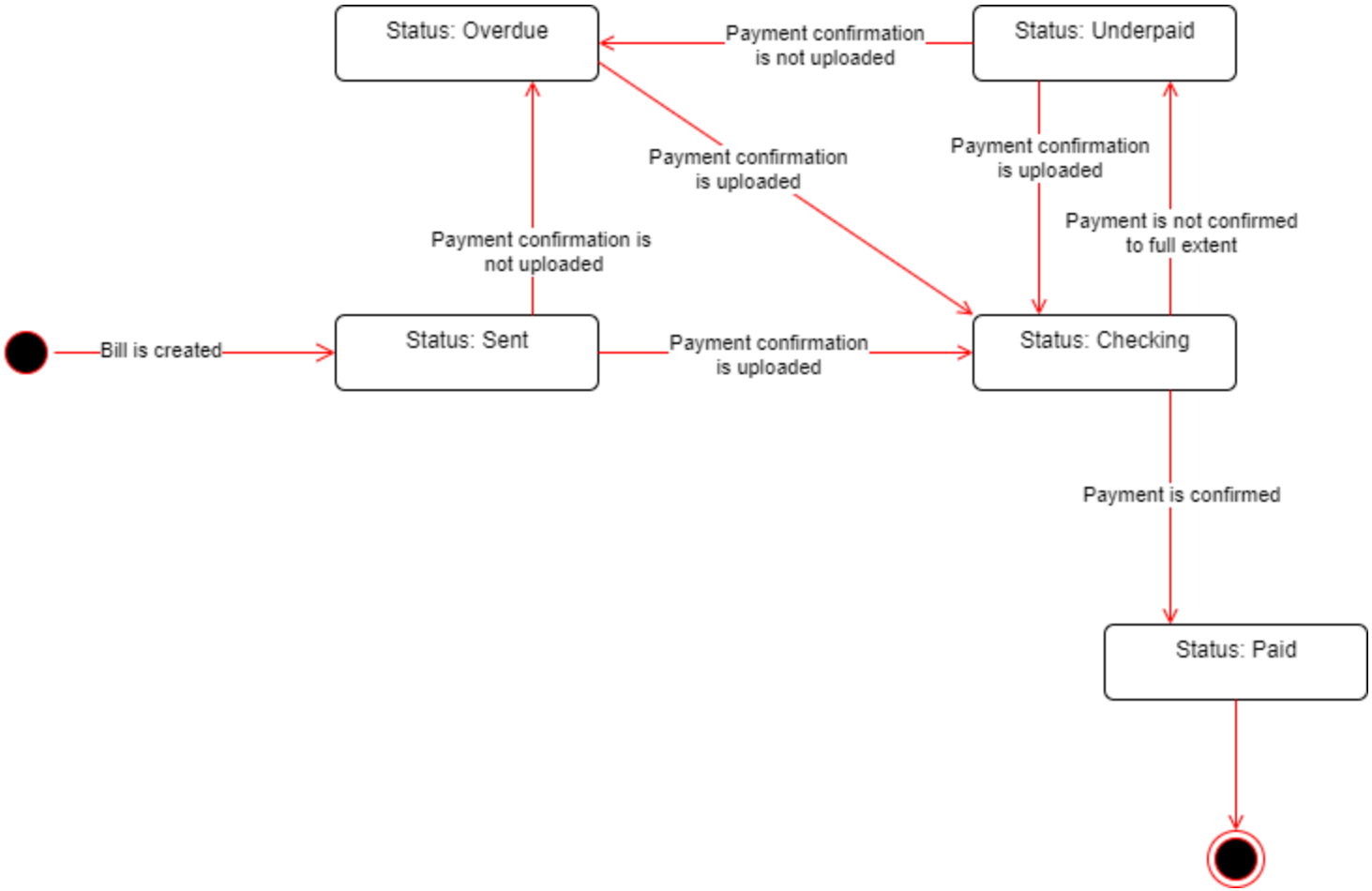
3.3. Partial Functional Design

The section contains diagrams, which describes the most important part of the functionality of the system.

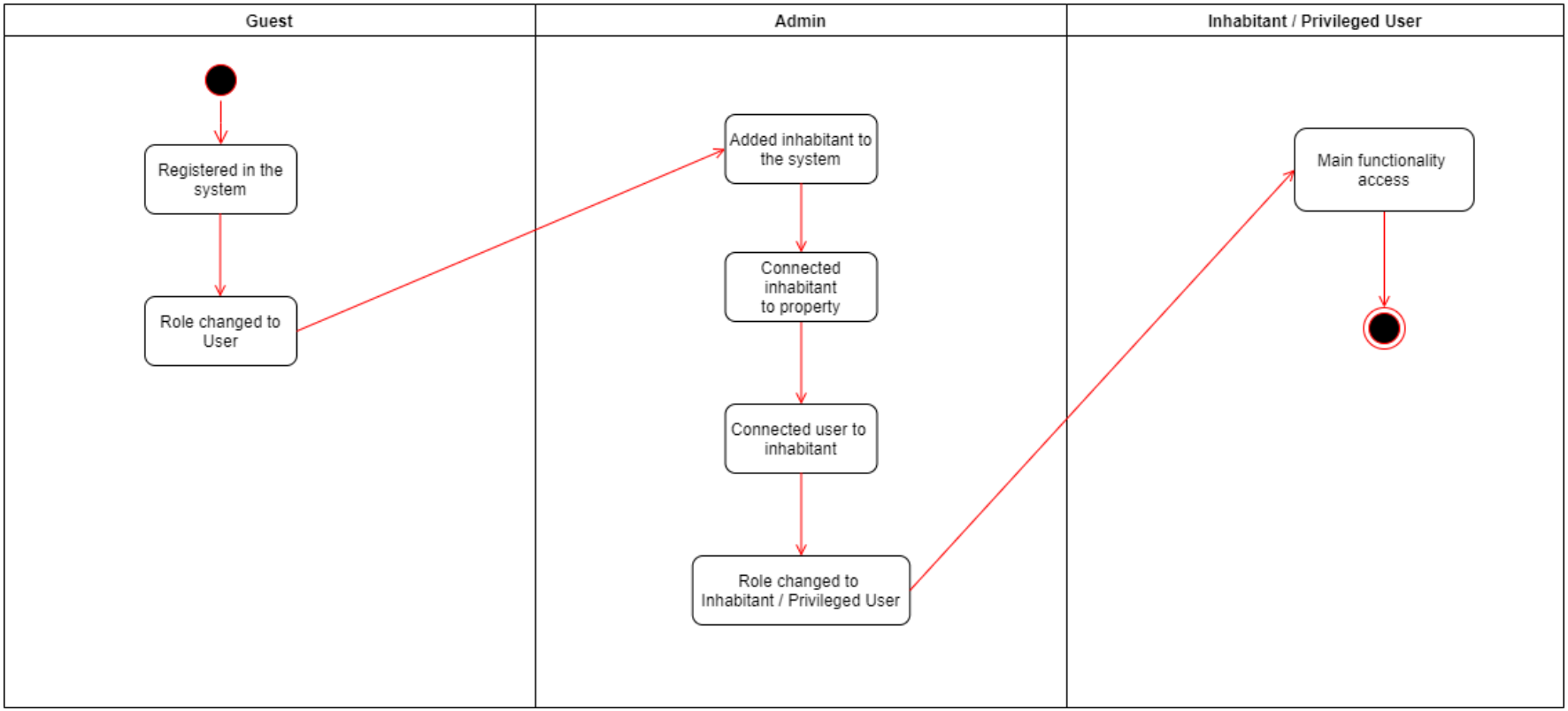
- The process of billing—from bill creation until it's marked as paid—is shown in the *pic. 52*. It's based on the function described in the *section 2.2.7.* (*tables 19, 22, 23, 24, 26*) and section 2.2.8. (*table 41*).
- Bills statuses transition (both manual and automatically) is shown in the *pic. 53.* Status checking is short version of Waiting for confirmation status shown in the *Appendix* (*table A6*) and *pic. 45* of GUI interface section. Mostly it's based on *table 26* of *section 2.2.7.*
- The process of configuring account for Inhabitant/Privileged user role is shown in the *pic. 54.* It's based on *section 2.2.3.* (*table 2*), *section 2.2.5* (*tables 9, 11, 12*).
- Class description of a structure in form of MVC approach is shown in the *pic. 55.* The centerpiece in the diagram is class Bill (controller) which uses various Model classes for querying the database. Even though the controller classes are shown as a hierarchy – practical implementation could be done in a sequential querying and calling each of controllers explicitly from the Bills (controller) class. By Bill controller is meant all functions shown *pic. 9* of *section 2.2.7.,* Model classes are corresponding database entities from *section 3.1.* Notification controller has all functions described in the *section 2.2.8.,* yet for the billing purposes the only relevant function is email notifications (*table 41*), which is shown without the rest of the functions in the *pic. 55.*
- The main parts of the system are shown in the *pic. 56.* It utilizes MVC approach, as well as shows possible necessary side packages and local storage. Inner framework files and configuration are not shown. It combines all sections described in the document: functional description (*sections 2.2.3.-2.2.8.*) as controllers, database structure (*section 3.1.*) in form of models (even though the database itself is not shown explicitly) and GUI, described in the *section 3.2.* as views.
- Process of login is shown in the *pic. 57.* It is based on function in *table 3, sections 2.2.3*
- Process of user registration is shown in the *pic. 58.* It is based on function in *table 2, sections 2.2.3*
- Pic. 58 contains fine functions, which based on functions shown in the *section 2.2.7. tables 27-33* and *section 2.2.8. table 41.* As both Inhabitant and Privileged User has the same functionality – they are aggregated as abstract Tenant representation for better visualization.



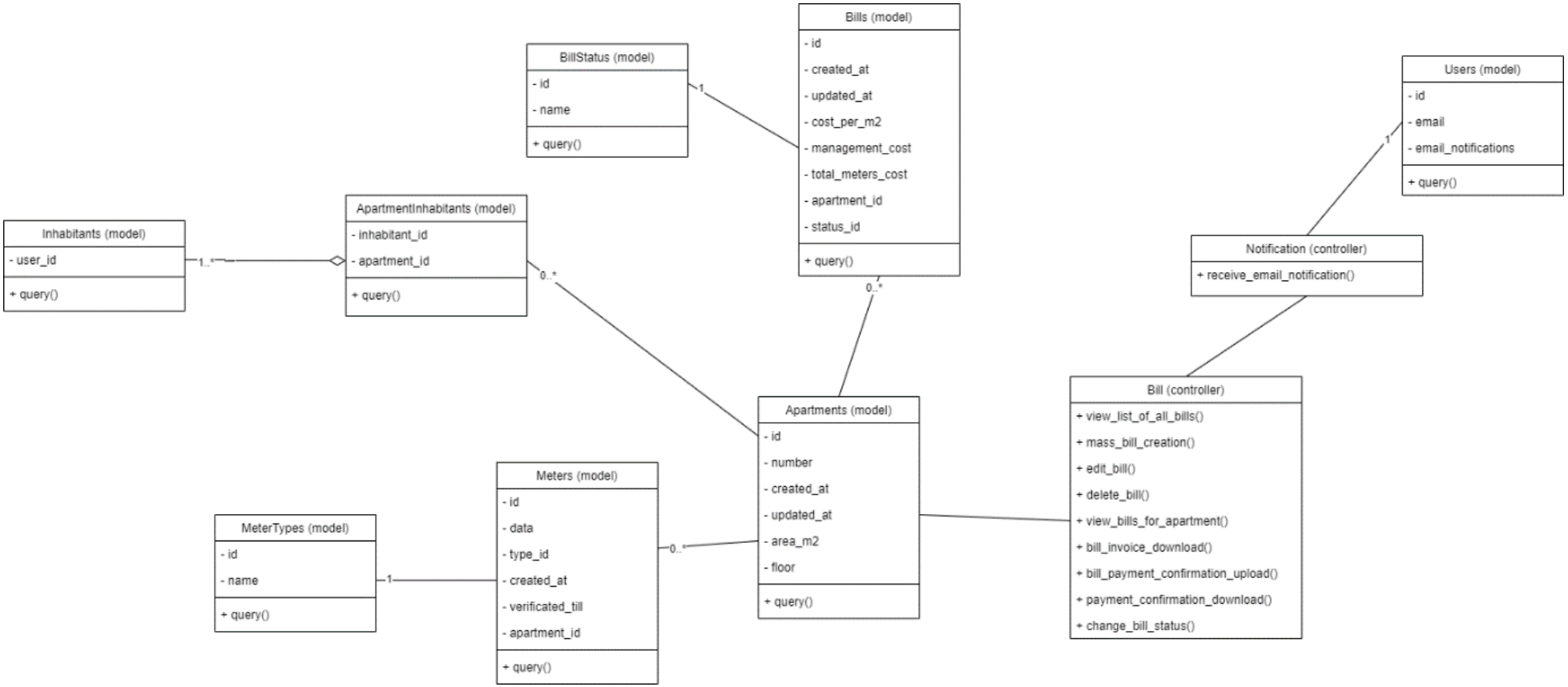
Pic. 52. Billing process (UML activity diagram)



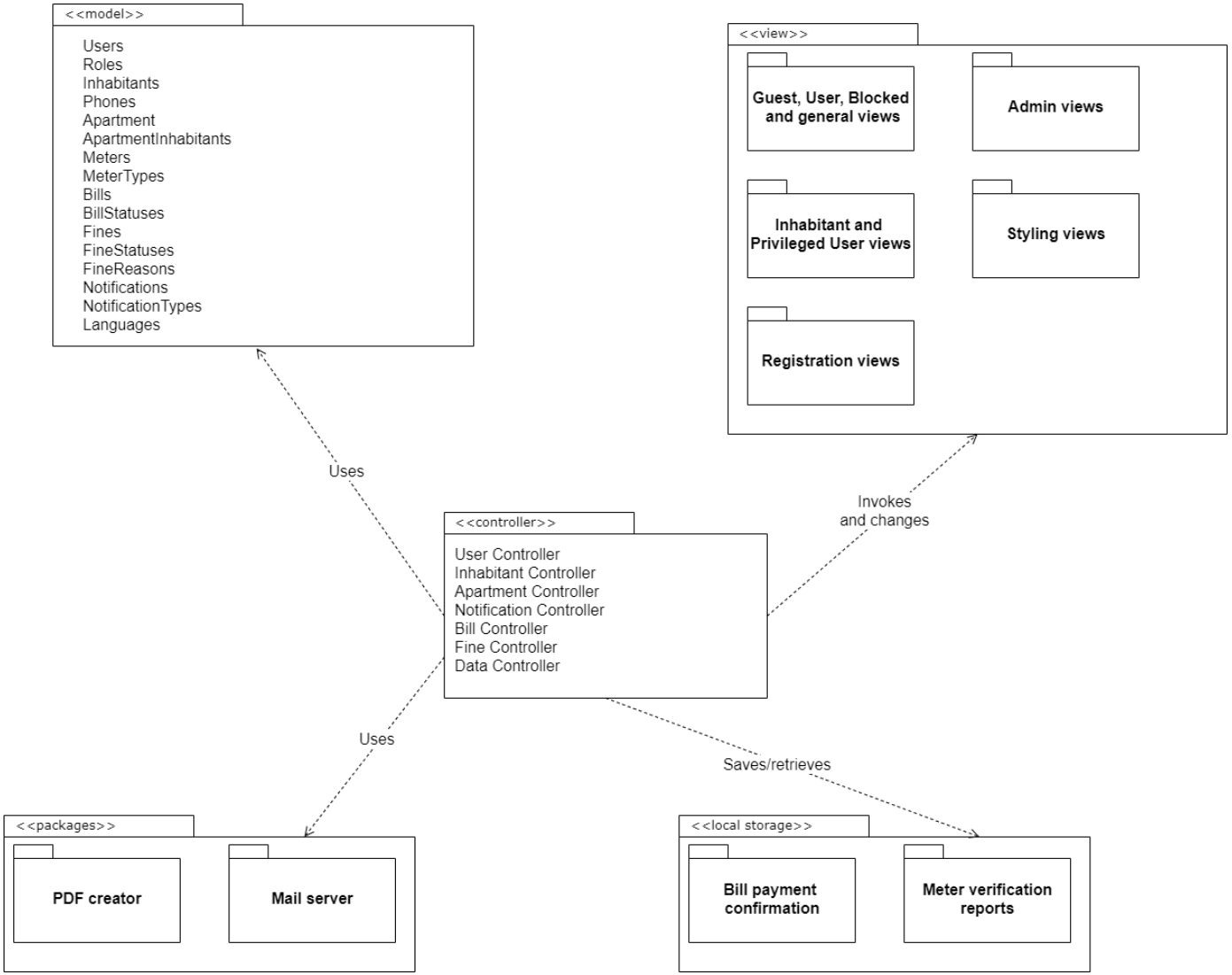
Pic. 53. Bill statuses (UML state transition diagram)



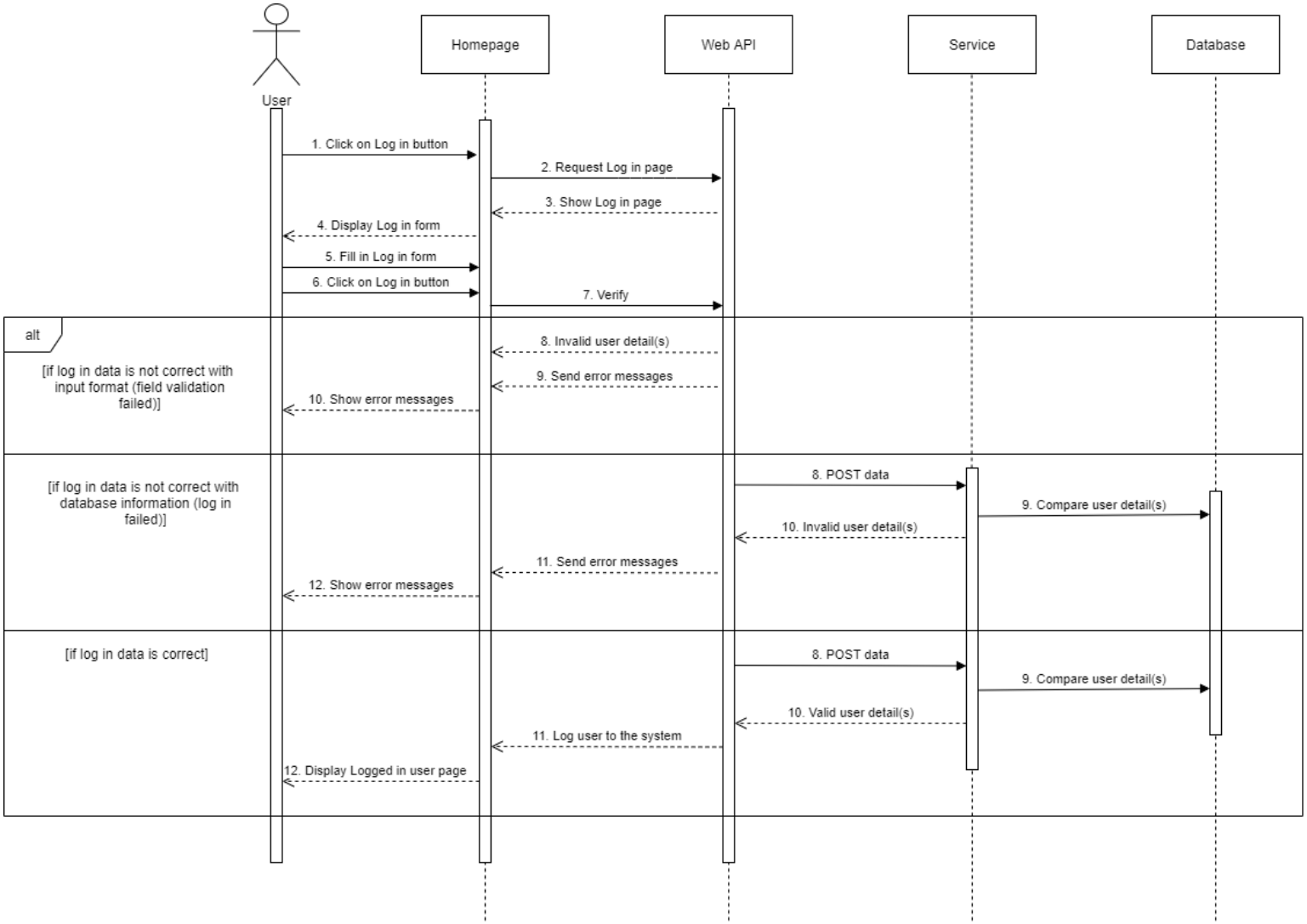
Pic. 54. Configuring working account (UML activity diagram)



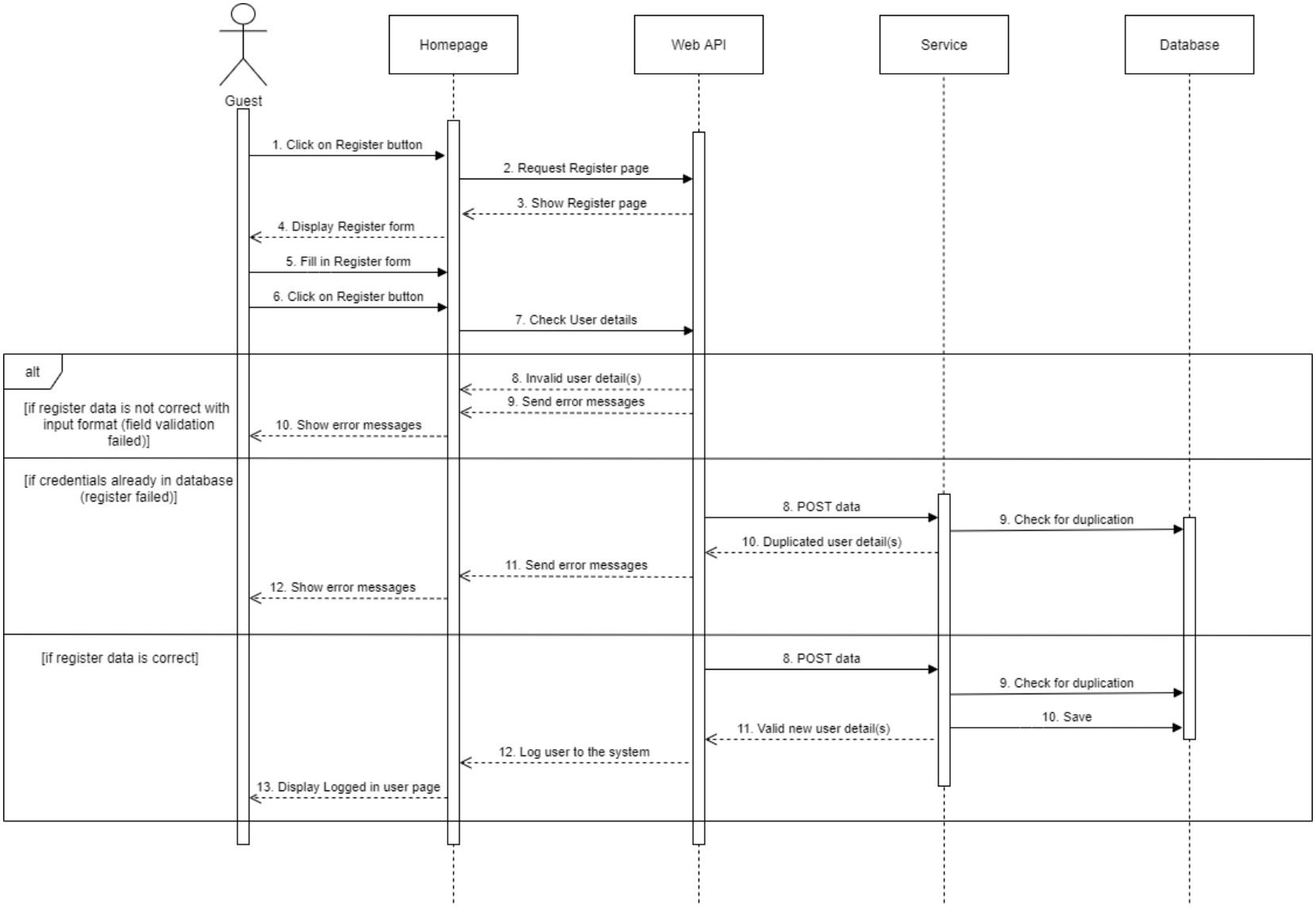
Pic. 55. Bill controller – MVC approach (UML class diagram)



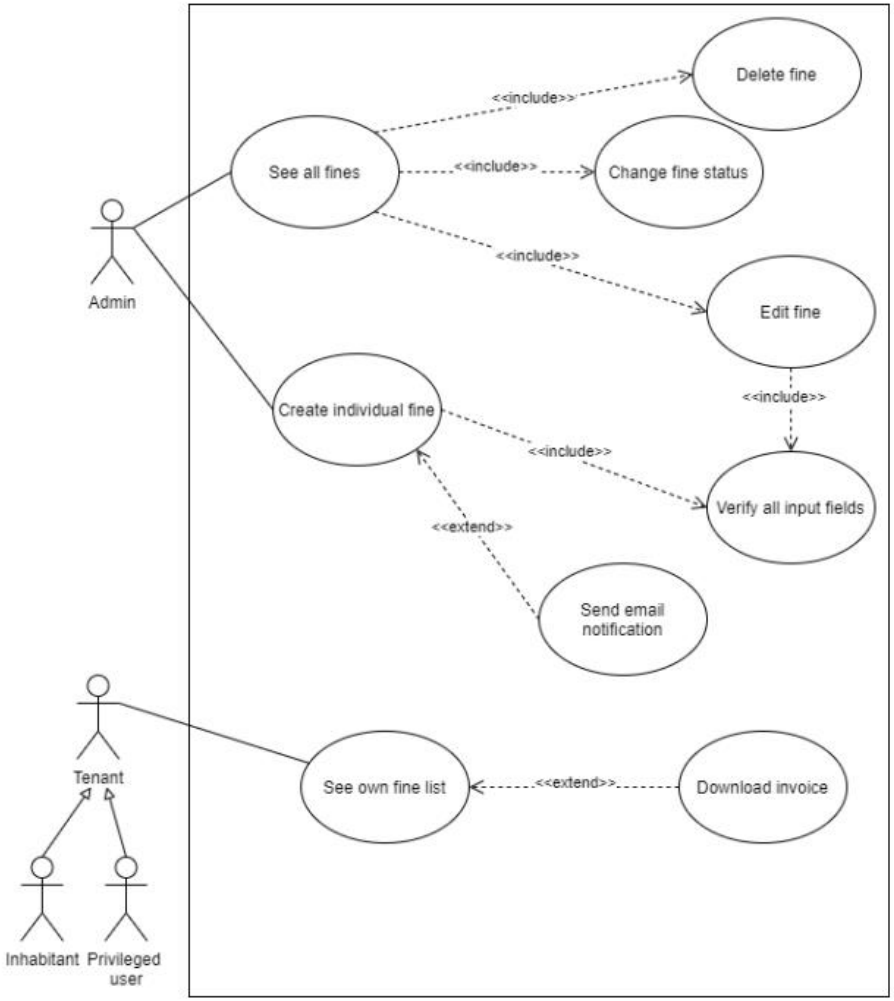
Pic. 56. Main parts of the system (UML package diagram)



Pic. 57. Log in process (UML sequence diagram)



Pic. 58. Register process (UML sequence diagram)



Pic. 59. Fines functions (UML use case diagram)

References and sources

1. “ISO International Standard - Codes for the representation of names of languages — Part 1: Alpha-2 code” in ISO 639-1:2002, pp. 1-37, Jul. 2002.
2. “ISO/IEC/IEEE International Standard - Systems and software engineering -- Life cycle processes -- Requirements engineering,” in ISO/IEC/IEEE 29148:2018(E), pp. 1-104, 30 Nov. 2018, DOI: 10.1109/IEEESTD.2018.8559686.

Appendix

The appendix contains possible initial database seeds examples for some entities described in the *section 3.1*. The name of the table is corresponding to the name of the database entity. *Tables A1-A7* contain various statuses seeding examples. *Tables A8* and *A9* contains examples of seeding large initial entities, such as Apartment and Meter.

Table A1. Roles seeding

id	**name**
1	User
2	Inhabitant
3	Privileged user
4	Admin
5	Blocked

Table A2. NotificationTypes seeding

id	**name**	**description**
1	Public	Any visitor will see the announcement
2	House	Only Inhabitant, Privileged user and Admin will see the announcement

Table A3. Languages seeding

id	**name**	**language code**
1	English	en
2	Russian	ru

Table A4. FineStatuses seeding

id	**name**	**description**
1	Issued	Fine with sum that is issued
2	Paid	Fine, that was paid by inhabitant

3	Warning	Fine without the sum to be paid
4	Cancelled	Fine that is cancelled due to some reason, but not deleted

Table A5. FineReasons seeding

id	**name**	**description**
1	Broken property	Broken property in house
2	Unconfirmed renovation	Renovation without notifying managing company
3	Dangerous items	Storing potentially dangerous items
4	Unappropriate behavior	Any kind of loud/drunk/etc. behavior
5	Being in restricted zones	Walking over lawn, etc.
6	Smoking	Smoking nearer 10m to the house

Table A6. BillStatuses seeding

id	**name**	**description**
1	Sent	Bill is sent to the inhabitant, yet no payment confirmation is received
2	Paid	Bill is paid and no more actions are needed
3	Underpaid	Bill is partially paid

4	Waiting for confirmation	Payment confirmation has been uploaded, but no decision made yet
5	Overdue	Bill is not paid in the time

Table A7. MeterTypes seeding

id	**name**
1	Cold water
2	Hot water
3	Heating

Table A8. Apartments seeding

id	**number**	**created_at**	**updated_at**	**area_m2**	**floor**
1	1	07/12/20	07/12/20	55.5	3
...					
75	75	07/12/20	07/12/20	84.9	5

Table A9. Meters seeding

id	**data**	**type_id**	**created_at**	**verificated_till**	**apartment_id**
1	0.00	1	07/12/20	07/12/21	1
...					
299	0.00	3	07/12/20	07/12/22	75